**Қарағанды жоғары политехникалық колледжі**
«Ақпараттық жүйелер және дизайн» кафедрасы

# КУРСТЫҚ ЖОБА

«Алгоритмдеу және бағдарламалау» пәні бойынша

**Тақырыбы:**

**«Көлеңке» — визуалды новелла, қозғалыс және жекпе-жек біріктірілген 2D ойыны**

Мамандығы: 06130100 — Бағдарламалық қамтамасыз ету (түрлері бойынша)

Орындаған: [аты-жөні]
Топ: [топ шифры]
Жетекші: [аты-жөні]
Қорғауға жіберілді: [күні]
Бағасы: [_____]

Қарағанды, 2026

# КУРСТЫҚ ЖОБАҒА ТАПСЫРМА

Студент: [аты-жөні], топ: [шифры]

**Жобаның тақырыбы:** «Көлеңке» — визуалды новелла, қозғалыс және жекпе-жек біріктірілген 2D ойыны.

**Бастапқы деректер:**

- Ойын мақсаты — жатақханадағы төрт таныстың көлеңкеден құтқару. Оқиға қыстағы үш түн ішінде өтеді; көлеңке адамды өлтірмейді, оның жадын жұтады: жадынан айырылған адам силуэтке айналып, өзінің күнделікті әдеттерімен соғады.
- Әр түн үш фазадан тұрады: сөйлесу → айналу → жекпе-жек. 1-түн (Дархан) — оқыту; 2-түн (Жанель және Мадина) — біреуімен ғана толық сөйлесуге болады; 3-түн (Бекзат) — финал, басты кейіпкердің өзі айнала бастайды. Аяқталу құтқарылғандар санына байланысты (төрт финал).
- Басқару: A/D немесе ← → — қозғалыс, E — әрекеттесу, Space — келесі реплика, 1–4 — жауап таңдау, Tab — журнал; жекпе-жекте J — жеңіл, K — ауыр соққы, L — блок (дәл сәтте — парирлеу), 1–3 — естелік, ESC — мәзір.
- Диалог жүйесі JSON файлдарынан жүктеледі (түйіндер мен таңдау бұтақтары); флагтар таңдауларды кейінгі сахналарға және жекпе-жек параметрлеріне байланыстырады.
- Жекпе-жек: кадрлық ақырлы автомат (FSM), hitbox/hurtbox, кадр дәлдігіндегі парирлеу терезесі, енгізу буфері. Бейімделгіш қарсылас ойыншы әрекеттерін жиілік кестесіне жазып (Марков тізбегі), келесі әрекетті болжайды; болжау дәлдігі өлшенеді.
- Графика мен дыбыс: силуэттік кадрлық анимация, бөлшектер жүйесі, парирлеудегі баяулау; буран, соққы, парирлеу және әр кейіпкердің мотиві (пышақ, барабан, тігін машинасы, метроном); 60 кадр/с.
- Технологиялар: Python, PyGame; сақтау — JSON; Windows үшін PyInstaller арқылы .exe жинау.

**Түсініктеме жазбаның мазмұны:** Кіріспе; 1 Зерттеу бөлімі (1.1 Пәндік аумақты сипаттау, 1.2 Бағдарламалау құралдарына шолу); 2 Технологиялық бөлім (2.1 Тапсырманың қойылымы, 2.2 Бағдарламалау тілін таңдау, 2.3 Құрылымдық және функционалдық сызба); 3 Арнайы бөлім (3.1 Модульдер мен функциялар, 3.2 Идентификаторлар кестесі, 3.3 Эксперименттік нәтижелер); Қорытынды; Қолданылған деректер тізімі; Қосымшалар (А — техникалық тапсырма, Ә — қолданушы басқаруы, Б — бағдарлама листингі, В — құрылымдық сызба, Г — функционалдық сызба).

**Графикалық бөлім:** құрылымдық сызба, функционалдық сызба (А4).

Тапсырма берілген күн: [күні] Орындау мерзімі: [күні]

Жетекші: [аты-жөні, қолы] Студент: [қолы]

## МАЗМҰНЫ

**КІРІСПЕ**.....4
**1 ЗЕРТТЕУ БӨЛІМІ**.....6
1.1 Пәндік аумақты сипаттау.....6
1.2 Бағдарламалау құралдарына шолу.....7
**2 ТЕХНОЛОГИЯЛЫҚ БӨЛІМ**.....9
2.1 Тапсырманың қойылымы.....9
2.2 Бағдарламалау тілін таңдау және оның негіздері.....10
2.3 Бағдарламаның құрылымдық және функционалдық сызбасы.....12
**3 АРНАЙЫ БӨЛІМ**.....14
3.1 Модульдер, процедуралар және функцияларды сипаттау.....14
3.2 Идентификаторлар кестесі.....20
3.3 Эксперименттік нәтижелер.....24
**ҚОРЫТЫНДЫ**.....28
**ҚОЛДАНЫЛҒАН ДЕРЕКТЕР ТІЗІМІ**.....30
**ҚОСЫМША А. ТЕХНИКАЛЫҚ ТАПСЫРМА**.....31
1 Кіріспе.....31
2 Дамудың негізі.....31
3 Тағайындау.....31
4 Бағдарламалық өнімге қойылатын талаптар.....31
5 Бағдарламалық құжаттамаға қойылатын талаптар.....32
**ҚОСЫМША Ә. ҚОЛДАНУШЫ БАСҚАРУЫ**.....33
1 Жүйені іске қосу тәртібі.....33
2 Басқару интерфейсі.....33
3 Жұмыс режимдерін таңдау және қауіпсіздік шаралары.....40
4 Қателерді жою бойынша кеңестер.....40
**ҚОСЫМША Б. БАҒДАРЛАМА ЛИСТИНГІ**.....42
**ҚОСЫМША В. ҚҰРЫЛЫМДЫҚ СЫЗБА**.....108
**ҚОСЫМША Г. ФУНКЦИОНАЛДЫҚ СЫЗБА**.....109

# КІРІСПЕ

Заманауи компьютерлік ойындар — бағдарламалық жасақтаманың ең күрделі салаларының бірі. Бір ойынның ішінде графика, дыбыс, пайдаланушы енгізуі, ойын күйін басқару, жасанды интеллект және нақты уақыттағы есептеулер бір мезгілде жұмыс істейді. Сондықтан ойын жасау студенттерге алгоритмдеу мен бағдарламалау ұғымдарын, модульдік құрылымды және тестілеуді практикада бекітуге мүмкіндік береді.

Курстық жобаның тақырыбы — «Көлеңке» визуалды новелла, қозғалыс және жекпе-жек біріктірілген 2D ойыны. Ойын оқиғасы қыстағы үш түн ішінде өтеді: буран жолды жауып тастайды, жатақханада жарық пен байланыс үзіледі. Көлеңке адамды өлтірмейді, оның жадын жұтады: жадынан айырылған адам силуэтке айналып, тек өзінің күнделікті әдеттерімен соғады. Ойыншының мақсаты — жатақханадағы төрт таныстың (Дархан, Жанель, Мадина, Бекзат) көлеңкеден құтқару.

Жобаның өзектілігі мен ерекшелігі бірнеше жағдайға байланысты. Біріншіден, бір бағдарламада үш түрлі ойын түрі (диалогқа негізделген новелла, жатақханада жүру және кадрлық жекпе-жек) біріктірілген, сондықтан олардың арасында нақты алгоритмдік байланыс қажет болды: диалог барысында ойыншы байқаған әрбір деталь флагқа жазылады, ал флаг жекпе-жекте қарсыластың дайындық қимылын экранда ашып, дәл уақытында парирлеуге мүмкіндік береді. Екіншіден, қарсылас ойыншының әрекеттерін жиілік кестесіне жазып, Марков тізбегі принципімен келесі әрекетті болжайды, ал болжау дәлдігі өлшеніп, жобаның эксперименттік нәтижесі ретінде беріледі. Үшіншіден, визуалды стиль сюжеттің тікелей бөлігі: кейіпкерлер қара силуэт және бір акцентті түс арқылы беріледі, көлеңке жұтқан сайын олардың детальдары жоғалады.

Жұмыстың мақсаты — Python тілінде PyGame кітапханасын пайдаланып, жоғарыда сипатталған ойынды толық жұмыс істейтін бағдарламалық өнім ретінде әзірлеу және оның алгоритмдік негіздерін негіздеу.

Мақсатқа жету үшін келесі міндеттер қойылды:

– ойын сценарийін түйіндер мен таңдау бұтақтарынан тұратын бағытталған граф түрінде JSON файлдарында сипаттап, оны орындайтын диалог жүйесін жасау;
– ойыншы таңдауларын жазатын флагтар жүйесі мен естеліктер журналын құру және оларды жекпе-жек параметрлерімен байланыстыру;

– жекпе-жекті кадрлық ақырлы автомат (FSM), hitbox/hurtbox, кадр дәлдігіндегі парирлеу терезесі және енгізу буфері негізінде іске асыру;
– ойыншының әдеттерін үйренетін бейімделгіш қарсыластың ИИ-ін жасап, болжау дәлдігін өлшейтін эксперимент жүргізу;
– силуэттік графиканы, жарықтандыруды, бөлшектер жүйесін және дыбысты бағдарламалау арқылы жүзеге асыру (сыртқы сурет немесе аудио файлдарсыз);
– үш түннен және төрт финалдан тұратын ойын циклін, сақтау жүйесін жасау;
– бағдарламаны автоматты тестілеп, PyInstaller арқылы Windows үшін орындалатын файлға жинау.

Жұмыстың практикалық маңызы: әзірленген бағдарлама толық ойын ретінде қолданыла алады, ал оның модульдік құрылымы мен ашық алгоритмдері (диалог графы, флагтар, кадрлық FSM, Марков болжамы) ұқсас жобаларды жасауға негіз бола алады.

# 1 ЗЕРТТЕУ БӨЛІМІ

## 1.1 Пәндік аумақты сипаттау

Компьютерлік ойындар индустриясы бүгінде кино және музыка салаларымен тең дәрежеде дамып отыр. Ойындар әртүрлі жанрларға бөлінеді, ал әр жанр өзіндік алгоритмдік шешімдерге сүйенеді. «Көлеңке» ойыны бірнеше жанрдың элементтерін біріктіретіндіктен, пәндік аумақты жанрлар бойынша қарастырған орынды.

Визуалды новелла — оқиға мәтін, суреттер мен ойыншы таңдауы арқылы баяндалатын жанр. Оның алгоритмдік негізі — диалог графы: графтың түйіні бір репликаны білдіреді, ал қабырғалары келесі репликаға немесе таңдау нұсқаларына апарады. Ойыншы таңдаулары флагтарға жазылып, кейінгі сахналарға әсер етеді. Мұндай жүйені жасау үшін деректер (сценарий) мен бағдарлама кодын бөлу маңызды: сценарийді бағдарламаны қайта жазбай-ақ өзгерту мүмкін болады. «Көлеңке» ойынында сценарий JSON файлдарында сақталады, ал флагтар мен естеліктер журналы ойын күйінің бөлігі болып табылады.

Жекпе-жек ойындарында басты ұғым — кадр деректері (frame data). Әрбір соққы үш фазаға бөлінеді: дайындық (startup), белсенді (active) және қалпына келу (recovery). Соққының зақымдайтын аймағы тек белсенді фазада болады, ал оның дәлдігі hitbox пен қорғалатын аймақ hurtbox тіктөртбұрыштарының қиылысуымен анықталады. Қорғанудың ерекше түрі — парирлеу: қорғану пернесі соққы тиер сәттің жанында, қысқа уақыт терезесінде басылса, шабуылдаушы тепе-теңдігін жоғалтады, ал қорғанушы артықшылық алады. Көптеген жекпе-жек және экшн ойындарында осындай механика бар. Ойыншының қимылы кадр дәлдігімен есептелуі керек, сондықтан логика бекітілген уақыт қадамымен (мысалы, секундына 60 рет) жаңартылады.

Ойын сезімі (game feel) — ойыншыға әрекеттің салмағын жеткізетін эффектілер жиынтығы. Оған соққы кезіндегі қысқа тоқтау (hitstop), экран дірілі, сәтті парирлеуде уақытты баяулату, бөлшектер мен жарық жарқылы кіреді. Бұл эффектілер ойынның логикасына әсер етпейді, бірақ ойыншының әсерін күшейтеді, сондықтан оларды жекпе-жек ядросынан бөлек, сызу деңгейінде іске асыру ыңғайлы.

Ойындағы жасанды интеллект — қарсылас мінез-құлқын басқаратын алгоритмдер жиынтығы. Қарапайым жағдайда қарсылас алдын ала жазылған сценарий бойынша әрекет етеді, бірақ ойыншы бұл заңдылықты тез үйреніп алады. Бейімделгіш қарсылас ойыншының әрекеттерін бақылап, статистика жинайды және келесі әрекетті болжайды. Бұл үшін Марков тізбегі қолданылады: келесі әрекеттің ықтималдығы тек соңғы бірнеше әрекетке

тәуелді деп есептеледі, ал ықтималдықтар жиілік кестесінен алынады. Мұндай тәсіл есептеу жағынан жеңіл және нақты уақытта жұмыс істейді, сондықтан ойындар үшін тиімді.

Визуалды стиль де ойынның мазмұнына қызмет ете алады. «Көлеңке» ойынында кейіпкерлер қара силуэт және бір акцентті түс түрінде беріледі. Көлеңке жұтқан сайын кейіпкердің бет белгілері, киім бөлшектері жоғалады, түстері солғындайды. Үшінші түнде басты кейіпкердің өзі айнала бастайды: экран қараяды, интерфейс элементтері біртіндеп жоғалады. Осылайша визуалды шешім сюжеттің тікелей бөлігіне айналады.

Дыбыс ойын атмосферасын жасауда маңызды рөл атқарады. Жобада дыбыс (буран, соққы, парирлеу) және әр кейіпкердің өз мотиві (пышақ, барабан, тігін машинасы, метроном) бағдарламалық синтез арқылы жасалады. Бұл сыртқы аудио файлдарға тәуелділікті жояды және бағдарламаны бір орындалатын файлға жинауды жеңілдетеді.

Қорытындылай айтқанда, пәндік аумақ бес негізгі міндеттен тұрады: сценарийді граф түрінде басқару, флагтар арқылы таңдау салдарын сақтау, жекпе-жекті кадрлық автомат түрінде есептеу, қарсыластың бейімделуін ұйымдастыру және осының бәрін 60 кадр/с жылдамдықпен көрсету.

### 1.2 Бағдарламалау құралдарына шолу

Ойын жасау үшін қолданылатын құралдарды екі топқа бөлуге болады: дайын ойын қозғалтқыштары (engine) және кітапханалар. Қозғалтқыштар сахна редакторын, физиканы, анимацияны және басқа модульдерді дайын күйінде ұсынады. Кітапханалар жалпы мақсаттағы бағдарламалау тілінің үстінен тек негізгі мүмкіндіктерді (терезе, графика, енгізу, дыбыс) береді, ал қалған архитектураны әзірлеуші өзі жасайды.

1.1-кестеде курстық жоба тақырыбына жақын құралдар салыстырылған.

1.1-кесте — Ойын жасау құралдарын салыстыру

| Құрал | Тілі | Түрі | Артықшылығы | Кемшілігі |
|---|---|---|---|---|
| Unity | C# | Қозғалтқыш | Күшті редактор, кең мүмкіндік | Ауыр орта, алгоритмдердің көбі жасырын, курс міндеттеріне артық |
| Godot | GDScript | Қозғалтқыш | Ашық бастапқы код, жеңіл | Жеке жекпе-жек FSM мен диалог жүйесін бәрібір жазу керек, қосымша тіл |
| Ren'Py | Python негізіндегі DSL | Новелла қозғалтқышы | Новеллаға өте қолайлы | Нақты уақыттағы жекпе-жекке бейімделмеген |
| LÖVE | Lua | Кітапхана | Жеңіл, қарапайым | Курс бағдарламасындағы тіл емес |
| Arcade | Python | Кітапхана | Заманауи, OpenGL негізінде | Аз таралған, кадр дәлдігіндегі 2D үшін ресурс аз |

| Құрал | Тілі | Түрі | Артықшылығы | Кемшілігі |
|---|---|---|---|---|
| PyGame | Python | Кітапхана (SDL) | Курс тілі, толық бақылау, кең құжаттама | Графика, UI, жарықтандыру қолмен жасалады |

Unity мен Godot кең мүмкіндік береді, бірақ олардың жұмыс принципі көбіне редактор мен дайын компоненттерге негізделген. Курстық жобаның мақсаты — алгоритмдерді өзі жазу және олардың жұмысын түсіндіру, сондықтан бұл құралдардың артықшылығы бұл жағдайда кемшілікке айналады: ойын логикасының бір бөлігі әзірлеушіден жасырын қалады. Ren'Py новелла жасауға өте қолайлы, бірақ біздің ойынымызда негізгі жүк кадрлық жекпе-жекке және бейімделгіш ИИ-ге түседі, ал бұл жанр үшін қозғалтқыш бейімделмеген.

PyGame кітапханасы Python тілінде жазылған және SDL кітапханасына сүйенеді. Ол терезені, оқиғаларды, суреттерді (Surface), қаріптерді, дыбысты және уақытты бақылауды қамтамасыз етеді. Қалғаны — ойын циклі, сахналарды басқару, диалог жүйесі, жекпе-жек есептеулері, жарықтандыру — әзірлеушінің міндеті. Бұл курстық жоба үшін артықшылық, өйткені әрбір алгоритмді студент өзі жазып, қорғау кезінде түсіндіре алады.

Таңдауды негіздейтін тағы бір себеп — «Алгоритмдеу және бағдарламалау» пәнінде Python тілі мен PyGame кітапханасын қолдану бойынша қойылатын талап, сондай-ақ жобаны Windows 10 жүйесіндегі қарапайым компьютерде іске қосу мүмкіндігі (Intel Core i3 процессоры, 4 ГБ жедел жады). PyGame кітапханасымен құрылған ойынды PyInstaller құралы арқылы бір орындалатын файлға жинауға болады, сондықтан ойынды Python орнатпаған компьютерде де көрсету оңай.

Қорытынды: «Көлеңке» ойынын әзірлеу үшін Python тілі мен PyGame кітапханасы таңдалды. Графиканы, жарықтандыруды және дыбысты кодпен жасау тәсілі қосымша құралдар мен сыртқы файлдарға тәуелділікті азайтады және жобаның барлық бөлігін студенттің өзі түсіндіре алатындай ашық етеді.

# 2 ТЕХНОЛОГИЯЛЫҚ БӨЛІМ

## 2.1 Тапсырманың қойылымы

Курстық жобада «Көлеңке» ойынының бағдарламасын әзірлеу тапсырылды. Бағдарлама ойыншыға жатақханадағы төрт таныстың көлеңкеден құтқару мүмкіндігін беруі керек. Ойын үш түннен тұрады, ал әр түн сөйлесу, айналу және жекпе-жек фазаларынан құралады. Төменде бағдарламаға қойылатын функционалдық талаптар келтірілген.

**Ойын сахнасы мен элементтері.** Басты элементтер — басты кейіпкер, төрт таныс және жатақхана локациялары (дәліз және үш бөлме). Кейіпкерлер қара силуэт пен бір акцентті түс арқылы беріледі: Дарханға — қызғылт сары, Жанельге — көгілдір-жасыл, Мадинаға — күлгін, Бекзатқа — алтын сары, басты кейіпкерге көк түс бекітілген. Көлеңке жұтқан сайын кейіпкер детальдары жоғалады: бет белгілері, киім бөлшектері, акцент жарқылы. Экранда түн индикаторы мен естеліктер журналының есептегіші, жекпе-жекте денсаулық жолағы, естелік жолағы және дайындық қимылының белгісі көрсетіледі.

**Түндердің құрылымы.** 1-түн (Дархан) — оқыту тарауы, қарсыластың дайындық қимылдары ұзақ. 2-түн (Жанель және Мадина) — екеуі бір түнде айналады, ал уақыт екеуіне жетпейді: ойыншы біреуімен ғана толық сөйлесіп, оның дайындық қимылдарын біледі, екіншісімен соқыр күйде шайқасады. 3-түн (Бекзат) — финал: басты кейіпкердің өзі айнала бастайды, экран қараяды, интерфейс жоғала бастайды. Ойынның аяқталуы құтқарылғандар санына байланысты төрт түрлі болады.

**Диалог және флагтар.** Диалог жүйесі JSON файлдарынан жүктеледі. Сценарий түйіндер мен таңдау бұтақтарынан тұратын бағытталған граф болып табылады. Таңдау нұсқасы флагтың мәніне байланысты көрінуі немесе жасырылуы мүмкін. Флагтар ойыншының таңдауларын сақтап, кейінгі сахналарға және жекпе-жек параметрлеріне (қарсыластың күші, дайындық белгілерінің көрінуі, естелік қоры) әсер етеді.

**Жекпе-жек.** Жекпе-жек кадрлық ақырлы автомат, hitbox/hurtbox, кадр дәлдігіндегі парирлеу терезесі және енгізу буфері негізінде құрылады. Ойыншы жеңіл және ауыр соққы жасайды, қорғанады, ал соққы тиер сәтте қорғану пернесін дәл басса, парирлеу іске асады. Қарсыласты өлтіруге де, естеліктер арқылы адами кейпіне қайтаруға да болады. Қайтару ойыншының өз естелігінің есебінен жүреді және келесі түндегі естелік қорын азайтады.

**Бейімделгіш қарсылас.** Қарсылас ойыншының әрекеттерін жиілік кестесіне жазып, Марков тізбегі принципімен келесі әрекетін болжайды. Болжау дәлдігі өлшенуі және жобаның эксперименттік нәтижесі ретінде ұсынылуы керек.

**Басқару.** Басқару пернелері 2.1-кестеде келтірілген.

2.1-кесте — Басқару пернелері

| **Режим** | **Перне** | **Әрекет** |
|---|---|---|
| Жүру | A/D немесе ← → | Қозғалыс |
| Жүру | E | Есікті ашу, кейіпкермен сөйлесу немесе шайқасты бастау |
| Диалог | Space | Келесі реплика (теру эффектісін аяқтау) |
| Диалог | 1–4 | Жауап нұсқасын таңдау |
| Барлық сахна | Tab | Естеліктер журналы |
| Жекпе-жек | J | Жеңіл соққы |
| Жекпе-жек | K | Ауыр соққы |
| Жекпе-жек | L | Блок; соққы тиер сәтте басылса — парирлеу |
| Жекпе-жек | 1–3 | Естелікті қолдану (жылу, зейін, қайтару) |
| Барлық сахна | ESC | Мәзір немесе артқа қайту |
| Жүйе | F11, M | Толық экран/терезе, дыбысты өшіру |

Функционалдық емес талаптар: ойын 60 кадр/с жылдамдықпен жаңарады, кадр уақыты 16,7 мс-тан аспауы тиіс; бағдарлама Windows 10 және одан жоғары жүйеде, Intel Core i3 процессорлы және 4 ГБ жедел жады бар компьютерде жұмыс істейді; сақтау JSON форматында жүргізіледі; тұрақты жұмыс істеуі және қате жағдайларын өңдеуі қамтамасыз етіледі (бүлінген сақтау файлы ойынды тоқтатпайды, аудио құрылғы жоқ болса ойын үнсіз жұмыс істей береді).

## 2.2 Бағдарламалау тілін таңдау және оның негіздері

Жобаны іске асыру үшін Python бағдарламалау тілі таңдалды. Python — жоғары деңгейлі, интерпретацияланатын, динамикалық типтелген тіл. Оның синтаксисі қысқа әрі оқуға жеңіл, сондықтан алгоритмдерді түсіндіру қарапайым. Тілдің стандартты кітапханасы JSON өңдеу (json модулі), жолдармен жұмыс, математикалық есептеулер (math, random), бинарлық деректермен жұмыс (array) және командалық жол аргументтері (argparse) сияқты қажетті мүмкіндіктерді қамтиды, сондықтан жобаға ауыр сыртқы тәуелділік қосу қажет болмады: орындалу кезінде жалғыз сыртқы кітапхана — PyGame.

**PyGame ортасының ерекшеліктері.** PyGame кітапханасы SDL негізінде жұмыс істейді және төмендегі объектілермен қамтамасыз етеді:

– display — терезе немесе толық экран; жобада pygame.SCALED жалаушасымен 1280×720 ішкі ажыратымдылық пайдаланылады, ол экранның нақты өлшеміне өздігінен масштабталады;

– Surface — кез келген кескін; сахнаның барлық бөлігі Surface түрінде сақталып, blit операциясымен біріктіріледі;
– event — пернелер мен терезе оқиғалары;
– time.Clock — кадр жиілігін шектеу; жобада Clock.tick(60) қолданылады;
– font — мәтін; қазақ әріптері (ә, ғ, қ, ң, ө, ұ, ү, h, i) бар DejaVu Sans қарпі пайдаланылады;
– mixer — дыбыс; дыбыс буфері бағдарламалық түрде есептелген үлгілерден жасалады.

**Бағдарламалау ортасының артықшылықтары.** Бірінші артықшылық — тест жазу мүмкіндігі. Ойын логикасы (граф, флагтар, жекпе-жек ядросы, ИИ) сызудан бөлек жазылған, сондықтан pytest арқылы терезесіз тексеріледі, ал сызу сахналарын тексеру үшін SDL_VIDEODRIVER=dummy режимі қолданылады. Жобада 261 автоматты тест бар. Екінші артықшылық — қайталанатын нәтиже: кездейсоқ сандар генераторы бастапқы мәнмен (seed) іске қосылады, сондықтан эксперименттің нәтижелері әр жолы бірдей болады. Үшінші артықшылық — өнімділікті өлшеу: кадр уақыты автоматты тестте өлшенеді.

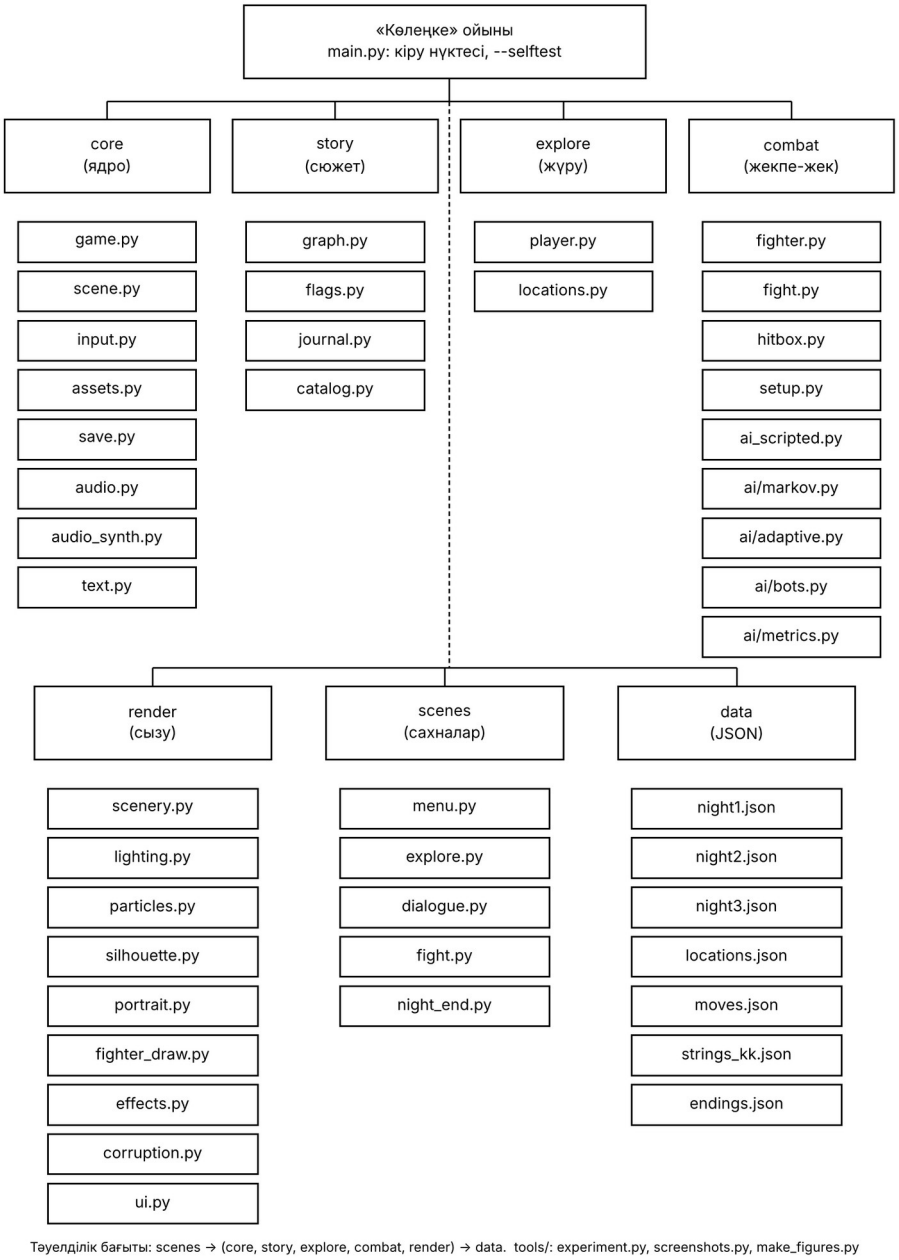
**Тәжірибелік аспектілер.** Бағдарламаны жасауда келесі принциптер сақталды.

– Логика мен сызуды бөлу. Жекпе-жек есептеуі бекітілген 1/60 с қадаммен жүреді (Fight.tick), ал сызу мен эффектілер сахнада жасалады. Бұл кадр дәлдігін қамтамасыз етеді және баяулау (slow-mo) мен hitstop сияқты эффектілерді логикаға әсер етпей қосуға мүмкіндік береді.
– Деректер мен кодты бөлу. Диалог мәтіндері, кейіпкерлердің соққы параметрлері (кадр деректері), локациялар мен интерфейс жолдары JSON файлдарында сақталады. Бұл балансты (мысалы, дайындық кадрларының санын) кодты өзгертпей-ақ реттеуге мүмкіндік береді.
– Ауыр есептеулерді алдын ала жасау. Локация фоны, портреттер және дыбыстар бір рет жасалып, кэште сақталады; әр кадрда тек динамикалық бөлік (жарық, бөлшектер, қозғалатын силуэттер) сызылады.
– Қате жағдайларын өңдеу. Сақтау файлы толық тексеріліп оқылады, бүлінген болса ойын күйі өзгермей, жаңа ойын ұсынылады. Аудио құрылғы жоқ болса, дыбыс жүйесі өшіп, ойын үнсіз жұмыс істейді.

**Жинау.** Windows үшін орындалатын файл PyInstaller құралымен жасалады. Қаріп пен JSON файлдары орындалатын файлдың ішіне қосылады, ал бағдарлама оларды уақытша қалтадан (sys._MEIPASS) оқиды. Жинау build_exe.bat файлы арқылы бір пәрменмен орындалады.

### 2.3 Бағдарламаның құрылымдық және функционалдық сызбасы

Бағдарлама жеті топқа бөлінген модульдерден тұрады. Құрылымдық сызба (В қосымшасы және 2.1-сурет) модульдер арасындағы бөліністі көрсетеді: core — ойын циклі мен жалпы қызметтер, story — диалог графы, флагтар және журнал, explore — жатақханада жүру, combat — жекпе-жек ядросы мен ИИ, render — графика мен эффектілер, scenes — сахналар, data — JSON деректері.




2.1-сурет — Бағдарламаның құрылымдық сызбасы

Модульдер арасындағы тәуелділік бір бағытта жүреді: сахналар (scenes) қалған модульдерді шақырады, ал жекпе-жек ядросы (combat) мен диалог жүйесі (story) графика мен PyGame терезесіне тәуелді емес. Бұл ядроны

терезесіз тестілеуге және эксперименттерді (tools/experiment.py) графиксіз жүргізуге мүмкіндік берді.

Сахналар стек түрінде басқарылады: ойын циклі әрқашан үстіңгі сахнаға оқиға береді және оны жаңартып, сызады. Мәзір сахнасы жатақхана сахнасын ашады, жатақхана сахнасы диалогты немесе жекпе-жекті ашады, ал ол жабылған кезде алдыңғы сахнаның on_resume әдісі шақырылады. Осы тәсіл (Game Programming Patterns кітабындағы күй стегі) сахналар арасындағы өтуді қарапайым әрі тексеруге ыңғайлы етеді.

Функционалдық сызба (Г қосымшасы) бағдарламаның жұмыс барысын көрсетеді. Ол мына тізбекті суреттейді: басты мәзір, сақтауды жүктеу немесе жаңа ойын, түнді бастау, жатақханада жүру, есік немесе кейіпкермен әрекеттесу, диалог, шайқасты дайындау, шайқас циклі, нәтиже, түн қорытындысы, автосақтау және финал. Шайқас циклінде енгізу пернелері буфер арқылы өңделеді, әр логикалық кадрда FSM күйлері жаңартылады, соққылар hitbox пен hurtbox қиылысуы бойынша тексеріледі және ИИ жиілік кестесі жаңарады. Диалог пен шайқас барысында флагтар мен журнал (оперативті жадта) оқылып, жазылады, ал түн соңында олар save.json файлына сақталады.

Бағдарламаның деректер ағыны мынадай. Диалог графындағы түйінге кіргенде оның set өрісі флагтарды, memory өрісі журналды, states өрісі кейіпкер портретінің күйін өзгертеді. Шайқас басталғанда fight_params функциясы флагтар мен журналды Fight конструкторының параметрлеріне айналдырады: қарсыластың қай қимылдарына дайындық белгісі көрінетінін (known_tells), естелік жолағының бастапқы мәнін, қарсыластың күшін және ойыншының осы кейіпкер туралы жазбаларының санын анықтайды. Шайқас аяқталғанда нәтиже (killed немесе saved) флагқа жазылып, естелік қоры өзгереді, ал түн қорытындысында құтқарылғандар саны финалды таңдайды.

# 3 АРНАЙЫ БӨЛІМ

## 3.1 Модульдер, процедуралар және функцияларды сипаттау

Бағдарлама жеті топқа бөлінген модульдерден тұрады. Әр модульдің міндеті мен өзара байланысы төменде сипатталады, ал негізгі функциялардың атауы, параметрлері және мақсаты кестелерде берілген.

**Ойынның басталу кезеңі.** Бағдарлама main.py файлынан басталады. Командалық жол аргументтері (argparse) оқылады: әдепкіде ойын толық экранда ашылады, --windowed терезеде, --selftest терезесіз тексеру режимінде іске қосады. Содан кейін Game класының объектісі жасалады: pygame.init() шақырылады, 1280×720 өлшемді терезе pygame.SCALED жалауымен ашылады (ол экранның нақты ажыратымдылығына масштабталады), Clock, шрифттер кэші, дыбыс менеджері, интерфейс жолдары (strings_kk.json), соққы кестелері (moves.json), ойын күйі (флагтар, журнал, естелік қоры, түн нөмірі) дайындалады. Соңында стекке MenuScene қосылып, Game.run() негізгі циклі іске қосылады.

**Кейіпкерлер мен нысандарды инициализациялау.** Жатақхана локациялары (дәліз және үш бөлме) locations.json файлынан жүктеліп, load_locations функциясымен тексеріледі: есіктің мақсатты локациясы, нысанның координаталары мен кейіпкердің диалог файлы бар-жоғы қаралады. Әр локацияның фоны, терезелері және жарықтары build_scenery функциясымен бір рет сызылып, кэште сақталады. Басты кейіпкер Player объектісі ретінде локацияның бастапқы нүктесіне қойылады. Шайқас басталғанда Fighter класының екі объектісі жасалады: moves.json файлындағы fighters бөлімінен денсаулық, жылдамдық, қимылдар тізімі және акцент түсі алынады, ал fight_params функциясы флагтар мен журналға қарап қарсылас параметрлерін өзгертеді.

**Негізгі цикл.** Game.run әр итерацияда Clock.tick(60) арқылы кадр уақытын алып, Game.step(dt) шақырады. Step әдісі ретімен төмендегі әрекеттерді орындайды: PyGame оқиғаларын оқиды (пернелер әрекет атына түрленеді, басулы әрекеттер held жиынына жазылады), үстіңгі сахнаның update әдісін шақырады, енгізу буферін жаңартады, сахнаны сызады және жаңа сахнаға өткендегі жұмсақ ашылу эффектісін қосады.

**Жекпе-жектегі логика.** Шайқас сахнасы (FightScene) нақты уақытты жинақтайды да, бекітілген 1/60 с қадаммен Fight.tick әдісін шақырады (бір update ішінде ең көбі төрт қадам). Бұл әдіс бір логикалық кадрда мынадай әрекеттерді орындайды: hitstop белсенді болса, уақыт қатады; әр жауынгердің контроллері (ойыншы үшін PlayerController, қарсылас үшін AdaptiveAI) бағытты, команданы және блоктың басулы екенін қайтарады; естелік командалары (1–3) өңделеді; Fighter.update күйлер автоматын бір кадрға

жылжытады; денелер бір-біріне өтіп кетпеуі үшін _separate шақырылады; _resolve функциясы белсенді фазадағы соққылардың hitbox аймағын қарсыластың hurtbox аймағымен қиылыстырады. Соққы тисе, нәтиже үш түрлі болады: қорғанушы блокты соққы тиер сәттен 8 кадрдан артық емес уақыт бұрын бастаса — парирлеу (шабуылдаушы тепе-теңдігін жоғалтады, қорғанушы 15 бірлік естелік алады), одан кейін бастаса — блок (зиян 15%), әйтпесе — соққы тию (зиян, итеріліс, hitstop). Парирленген қарсыласқа келесі соққы 1,5 есе зиян келтіреді. Денсаулық нөлге түссе, ko оқиғасы пайда болады және шайқас аяқталады.

**Қозғалыс пен соқтығысу.** Жатақханада Player.update жылдамдану мен тежелуді ескеріп, A/D немесе бағыт пернелері бойынша орынды жаңартады және локация шегінен шығармайды. Есіктер мен кейіпкерлерге жақындау қашықтықпен тексеріледі (interactables): нысан енінің жартысы мен 40 пиксельден аз қашықтықта E пернесі әрекетке қосылады. Жекпе-жекте соқтығысу тіктөртбұрыштардың қиылысуымен (hitbox және hurtbox) анықталады.

**Ұпай есебі орнына ойын күйі.** Бұл ойында классикалық ұпай жоқ, оның орнына ойынның үш сандық күйі жүргізіледі: денсаулық, естелік жолағы және естелік қоры. Естелік жолағы сәтті парирлеуде толады, ал үш слотты қолдануға жұмсалады (әрқайсысы 33 бірлік): «Жылу» денсаулықты 25-ке қалпына келтіреді, «Зейін» парирлеу терезесін 6 кадрға ұзартады, «Қайтару» адамды көлеңкеден қайтарады. Қайтару үшін ойыншы осы кейіпкер туралы кем дегенде бір жазбаны журналға жазған болуы және қарсыластың денсаулығы 35%-тен төмен немесе қарсылас тепе-теңдігін жоғалтқан болуы керек. Қайтарылған кейіпкер saved флагымен белгіленіп, естелік қоры 25-ке азаяды, ал келесі түнде жолақтың бастапқы мәні қордың жартысына тең болады. Өлтірілген кейіпкер killed флагын алады. Түн соңында saved флагтарының саны финалды таңдайды.

**Жеңіс және жеңіліс жағдайындағы хабарламалар.** Шайқас аяқталған соң экранда «Жеңіс» немесе «Адам қайтып келді» хабарламасы шығады, ойыншы жеңілсе «Жеңілдің» хабарламасы көрсетіледі және Space пернесімен шайқас қайта басталады. Түннің барлық шайқасы біткен соң «N-түн аяқталды» қорытынды экраны пайда болады: онда әр досының портреті (қайтқаны — адам кейпінде, қалғаны — көлеңке түрінде), күйі және естелік қоры көрсетіледі. Үшінші түннен кейін құтқарылғандар санына қарай төрт финалдың бірі көрсетіледі: 0 адам — «Мәңгілік түн», 1–2 адам — «Жарым таң», 3 адам — «Ащы таң», 4 адам — «Таңғы жарық».

**Бейімделгіш қарсылас.** AdaptiveAI класы ойыншының әр жаңа әрекетін (жеңіл соққы, ауыр соққы, блок) байқап, MarkovPredictor жиілік кестесіне жазады. Болжау екінші ретті (соңғы екі әрекет) контекст бойынша

жасалады, ал контексте дерек аз болса, қысқа контекстке (backoff) өтеді; ықтималдықтар Лаплас тегістеуімен есептеледі. Ойыншы соққы бастағанда қарсылас болжамға қарайды: болжам дұрыс және сенімділік шегінен жоғары болса, қорғану реакциясы 0–3 кадр ішінде жүреді (парирлеу терезесіне түседі), ал болжам қате болса, реакция 16–24 кадр кешігеді. «Блок» болжанса, қарсылас фінт жасайды: қимылды бастап, дайындықтың 55%-ында тоқтатады және ойыншының блоктан кейінгі кулдауынан кейін шабуылдайды. Парирленген ойыншыға қарсылас дереу жауап қайтарады. Сенім шегі түндерге байланысты 0,70 (1-түн, оқыту), 0,55 және 0,45 болып төмендейді.

Кестелерде модульдер бойынша негізгі функциялар мен әдістер берілген.

core пакеті ойын циклін, сахналар стегін, енгізуді, ресурстарды, сақтауды және дыбысты қамтамасыз етеді.

3.1-кесте — core пакетінің негізгі функциялары мен әдістері

| **Модуль** | **Функция / әдіс** | **Параметрлері** | **Мақсаты** |
|---|---|---|---|
| game.py | Game.__init__ | fullscreen | Терезені, шрифттерді, дыбысты, сахналар стегін және ойын күйін дайындайды |
| game.py | Game.start_night | n | Түнді бастайды: түн нөмірін қояды, үшінші түнде бүліну деңгейін 0,25 етіп орнатады |
| game.py | Game.tick_corruption | dt | 3-түнде уақыт өткен сайын басты кейіпкердің бүліну деңгейін арттырады |
| game.py | Game.new_game_state | — | Жаңа ойын күйін дайындайды: флагтар, журнал, естелік қоры = 100 |
| game.py | Game.push | scene | Сахнаны стекке қосып, жұмсақ ашылу эффектісін бастайды |
| game.py | Game.pop | — | Үстіңгі сахнаны алып тастап, астындағы сахнаның on_resume әдісін шақырады |
| game.py | Game.pop_to_menu | — | Стектегі барлық сахнаны жауып, мәзірге оралады |
| game.py | Game.is_held | action | Перне қазір басулы ма (үздіксіз қозғалыс үшін) |
| game.py | Game.process_events | — | PyGame оқиғаларын әрекеттерге айналдырып, үстіңгі сахнаға береді |
| game.py | Game.step | dt | Бір кадр: оқиғалар, логика, сызу және ашылу эффектісі |
| game.py | Game.run | — | Негізгі цикл: Clock.tick(60) және step шақыру |
| input.py | action_for_key | key | Перне кодын әрекет атына түрлендіреді (KEYMAP кестесі бойынша) |
| input.py | InputBuffer.push | action | Әрекетті енгізу буферіне жазады |
| input.py | InputBuffer.tick | — | Кадр санағышын арттырып, терезеден шыққан әрекеттерді өшіреді |

| Модуль | Функция / әдіс | Параметрлері | Мақсаты |
|---|---|---|---|
| input.py | InputBuffer.consume | action | Буферде бар әрекетті алып тастап, True қайтарады |
| save.py | save_game | game, path | Ойын күйін JSON файлына атомдық түрде жазады |
| save.py | load_game | game, path | Сақтауды тексеріп оқиды; бүлінген болса False қайтарады, күй өзгермейді |
| save.py | has_save | path | Сақтау файлының бар-жоғын тексереді |
| save.py | delete_save | path | Финалдан кейін сақтауды жояды |
| audio.py | Audio.play | name, volume | Синтезделген дыбысты ойнатады (кэштен) |
| audio.py | Audio.loop | name, volume | Фондық дыбысты (боран) үзіліссіз қайталайды |
| audio.py | Audio.toggle_mute | — | Дыбысты өшіреді немесе қосады (M пернесі) |
| audio.py | Audio.motif_for | name, move | Кейіпкердің мотивін қайтарады: пышақ, барабан, тігін, метроном |

story пакеті диалог графын, флагтар жүйесін және естеліктер журналын іске асырады.

3.2-кесте — story пакетінің негізгі функциялары мен әдістері

| Модуль | Функция / әдіс | Параметрлері | Мақсаты |
|---|---|---|---|
| graph.py | DialogueGraph.validate | — | Графты тексереді: бастапқы түйін, өтулер, естеліктер, портрет күйлері, қол жетпейтін түйіндер |
| graph.py | DialogueGraph.reachable | — | Бастапқы түйіннен қол жететін түйіндер жиынын қайтарады |
| graph.py | DialogueRunner.advance | — | Таңдауы жоқ түйіннен келесіге өтеді |
| graph.py | DialogueRunner.choose | visible_index | Таңдауды орындайды: флагтар мен журнал жаңарады, келесі түйінге өтеді |
| graph.py | DialogueRunner.choices | — | Флаг шарттарына сай көрінетін таңдаулар тізімін қайтарады |
| flags.py | Flags.set | name, value | Флагқа мән жазады |
| flags.py | Flags.get | name, default | Флагтың мәнін қайтарады (жоқ болса default) |
| flags.py | Flags.check | requires | Барлық шарттың орындалуын тексереді ({флаг: мән}) |
| journal.py | Journal.add | memory_id | Естеліктер журналына жазба қосады (қайталанбайды) |
| journal.py | Journal.entries | — | Табылған жазбалар тізімін қайтарады |

explore пакеті жатақханада жүру логикасын (басты кейіпкер қозғалысы және локациялар) қамтиды.

3.3-кесте — explore пакетінің негізгі функциялары мен әдістері

| Модуль | Функция / әдіс | Параметрлері | Мақсаты |
|---|---|---|---|
| player.py | Player.update | dt, left, right, width | Жылдамдану, тежелу және локация шегін ескеріп қозғалысты жаңартады |
| locations.py | load_locations | path | Локацияларды JSON-нан жүктеп, тексереді |
| locations.py | Obj.active | night | Нысан осы түнде белсенді ме |
| locations.py | Obj.enemies | flags | Кейіпкермен өтетін шайқастар реті (диалогтан кейін анықталады) |

combat пакеті жекпе-жектің графиксіз ядросын (FSM, hitbox, парирлеу, естеліктер) және бейімделгіш ИИ-ді қамтиды.

3.4-кесте — combat пакетінің негізгі функциялары мен әдістері

| Модуль | Функция / әдіс | Параметрлері | Мақсаты |
|---|---|---|---|
| fighter.py | Fighter.update | walk, cmd, hold_block, opp_x | Бір логикалық кадр: FSM күйін, қозғалыс пен буферден келген команданы өңдейді |
| fighter.py | Fighter.take_hit | damage, hitstun, knock_dir, knock | Соққы алады: денсаулық, қатып қалу, итеріліс |
| fighter.py | Fighter.take_block | damage, knock_dir, knock | Блокталған соққы: аз зиян және блоктан шығуды кідірту |
| fighter.py | Fighter.get_staggered | frames | Парирленген жауды тепе-теңдігінен айырады |
| hitbox.py | hitbox | x, facing, move | Соққының зақымдау аймағын (тіктөртбұрыш) есептейді |
| hitbox.py | hurtbox | x | Кейіпкердің қорғалатын аймағын қайтарады |
| fight.py | Fight.tick | — | Бір логикалық кадр (1/60 с): енгізу, FSM, соққылар, hitstop, оқиғалар тізімі |
| fight.py | Fight._resolve | atk, dfn | Соққыны тексереді: парирлеу, блок немесе тию; Event қайтарады |
| fight.py | Fight.usable | slot | Естелік слотын қазір қолдануға бола ма |
| fight.py | Fight._use_memory | slot | Естелікті қолданады: жылу, зейін немесе адамды қайтару |
| fight.py | PlayerController.decide | fight, me, opp | Ойыншы енгізуін буфердің көмегімен командаға айналдырады |
| setup.py | known_tells | data, enemy, flags | Ойыншы танитын қарсылас қимылдарының жиынын анықтайды |
| setup.py | fight_params | flags, journal, reserve, enemy, data | Флагтар мен журналды Fight параметрлеріне айналдырады |

| Модуль | Функция / әдіс | Параметрлері | Мақсаты |
|---|---|---|---|
| ai/markov.py | MarkovPredictor.predict | — | Келесі әрекетті және оның ықтималдығын болжайды (backoff + Лаплас) |
| ai/markov.py | MarkovPredictor.observe | action | Шын әрекетті жазып, болжам дұрыстығын журналға түсіреді |
| ai/adaptive.py | AdaptiveAI.decide | fight, me, opp | Қарсыластың шешімі: қорғану, фінт, жазалау немесе қарапайым шабуыл |
| ai/adaptive.py | AdaptiveAI._observe | fight, player | Ойыншының жаңа әрекетін байқап, болжамға салыстырады |
| ai/metrics.py | online_accuracy | sequence, predictor | Тізбек бойынша болжау дәлдігін және оқу қисығын есептейді |
| ai/metrics.py | compare | sequence | Барлық әдістің дәлдігін бір сөздікте қайтарады |

render пакеті жарықтандыруды, портреттерді, бөлшектерді және эффектілерді сызады.

3.5-кесте — render пакетінің негізгі функциялары мен әдістері

| Модуль | Функция / әдіс | Параметрлері | Мақсаты |
|---|---|---|---|
| lighting.py | Lighting.render | screen, lights, cam_x, t, offset | Жарықтандыру: қараңғы қабат пен жарық дақтарын сахнаға көбейтеді |
| lighting.py | make_vignette | size, strength | Виньетка бетін жасайды |
| portrait.py | build_portrait | name, state | Портрет бетін жасап, кэштейді (human, fading, shadow) |
| portrait.py | draw_portrait | surface, name, state, cx, bottom, t, active, scale | Портретті жарқылымен және тыныс қозғалысымен сызады |
| corruption.py | draw_corruption | surface, level, t | 3-түнде экранды қараңғылатып, жоғалу жолақтарын сызады |
| corruption.py | hidden | level, threshold | Интерфейс элементі осы деңгейде жасырылады ма |
| effects.py | Fade.start | callback, duration | Қараңғыға өшіп, callback орындап, қайта ашылады |
| effects.py | ScreenShake.add | amount | Экран дірілін күшейтеді |

scenes пакеті ойын сахналарын біріктіреді: мәзір, жүру, диалог, жекпе-жек, түн қорытындысы және финал.

3.6-кесте — scenes пакетінің негізгі функциялары мен әдістері

| Модуль | Функция / әдіс | Параметрлері | Мақсаты |
|---|---|---|---|
| fight.py | FightScene.update | dt | Бекітілген қадаммен логиканы жүргізеді, баяулау мен эффектілерді басқарады |
| fight.py | FightScene._on_event | ev | Жекпе-жек оқиғасына дыбыс, діріл, зум, бөлшектер қосады |
| fight.py | FightScene._finish | — | Нәтижені (saved/killed) флагтарға жазады, естелік қорын азайтады |
| explore.py | ExploreScene.interactables | — | Жақын тұрған белсенді есіктер мен кейіпкерлер |
| explore.py | ExploreScene.night_complete | — | Түннің барлық диалогы мен шайқасы аяқталды ма |
| dialogue.py | DialogueScene.handle_action | action | Space, 1–4, Tab, ESC әрекеттерін өңдейді |
| dialogue.py | glitch | text, level, t, node_id | 3-түнде мәтіннің кейбір әріптерін жояды |
| night_end.py | saved_count | flags | Құтқарылған достар санын санайды |
| night_end.py | ending_key | count | Құтқарылғандар санына қарай финал кілтін қайтарады |

### 3.2 Идентификаторлар кестесі

Төмендегі кестеде бағдарламада қолданылатын негізгі тұрақтылар, кластар және маңызды атрибуттар берілген. Тұрақтылардың мәндері ойынның баланстау параметрлері болып табылады және бір орында (өзінің модулінде) жарияланады, сондықтан оларды өзгерту бағдарламаның басқа бөліктеріне әсер етпейді. Идентификаторлардың атаулары ағылшын тілінде жазылған (PEP 8 ережесіне сай), ал түсіндірмелері қазақ тілінде берілген.

3.7-кесте — Бағдарламаның негізгі идентификаторлары

| Модуль | Идентификатор | Түрі | Мағынасы |
|---|---|---|---|
| core/game.py | LOGICAL_SIZE | тұрақты | Ойынның ішкі ажыратымдылығы (1280, 720); экранға масштабталады |
| core/game.py | FPS | тұрақты | Кадр жиілігі: 60 кадр/с |
| core/game.py | FADE_IN | тұрақты | Сахнаға өткендегі жұмсақ ашылу ұзақтығы, секунд |
| core/game.py | Game | класс | Ойын циклі, сахналар стегі және ойын күйі |
| core/game.py | Game.held | атрибут | Қазір басулы әрекеттер жиыны |
| core/game.py | Game.reserve | атрибут | Естелік қоры (бастапқы мәні 100); адамды қайтарған сайын азаяды |

| Модуль | Идентификатор | Түрі | Мағынасы |
|---|---|---|---|
| core/game.py | Game.night | атрибут | Қазіргі түн нөмірі (1–3) |
| core/game.py | Game.corruption | атрибут | Басты кейіпкердің бүліну деңгейі, 0…1 (үшінші түнде өседі) |
| core/game.py | Game.flags | атрибут | Ойыншы таңдауларының флагтары |
| core/game.py | Game.journal | атрибут | Естеліктер журналы |
| core/input.py | KEYMAP | тұрақты | Перне кодтарын әрекет аттарына сәйкестендіру кестесі |
| core/input.py | InputBuffer | класс | Соңғы 6 кадрдағы әрекеттерді сақтайтын енгізу буфері |
| core/assets.py | ROOT | тұрақты | Ресурстар қалтасы (PyInstaller-де sys._MEIPASS) |
| core/assets.py | Assets | класс | Шрифттер кэші |
| core/save.py | VERSION | тұрақты | Сақтау файлының пішім нұсқасы |
| core/audio_synth.py | RATE | тұрақты | Дыбыс жиілігі: 22050 Гц |
| core/audio_synth.py | SOUNDS | тұрақты | Дыбыс атаулары мен синтез функцияларының сөздігі |
| core/audio.py | MOTIF | тұрақты | Кейіпкер мотивтері: пышақ, барабан, тігін, метроном |
| story/graph.py | VALID_STATES | тұрақты | Портрет күйлері: human, fading, shadow |
| story/graph.py | VALID_BG | тұрақты | Диалог фонына рұқсат етілген бөлмелер |
| story/graph.py | DialogueGraph | класс | Түйіндер мен таңдау бұтақтарынан тұратын диалог графы |
| story/graph.py | DialogueRunner | класс | Графпен жүру: флагтар, журнал және портрет күйлерін жаңарту |
| story/graph.py | GraphError | класс | Граф қатесі (тексеру кезінде) |
| story/flags.py | Flags | класс | Флагтар сөздігі және шартты тексеру |
| story/journal.py | Journal | класс | Естеліктер журналы |
| explore/player.py | SPEED | тұрақты | Жүрудің ең жоғары жылдамдығы, px/c |
| explore/player.py | ACCEL | тұрақты | Жылдамдану мен тежелу қарқыны |
| explore/player.py | MARGIN | тұрақты | Локация шетінен қашықтық, px |
| explore/locations.py | Obj | класс | Локация нысаны: есік немесе кейіпкер |
| explore/locations.py | Location | класс | Жатақхана локациясы |
| combat/fighter.py | ARENA | тұрақты | Жекпе-жек алаңының шектері, px |
| combat/fighter.py | Fighter | класс | Кадрлық FSM-мен жұмыс істейтін жауынгер |

| Модуль | Идентификатор | Түрі | Мағынасы |
|---|---|---|---|
| combat/fighter.py | Fighter.state | атрибут | FSM күйі: idle, block, соққы атауы, hurt, stagger, dead |
| combat/fighter.py | Fighter.frame | атрибут | Күйдегі кадр нөмірі |
| combat/fighter.py | Fighter.hp | атрибут | Денсаулық |
| combat/fighter.py | Fighter.memory | атрибут | Естелік жолағының мәні |
| combat/fighter.py | Fighter.stun | атрибут | Қатып қалу немесе блоктан шығуды кідірту ұзақтығы |
| combat/fighter.py | Fighter.has_hit | атрибут | Қазіргі соққы тиіп қойды ма (бір соққы бір рет тиеді) |
| combat/fighter.py | Fighter.block_cooldown | атрибут | Парирлеуді қайталап басуға тыйым салатын кулдаун |
| combat/hitbox.py | GROUND | тұрақты | Кейіпкер табаны тұратын сызық, px |
| combat/hitbox.py | HURT_W | тұрақты | hurtbox ені, px |
| combat/hitbox.py | HURT_H | тұрақты | hurtbox биіктігі, px |
| combat/fight.py | Event | класс | Жекпе-жек оқиғасы: hit, block, parry, ko, mem_heal, mem_focus, restore |
| combat/fight.py | Fight | класс | Жекпе-жек ядросы |
| combat/fight.py | Fight.hitstop | атрибут | Соққыдан кейінгі қатып қалу кадрлары |
| combat/fight.py | Fight.focus_left | атрибут | «Зейін» эффектісінің қалған кадрлары |
| combat/fight.py | Fight.outcome | атрибут | Нәтиже: kill, restore немесе lose |
| combat/fight.py | STAGGER_BONUS | тұрақты | Парирленген жауға қосымша зиян көбейткіші (1,5) |
| combat/fight.py | PARRY_MEMORY_GAIN | тұрақты | Сәтті парирлеу берген естелік (15) |
| combat/fight.py | MEMORY_COST | тұрақты | Бір естелік слотының бағасы (33) |
| combat/fight.py | HEAL_AMOUNT | тұрақты | «Жылу» естелігінің емдеуі (25) |
| combat/fight.py | FOCUS_BASE | тұрақты | «Зейін» ұзақтығы кадрмен (180) |
| combat/fight.py | FOCUS_PARRY_BONUS | тұрақты | Парирлеу терезесіне қосымша кадр (6) |
| combat/fight.py | RESTORE_HP_FRACTION | тұрақты | Қайтару үшін қарсылас денсаулығының шегі (35%) |
| combat/fight.py | UNKNOWN_TELL_SPEEDUP | тұрақты | Белгісіз қимылдың дайындығы жылдамдауы (×0,7) |
| combat/setup.py | START_MEMORY_FRACTION | тұрақты | Шайқас басындағы естелік жолағы = қордың жартысы |
| combat/setup.py | CALLED_NAME_HP_MULT | тұрақты | Атын атаған жаудың денсаулық көбейткіші (0,85) |
| combat/setup.py | RESTORE_RESERVE_COST | тұрақты | Адамды қайтарудың естелік қорына бағасы (25) |

| Модуль | Идентификатор | Түрі | Мағынасы |
|---|---|---|---|
| combat/ai/markov.py | ACTIONS | тұрақты | Болжанатын әрекеттер: light, heavy, block |
| combat/ai/markov.py | MarkovPredictor | класс | Жиілік кестесі бойынша келесі әрекетті болжау |
| combat/ai/adaptive.py | NIGHT_THRESHOLD | тұрақты | Түндер бойынша болжам сенім шегі: 0,70; 0,55; 0,45 |
| combat/ai/adaptive.py | ANTICIPATED_DELAY | тұрақты | Болжам дұрыс болғанда реакция кідірісі (0–3 кадр) |
| combat/ai/adaptive.py | UNANTICIPATED_DELAY | тұрақты | Болжам қате болғанда реакция кідірісі (16–24 кадр) |
| combat/ai/adaptive.py | AdaptiveAI | класс | Бейімделгіш қарсылас ИИ |
| render/scenery.py | WALL_BOTTOM | тұрақты | Қабырға мен еденнің шекарасы, px |
| render/scenery.py | Scenery | класс | Алдын ала сызылған локация фоны, терезелер және жарықтар |
| render/portrait.py | CHARACTERS | тұрақты | Кейіпкерлердің акцент түстері |
| render/portrait.py | LOOK | тұрақты | Кейіпкер келбеті: тері, шаш, киім түстері және шаш үлгісі |
| render/particles.py | ParticleSystem | класс | Бөлшектер жүйесі: ұшқын, шаң, көлеңке бөлшектері |
| render/particles.py | Blizzard | класс | Боран бөлшектері |
| render/effects.py | Fade | класс | Қараңғыға өту эффектісі |
| render/effects.py | ScreenShake | класс | Экран дірілі (trauma әдісі) |
| scenes/fight.py | TICK | тұрақты | Логикалық қадам: 1/60 с |
| scenes/fight.py | SLOW_SCALE | тұрақты | Парирлеудегі уақыт баяулау коэффициенті (0,25) |
| scenes/fight.py | ARENA_ROOM | тұрақты | Қарсылас пен жекпе-жек бөлмесінің сәйкестігі |
| scenes/fight.py | FightScene | класс | Жекпе-жек сахнасы |
| scenes/dialogue.py | CHARS_PER_SEC | тұрақты | Мәтін теру жылдамдығы (60 таңба/с) |
| scenes/dialogue.py | DialogueScene | класс | Диалог сахнасы (визуалды новелла) |
| scenes/dialogue.py | JournalScene | класс | Естеліктер журналы сахнасы |
| scenes/explore.py | ExploreScene | класс | Жатақханада жүру сахнасы |
| scenes/night_end.py | FRIENDS | тұрақты | Төрт достың идентификаторлары |
| scenes/night_end.py | NIGHT_FRIENDS | тұрақты | Әр түндегі айналатын достар |
| scenes/night_end.py | NightEndScene | класс | Түн қорытындысы |
| scenes/night_end.py | EndingScene | класс | Финал сахнасы |

### 3.3 Эксперименттік нәтижелер

Эксперименттің мақсаты — қарсыластың ойыншы әрекетін болжау дәлдігін өлшеу, оны қарапайым әдістермен салыстыру және бейімделудің жекпе-жекке әсерін бағалау. Эксперимент tools/experiment.py бағдарламасымен жүргізіледі және терезесіз, бекітілген кездейсоқ бастапқы мәндермен (seed) орындалады, сондықтан нәтиже қайталанады.

**Әдістеме.** Ойыншының орнына төрт «бот» қолданылды: aggressor (көбіне жеңіл соққы жасайды: 65% жеңіл, 30% ауыр, 5% блок), pattern (қатаң қайталанатын тізбек: жеңіл, жеңіл, ауыр, блок), random (үш әрекеттің бірін біркелкі кездейсоқ таңдайды) және reactive (қарсыластың дайындық қимылын көріп, парирлеуге тырысады, содан кейін шабуылдайды). Әр бот үшін 30 жекпе-жек жүргізілді. Қарсылас ретінде ең жылдам қимылдары бар Бекзат таңдалды, ИИ-дің сенім шегі 0,55. Әр жекпе-жек ең көбі 90 секундқа созылды. Бот жеңіл соққы, ауыр соққы немесе блок жасаған әр сәт бір бақылау болып саналды.

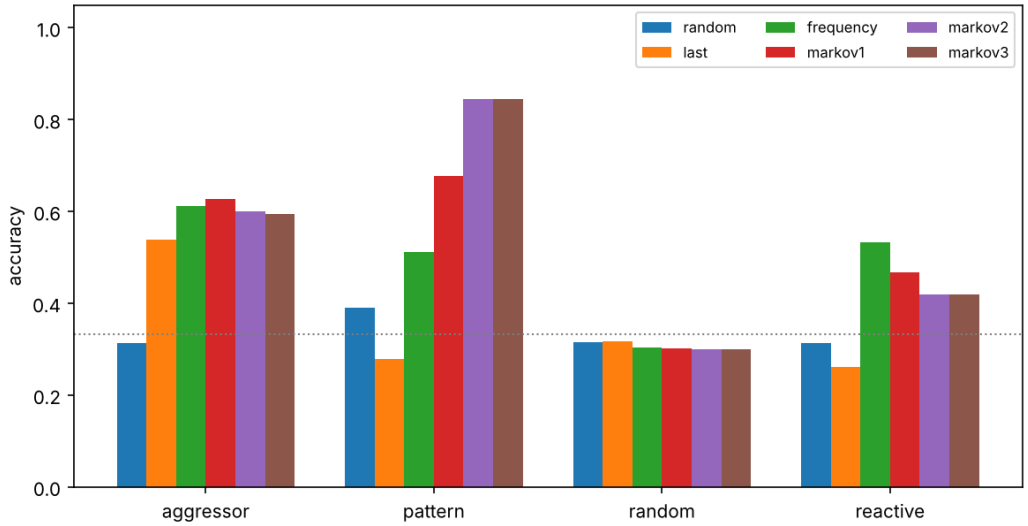
Болжау дәлдігі «дұрыс болжам саны / барлық болжам саны» ретінде есептелді; әр бақылауға дейін болжам жасалып, содан кейін ғана шын әрекет жазылды (онлайн бағалау). Салыстыру үшін мынадай әдістер қолданылды: random — әрекетті кездейсоқ болжау (теориялық дәлдігі 1/3); last — алдыңғы әрекет қайталанады деп болжау; frequency — әрқашан ең жиі кездескен әрекетті болжау (0-ші ретті модель); markov1, markov2, markov3 — 1-, 2- және 3-ші ретті Марков модельдері.

**Нәтижелер.** Болжау дәлдігі 3.8-кестеде көрсетілген.

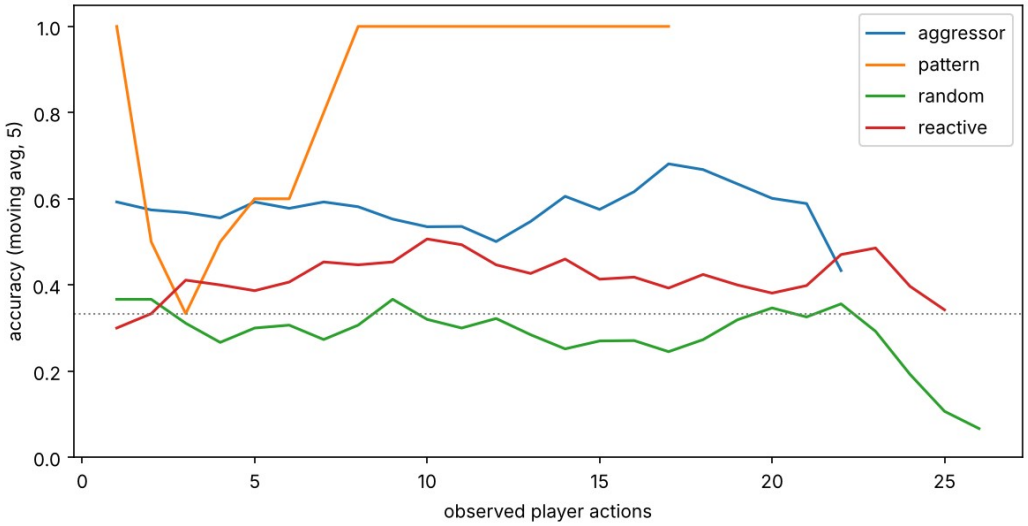
3.8-кесте — Болжау дәлдігі: әр бот үшін 30 жекпе-жек, қарсылас — Бекзат

| Бот | random | last | frequency | markov1 | markov2 | markov3 | ИИ (жекпе-жек) |
|---|---|---|---|---|---|---|---|
| aggressor | 31,4% | 53,9% | 61,1% | 62,8% | 60,0% | 59,4% | 60,0% |
| pattern | 39,0% | 27,9% | 51,2% | 67,8% | 84,4% | 84,4% | 84,4% |
| random | 31,5% | 31,7% | 30,4% | 30,1% | 29,9% | 29,9% | 29,9% |
| reactive | 31,4% | 26,1% | 53,3% | 46,7% | 41,9% | 41,9% | 41,9% |

3.1-суретте әдістердің дәлдігі, ал 3.2-суретте ИИ-дің алғашқы 40 бақылау барысындағы үйрену қисығы көрсетілген.




3.1-сурет — Әр бот үшін болжау әдістерінің дәлдігі (нүктелі сызық — 1/3)




3.2-сурет — ИИ-дің үйрену қисығы (5 бақылау бойынша жылжымалы орташа)

**Нәтижелерді талдау.** Қатаң қайталанатын тізбекпен ойнайтын pattern ботына қатысты екінші ретті Марков моделі 84,4% дәлдікке жетті, ал frequency әдісі 51,2%, кездейсоқ болжау 39,0%, «алдыңғы әрекетті қайталау» тек 27,9% көрсетті. Бұл Марков тізбегінің дәл осындай әдеттерді ұстайтынын растайды: бірінші ретті модель (67,8%) бір ғана алдыңғы әрекетке қарайды және L, L, H, B тізбегіндегі «жеңіл соққыдан кейін не болады» деген сұраққа анық жауап бере алмайды, ал екінші ретті модель екі әрекетке қарап, бұл екі мағыналықты шешеді. Үшінші ретті модель қосымша артықшылық бермейді (84,4%), яғни екінші ретті модель жеткілікті.

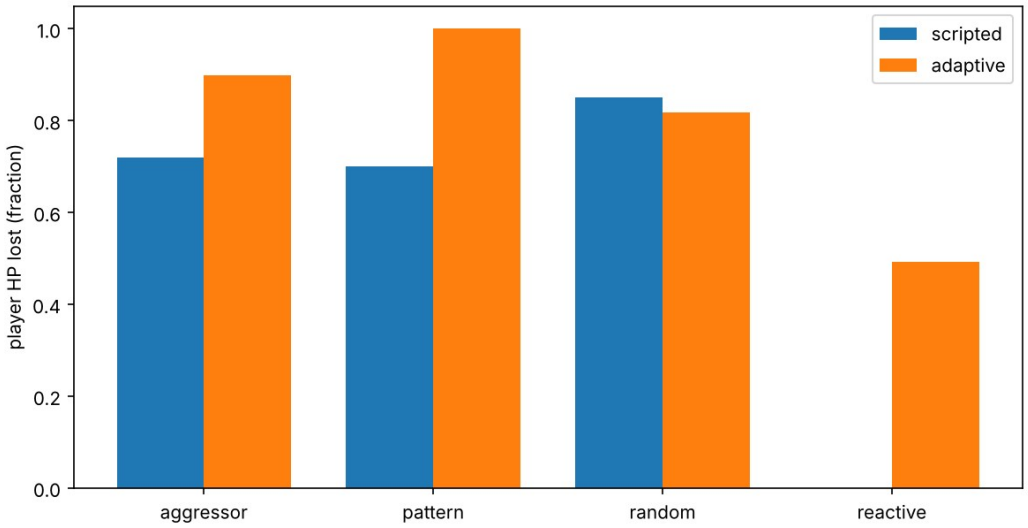
Көбіне жеңіл соққы жасайтын aggressor ботына қатысты ең жақсы нәтижені бірінші ретті модель берді (62,8%), бұл frequency (61,1%) мен кездейсоқ болжаудан (31,4%) жоғары: бот әрекеттері дерлік тәуелсіз сынақтар тізбегі болғандықтан, ең жиі әрекетті болжау да жақсы нәтиже береді, ал

Марков моделі оған аздап қосымша береді. Біркелкі кездейсоқ ойнайтын random ботына қатысты барлық әдістің дәлдігі шамамен 1/3 деңгейінде қалды (markov2: 29,9%), бұл күтілген нәтиже: кездейсоқ әрекетті болжап болмайды. Reactive ботының әрекеттері қарсыластың қимылына байланысты, сондықтан тек ойыншы әрекеттерінің тарихына негізделген модель мұнда нашар жұмыс істейді (markov2: 41,9%), ал frequency әдісі жақсы нәтиже берді (53,3%). Бұл модельді дамытуға болатын бағытты көрсетеді: контекстке қарсыластың күйін қосу.

**Бейімделудің жекпе-жекке әсері.** Болжау тек қарсылас оны пайдаланғанда ғана маңызды болады. 3.9-кестеде бір ботқа қарсы бейімделмейтін (ScriptedAI) және бейімделгіш (AdaptiveAI) қарсыластың нәтижелері салыстырылған.

3.9-кесте — Бейімделмейтін және бейімделгіш қарсыластың нәтижелері

| Бот | Қарсылас | Бот жоғалтқан денсаулық | Қарсылас жоғалтқан денсаулық | Ботның жеңу үлесі | Орташа ұзақтық, с |
|---|---|---|---|---|---|
| aggressor | бейімделмейтін | 72,0% | 96,3% | 73,3% | 19,7 |
| aggressor | бейімделгіш | 89,9% | 60,3% | 26,7% | 17,5 |
| pattern | бейімделмейтін | 70,1% | 93,6% | 76,7% | 20,4 |
| pattern | бейімделгіш | 100,0% | 33,4% | 0,0% | 15,6 |
| random | бейімделмейтін | 85,0% | 83,4% | 43,3% | 21,0 |
| random | бейімделгіш | 81,8% | 82,1% | 46,7% | 22,0 |
| reactive | бейімделмейтін | 0,0% | 100,0% | 100,0% | 13,0 |
| reactive | бейімделгіш | 49,2% | 98,9% | 93,3% | 17,4 |




3.3-сурет — Бот жоғалтқан денсаулық үлесі: бейімделмейтін және бейімделгіш қарсылас

Бейімделгіш қарсылас әдеті бар ойыншыларға қарсы айтарлықтай күшті: aggressor ботының жеңу үлесі 73,3% мәнінен 26,7% мәніне, ал pattern ботынікі 76,7% мәнінен 0,0% мәніне түсті. Дәл осы жерде қарсыластың болжамы соққыны парирлеуге және жазалауға мүмкіндік береді. Керісінше, кездейсоқ ойнайтын ботқа қатысты нәтиже іс жүзінде өзгермеді (жеңу үлесі 43,3% және 46,7%), яғни ИИ болжай алмайтын ойыншыны басымдыққа ие етпейді. Парирлеуге бейімделген reactive ботына қарсы бейімделмейтін қарсылас ешқандай зиян келтіре алмады (жоғалған денсаулық 0,0%), ал бейімделгіш қарсылас 49,2% зиян келтірді.

**Қорытынды.** Эксперимент бейімделу механикасының екі қасиетін көрсетті: біріншіден, ол ойыншының нақты әдеттеріне ғана әсер етеді және оны өзгертуге ынталандырады (бұл ойын дизайны үшін қалаған нәтиже); екіншіден, болжау дәлдігі әдеттің қатаңдығына байланысты — қатаң тізбекте 84,4%, кездейсоқ ойында ≈ 1/3. Шектеулер: ботлар нағыз ойыншы емес, сондықтан нәтиже — қарсылас моделінің жұмыс істеу принципін көрсететін модельдік эксперимент; тәжірибе жеткілікті өлшем үшін 30 жекпе-жекпен шектелді; нақты адамдармен сынақ жүргізу болашақ жұмыстың бағыты болып табылады.

# ҚОРЫТЫНДЫ

Курстық жоба барысында Python тілінде PyGame кітапханасын пайдаланып, «Көлеңке» — визуалды новелла, қозғалыс және жекпе-жек біріктірілген 2D ойыны әзірленді. Бастапқы тапсырмада қойылған міндеттер орындалды.

Жоба барысында шешілген негізгі міндеттер:

- сценарий түйіндер мен таңдау бұтақтарынан тұратын бағытталған граф түрінде JSON файлдарында сипатталды; граф жүктелген кезде тексеріледі (өтулер, қол жетпейтін түйіндер, естеліктер, портрет күйлері);
- флагтар жүйесі мен естеліктер журналы жасалды, ал флагтар жекпе-жек параметрлеріне әсер етеді: дайындық белгісінің көрінуі, қарсыластың күші, естелік қорының бастапқы мәні;
- жекпе-жек кадрлық ақырлы автомат, hitbox/hurtbox, 8 кадрлық парирлеу терезесі және 6 кадрлық енгізу буфері негізінде іске асты; hitstop, экран дірілі, парирлеудегі баяулау және зум сияқты эффектілер логикадан бөлек жасалды;
- ойыншының әдеттерін Марков тізбегі арқылы үйренетін бейімделгіш қарсылас жасалып, болжау дәлдігі өлшенді: қатаң әдетті ойыншыда екінші ретті модель 84,4% дәлдікке жетті, ал кездейсоқ ойында ≈ 1/3 болды;
- силуэттік портреттер, жарықтандыру, бөлшектер, боран, виньетка және дыбыс (кейіпкер мотивтерімен) сыртқы суретсіз және аудио файлсыз бағдарламалық түрде жасалды;
- үш түн, төрт финал, JSON сақтау және үшінші түндегі бүліну (экранның қараюы, интерфейстің жоғалуы) іске асырылды;
- бағдарлама 261 автоматты тестпен қамтылды, PyInstaller арқылы жинау конфигурациясы және терезесіз өздігінен тексеру режимі (--selftest) жасалды.

**Жобаның тиімділігі мен сенімділігі.** Бағдарламаның бастапқы коды шамамен 3 900 жол. Сахналардағы бір кадрды есептеу және сызу уақыты әзірлеу компьютерінде шамамен 9 мс болды, ал 60 кадр/с үшін шек — 16,7 мс; яғни Core i3 деңгейіндегі компьютерге қойылатын талап үшін шамамен екі еселік қор қалдырылды. Ауыр бөліктер (фондар, портреттер, дыбыстар) бір рет жасалып кэштеледі. Бүлінген сақтау файлы, аудио құрылғының жоқтығы сияқты қате жағдайлары өңделеді және ойынды тоқтатпайды. Шектеу: өнімділік талабы нақты Core i3 құрылғысында тікелей тексерілмеді, ол қордың арқасында орындалады деп бағаланады; Windows үшін .exe файлы build_exe.bat арқылы Windows ортасында жиналады.

**Практикалық маңызы.** Әзірленген бағдарлама толық ойын ретінде көрсетіле алады. Диалог графы, флагтар жүйесі, кадрлық жекпе-жек ядросы және бейімделгіш ИИ бөлек модульдер ретінде жазылғандықтан, оларды ұқсас жобаларда қайта қолдануға болады, ал эксперимент нәтижелері ойынға арналған Марков негізіндегі қарсыласты бағалауға мысал бола алады.

**Мүмкін жетілдіру жолдары:** бастапқы кодты өзгертпей-ақ графикада қолмен салынған суреттерді қолдану (портреттер мен фондар интерфейс арқылы ауыстырылады); бейімделгіш ИИ-ге қарсыластың күйін және арақашықтықты контекст ретінде қосу; гейпадті қолдау; ойынның басқа тілдерге аударылуы (интерфейс жолдары бөлек JSON файлында); көбірек түндер мен кейіпкерлерді қосу; ИИ-ді нағыз ойыншылармен сынау.

## ҚОЛДАНЫЛҒАН ДЕРЕКТЕР ТІЗІМІ


1. Python Software Foundation. Python 3 Documentation. — URL: https://docs.python.org/3/
2. Pygame community. Pygame Documentation. — URL: https://www.pygame.org/docs/
3. Sweigart A. Making Games with Python & Pygame. — CreateSpace Independent Publishing Platform, 2012.
4. McGugan W. Beginning Game Development with Python and Pygame: From Novice to Professional. — Berkeley: Apress, 2007.
5. Lutz M. Learning Python. 5th ed. — Sebastopol: O'Reilly Media, 2013.
6. Nystrom R. Game Programming Patterns. — Genever Benning, 2014.
7. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. — Reading: Addison-Wesley, 1994.
8. Swink S. Game Feel: A Game Designer's Guide to Virtual Sensation. — Burlington: Morgan Kaufmann, 2009.
9. Millington I., Funge J. Artificial Intelligence for Games. 2nd ed. — Burlington: Morgan Kaufmann, 2009.
10. Norris J. R. Markov Chains. — Cambridge: Cambridge University Press, 1997.
11. Jurafsky D., Martin J. H. Speech and Language Processing. 2nd ed. — Upper Saddle River: Pearson Prentice Hall, 2009.
12. Schell J. The Art of Game Design: A Book of Lenses. — Burlington: Morgan Kaufmann, 2008.
13. Fullerton T. Game Design Workshop: A Playcentric Approach to Creating Innovative Games. 3rd ed. — Boca Raton: CRC Press, 2014.
14. Bray T. (ed.). RFC 8259. The JavaScript Object Notation (JSON) Data Interchange Format. — IETF, 2017. — URL: https://www.rfc-editor.org/rfc/rfc8259
15. van Rossum G., Warsaw B., Coghlan A. PEP 8 — Style Guide for Python Code. — URL: https://peps.python.org/pep-0008/
16. PyInstaller Team. PyInstaller Manual. — URL: https://pyinstaller.org/
17. pytest-dev. pytest Documentation. — URL: https://docs.pytest.org/
18. SDL Wiki. Simple DirectMedia Layer. — URL: https://wiki.libsdl.org/
19. Ren'Py Visual Novel Engine. Documentation. — URL: https://www.renpy.org/doc/html/
20. Godot Engine. Documentation. — URL: https://docs.godotengine.org/
21. Unity Technologies. Unity Manual. — URL: https://docs.unity3d.com/Manual/
22. Matplotlib development team. Matplotlib Documentation. — URL: https://matplotlib.org/stable/

# ҚОСЫМША А. ТЕХНИКАЛЫҚ ТАПСЫРМА

## 1 Кіріспе

Бұл техникалық тапсырма «Көлеңке» — визуалды новелла, қозғалыс және жекпе-жек біріктірілген 2D ойынын әзірлеуге қойылатын талаптарды сипаттайды. Жобаның мақсаты — Python тілінде PyGame кітапханасын пайдаланып, жатақханадағы төрт таныс көлеңкеден құтқару туралы ойынды жасау, ойынның диалог жүйесін, флагтар жүйесін, кадрлық жекпе-жегін және бейімделгіш қарсыласын іске асыру. Жобаның ерекшелігі — әртүрлі үш ойын түрінің біріктірілуі және визуалды стильдің сюжеттің бөлігі болуы: кейіпкерлер қара силуэт және бір акцентті түс арқылы беріледі, көлеңке жұтқан сайын олардың детальдары жоғалады.

## 2 Дамудың негізі

Әзірлеу негізі — 06130100 «Бағдарламалық қамтамасыз ету (түрлері бойынша)» мамандығының 3-курс студенттеріне арналған «Алгоритмдеу және бағдарламалау» пәні бойынша курстық жобаны орындауға берілген тапсырма. Жобаның тақырыбы: «Көлеңке» — визуалды новелла, қозғалыс және жекпе-жек біріктірілген 2D ойыны. Әзірлеудің басталуы мен аяқталуы оқу жоспарында және жетекшінің жеке жұмыс кестесінде бекітілген мерзімдермен анықталады.

## 3 Тағайындау

Бағдарлама оқу мақсатында ойын жасау арқылы алгоритмдеу дағдыларын көрсетуге арналған. Ойын мынадай мүмкіндіктерді береді: қыстағы үш түн ішінде жатақханада жүру; диалогтарда таңдау жасау және оларды флагтарға жазу; жасырын детальдарды естеліктер журналына жинау; көлеңкеге айналған достармен кадрлық жекпе-жек өткізу; қарсыласты өлтіру немесе естеліктер арқылы адамдық кейпіне қайтару; құтқарылғандар санына байланысты төрт финалдың бірін көру; ойын барысын JSON файлына сақтау және жалғастыру.

## 4 Бағдарламалық өнімге қойылатын талаптар

### 4.1 Функционалдық талаптар

– Пайдаланушы интерфейсі: ойын терезесі (толық экран және терезе режимдері), басты мәзір («Жалғастыру», «Жаңа ойын», «Шығу»), диалог терезесі, естеліктер журналы, түн индикаторы, денсаулық және естелік жолақтары, дайындық қимылының белгісі; дыбыс (буран, соққы, парирлеу, әр кейіпкердің мотиві: пышақ, барабан, тігін

машинасы, метроном); силуэттік графика, бөлшектер мен жарықтандыру.

- Ойын логикасы: қозғалыс (A/D немесе ← →); әрекеттесу (E); диалог бойынша жүру (Space, 1–4); жекпе-жекте жеңіл және ауыр соққы (J, K), блок және парирлеу (L), естеліктерді қолдану (1–3); соқтығысу (hitbox/hurtbox); денсаулық пен естелік есебі; жеңіс және жеңіліс шарттары (денсаулық нөлге түсу, адамды қайтару); 4 финал.
- Негізгі модульдер: main.py (іске қосу), kolenke/core (цикл, сахналар, енгізу, сақтау, дыбыс), kolenke/story (диалог графы, флагтар, журнал), kolenke/explore (жүру), kolenke/combat (жекпе-жек және ИИ), kolenke/render (сызу), kolenke/scenes (сахналар), kolenke/data (JSON деректері).
- Бейімделгіш ИИ: ойыншы әрекеттерін жиілік кестесіне жазу (Марков тізбегі принципі), келесі әрекетті болжау, болжау дәлдігін өлшеу.
- Тұрақты жұмыс және қате жағдайларын өңдеу: бүлінген сақтау файлы, аудио құрылғының жоқтығы, терезені жабу.

### 4.2 Сенімділік талаптары

Жүйе ойын барысында тоқтап қалмауы, мәліметтер жоғалмауы (сақтау файлы атомдық түрде жазылады) және терезелер дұрыс жабылуы тиіс. Қате деректермен (бүлінген сақтау, жоқ ресурс) бағдарлама ойын күйін бұзбай, түсінікті түрде жалғасуы керек.

### 4.3 Техникалық құралдарға қойылатын талаптар

- операциялық жүйе: Windows 10 және одан жоғары;
- процессор: Intel Core i3 немесе ұқсас;
- жедел жады (RAM): кемінде 4 ГБ;
- Python 3.10 және одан жоғары нұсқасы мен PyGame орнатылған болуы тиіс (немесе дайын Kolenke.exe файлы қолданылады);
- кадр жиілігі: 60 кадр/с.

## 5 Бағдарламалық құжаттамаға қойылатын талаптар

Құжаттама құрамына кіреді:

- 5.1 Түсініктеме жазба — компьютерде терілген 25 беттен кем емес көлемде; жобаның мақсаты, логикасы, құрылымы және бағдарламалық шешімдері жазылады;
- 5.2 Қолданушы нұсқаулығы — ойынды орнату, іске қосу және басқару тәсілдерін сипаттайтын қысқаша құжат (Ә қосымшасы);
- 5.3 Функционалдық сызба — модульдер арасындағы байланысты және бағдарлама жұмысының барысын көрсететін графикалық схема (А4 форматында; Г қосымшасы), құрылымдық сызба (В қосымшасы).

## ҚОСЫМША Ә. ҚОЛДАНУШЫ БАСҚАРУЫ

### 1 Жүйені іске қосу тәртібі

Ойынды екі тәсілмен іске қосуға болады.

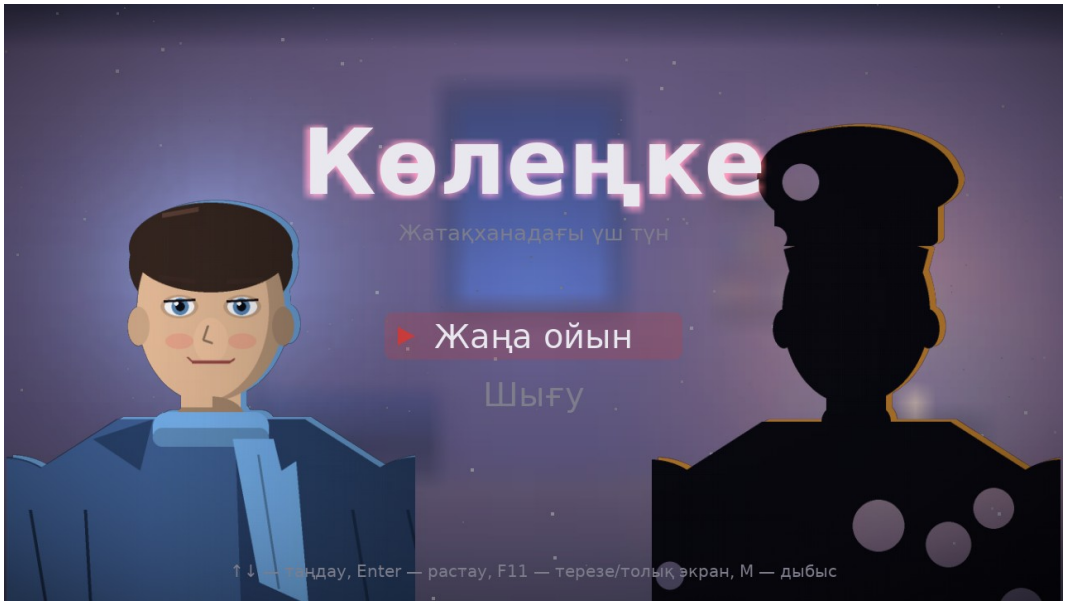
**Бірінші тәсіл — дайын орындалатын файл.** Windows жүйесінде Kolenke.exe файлын екі рет шертіп іске қосыңыз. Python орнату қажет емес. Егер Windows қорғаныс терезесі («Windows компьютеріңізді қорғады») шықса, «Қосымша мәлімет» және «Бәрібір іске қосу» батырмаларын басыңыз: файл әзірленген бағдарламаның өзі.

**Екінші тәсіл — бастапқы кодтан іске қосу.** Python 3.10 және одан жоғары нұсқасы орнатылған болуы керек. Бағдарлама қалтасын ашып, командалық жолда мына пәрмендерді орындаңыз:

- pip install -r requirements.txt — PyGame кітапханасын орнату;
- python main.py — ойынды толық экранда іске қосу;
- python main.py --windowed — ойынды терезеде іске қосу;
- python main.py --selftest — терезесіз өздігінен тексеру (ресурстар, сахналар, кадрлар);
- python -m pytest -q — автоматты тесттерді іске қосу.

### 2 Басқару интерфейсі

Ойын іске қосылғанда басты мәзір ашылады (Ә.1-сурет). Таңдау ↑ және ↓ пернелерімен жасалады, растау — Enter. Бұрын сақтау жасалған болса, «Жалғастыру» пункті пайда болады. Жоғарғы жағында ойын атауы, екі жақта басты кейіпкер мен көлеңке жұтқан достың портреті орналасқан, артта боран жауып тұр.




Ә.1-сурет — Басты мәзір

**Жатақханада жүру (Ә.2-сурет).** Басты кейіпкер A/D (немесе ← →) пернелерімен қозғалады. Сол жақ жоғарыда түн индикаторы (үш шеңбер: қазіргі түн қызыл), оң жақ жоғарыда естеліктер журналындағы жазба саны (Tab), ортасында локация атауы көрсетіледі. Есікке немесе кейіпкерге жақындағанда жоғарыда «E» белгісі бар қалқымалы хабарлама шығады: E басылса, есік ашылады, диалог немесе шайқас басталады. Қараңғы бөлмелерде жарықтың көзі: шам, терезе, есіктің астындағы жарық және басты кейіпкердің фонары.




Ә.2-сурет — Жатақхана дәлізі



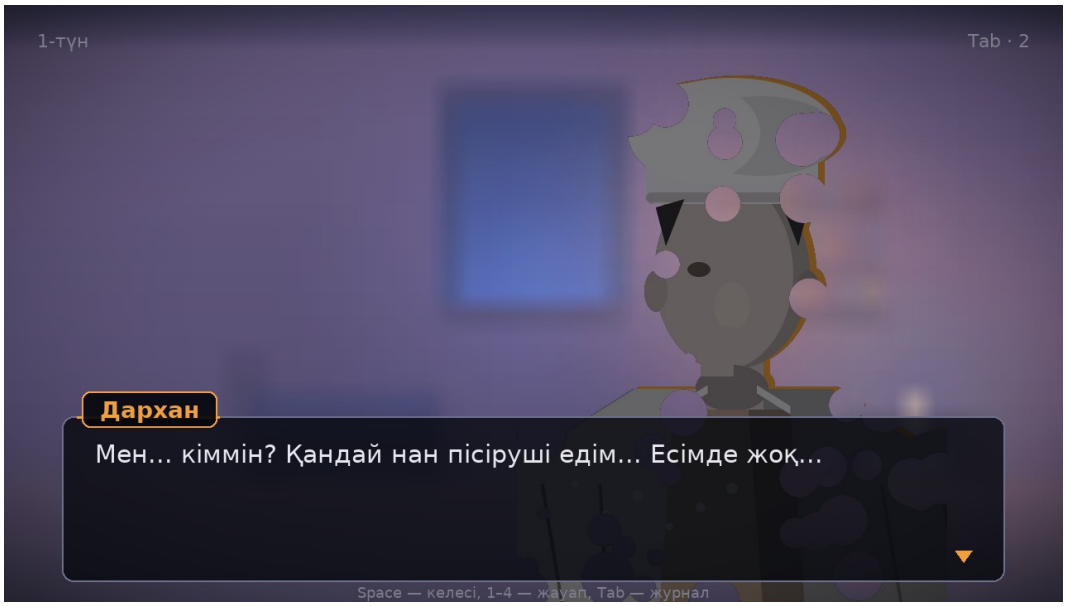

Ә.3-сурет — Дарханның бөлмесі: кейіпкермен сөйлесу ұсынысы

**Диалог (Ә.4–Ә.6-суреттер).** Төменде диалог терезесі орналасады: үстінде сөйлеушінің аты (оның акцент түсімен), ішінде мәтін. Мәтін әріптеп теріледі; Space пернесі қалғанын бірден көрсетеді, ал мәтін толық шыққан соң келесі репликаға өтеді. Таңдау болса, нұсқалар нөмірмен көрсетіледі (1–4).

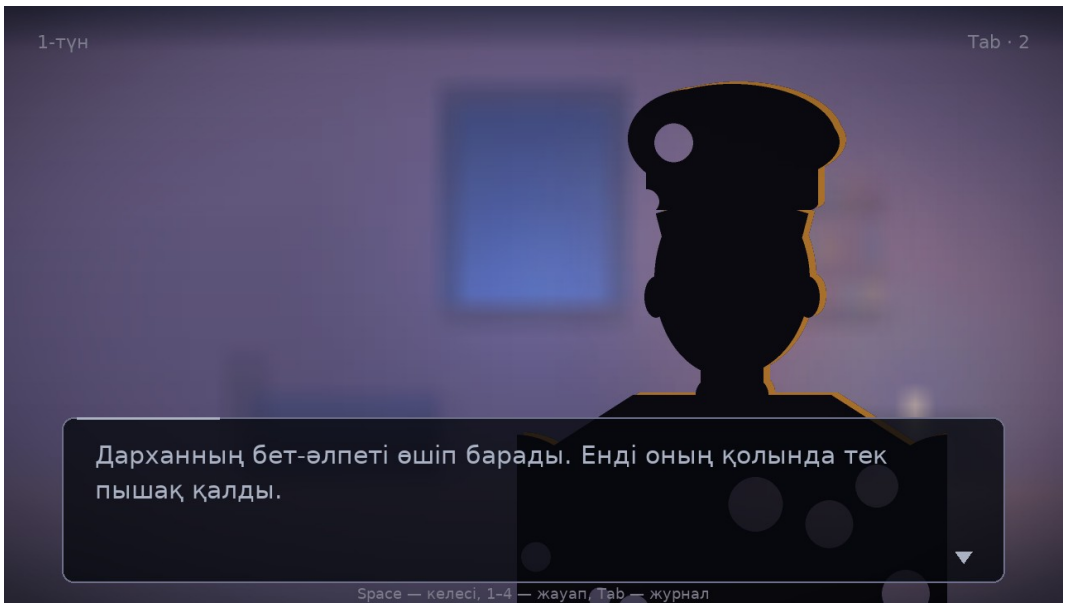
Жаңа жазба журналға түскенде жоғарыда «Жаңа жазба журналда» хабарламасы шығады. Көлеңке жұта бастағанда кейіпкер портреті солғындап, шеттері бұзылады, ал бет белгілері өшеді.




Ә.4-сурет — Диалогтағы таңдау нұсқалары

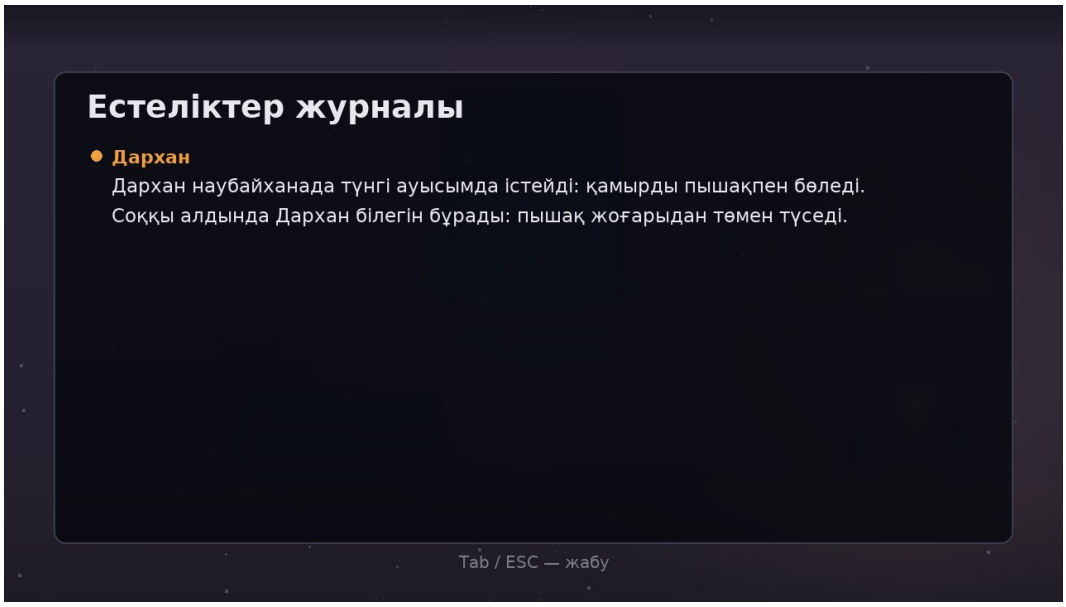



Ә.5-сурет — Кейіпкердің портреті көлеңке жұта бастағанда



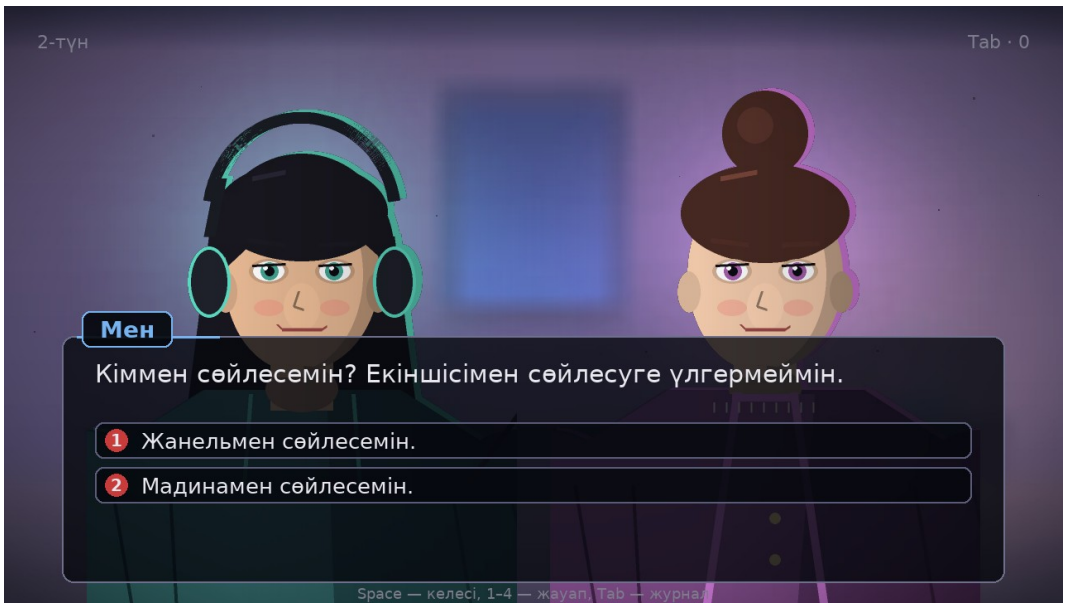

Ә.6-сурет — Кейіпкер көлеңкеге толық айналды

**Естеліктер журналы (Ә.7-сурет)** Tab пернесімен ашылады. Онда диалогта байқалған бөлшектер кейіпкерлер бойынша топтастырылған. Дәл осы бөлшектер шайқаста қарсыластың дайындық қимылын көрсетеді.



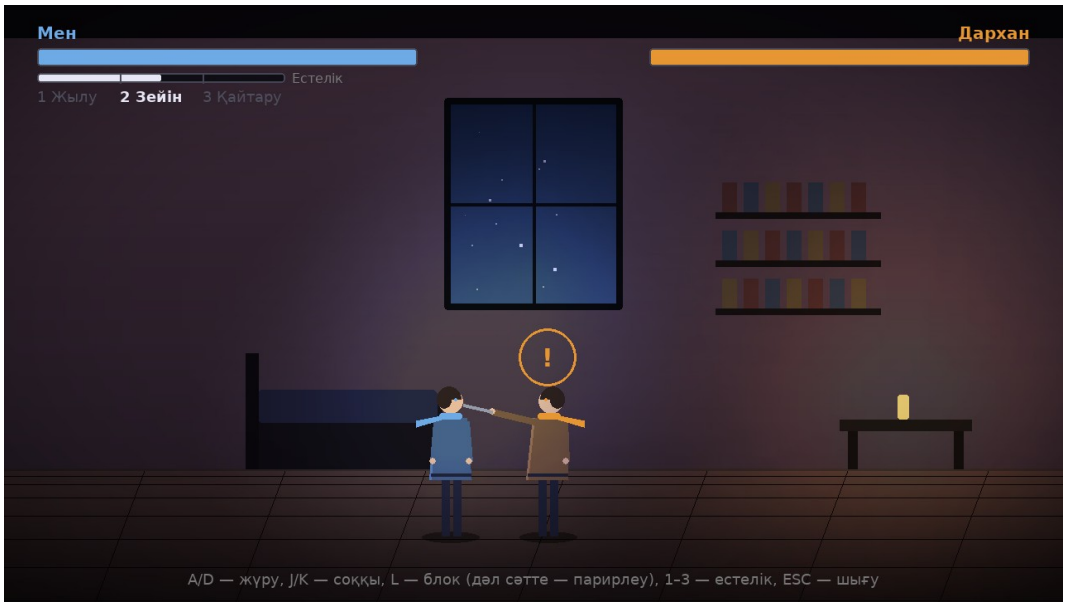

Ә.7-сурет — Естеліктер журналы

Екінші түнде бір бөлмеде екі кейіпкер бірден көрсетіледі; сөйлесіп жатқанының портреті ашық, екіншісі күңгірт.

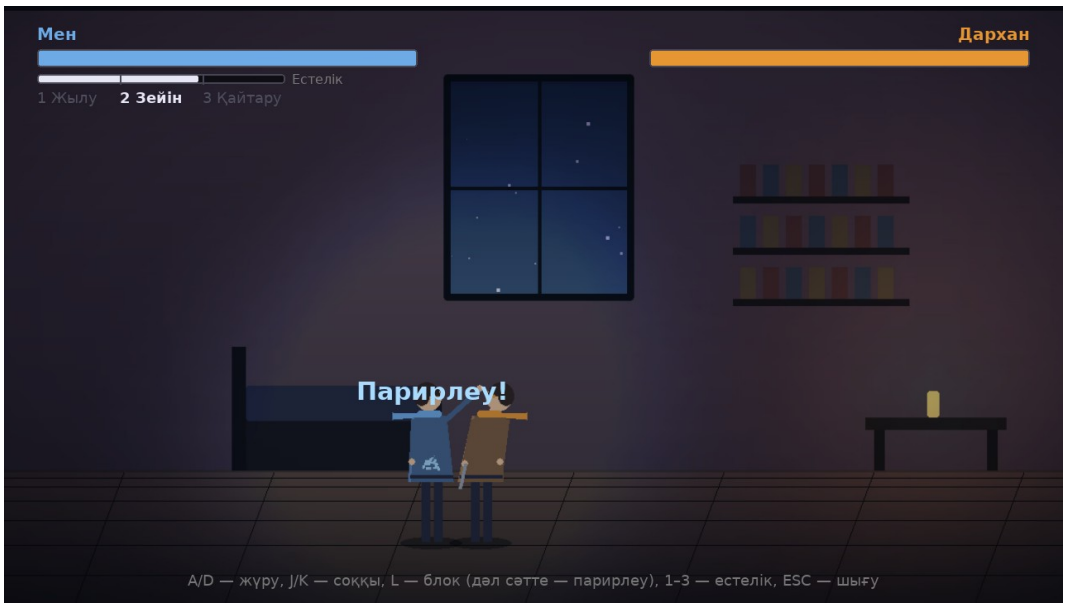



Ә.8-сурет — Екінші түн: екі кейіпкер бір бөлмеде

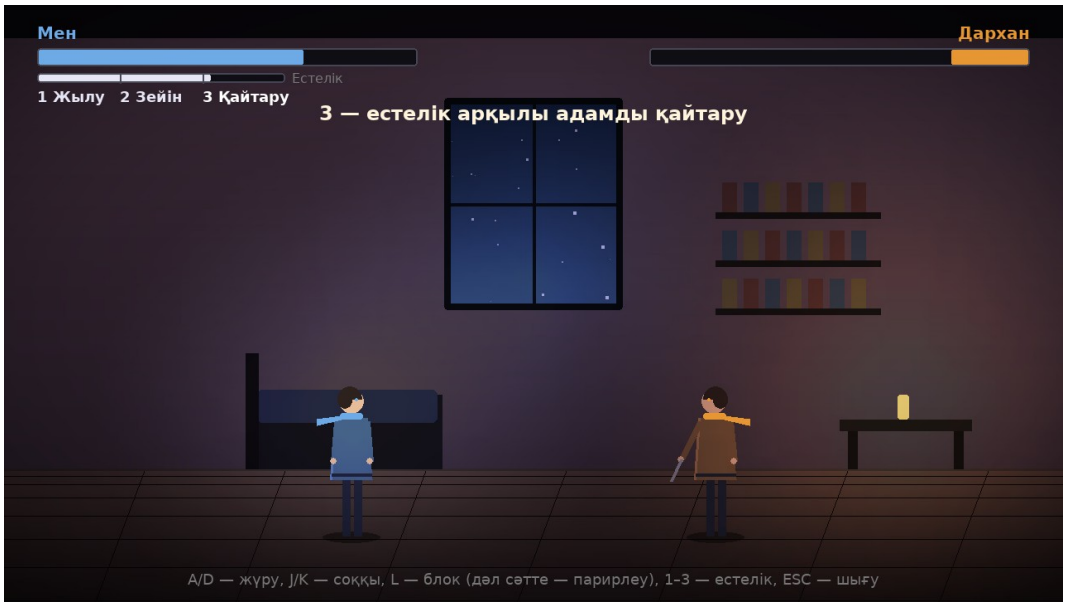
**Жекпе-жек (Ә.9–Ә.11-суреттер).** Жоғарғы сол жақта ойыншының денсаулық жолағы (көк), одан төменде естелік жолағы (үш бөлікке бөлінген) және естелік слоттары (1 «Жылу», 2 «Зейін», 3 «Қайтару»); оң жақта қарсыластың денсаулық жолағы. Қарсылас соққыға дайындалғанда оның басының үстінде белгі шығады: сөйлесуде сол детальді байқасаңыз ғана көрінеді. Шеңбер кішірейіп, ақ түске боялғанда L пернесін басыңыз: парирлеу іске асады — «Парирлеу!» жазуы шығады, уақыт баяулайды, камера жақындайды, қарсылас тепе-теңдігін жоғалтады. Қарсыластың денсаулығы 35%-тен төмен түскенде (немесе ол парирленгенде) және естелік жеткілікті болса, экранда «3 — естелік арқылы адамды қайтару» хабарламасы пайда болады.




Ә.9-сурет — Шайқас: қарсыластың дайындық қимылының белгісі




Ә.10-сурет — Сәтті парирлеу

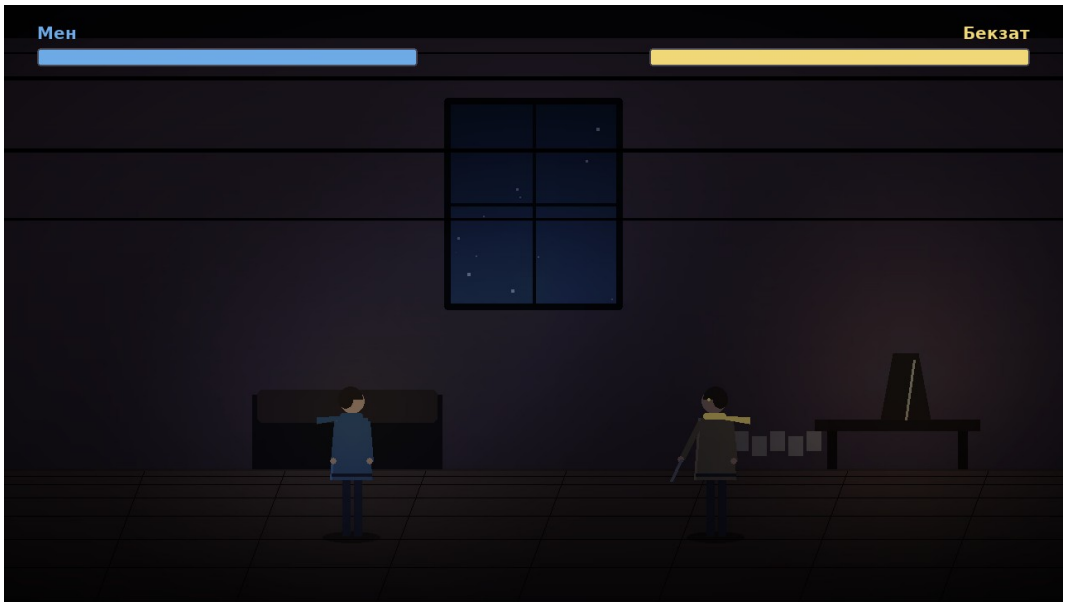



Ә.11-сурет — Адамды қайтару мүмкіндігі

**Түн қорытындысы және финал (Ә.12–Ә.14-суреттер).** Түннің барлық шайқасы біткен соң қорытынды экран шығады: әр досының портреті (қайтқаны — адам кейпінде, қалғаны — көлеңке), оның күйі және естелік қоры. Space пернесі келесі түнге өткізеді және ойынды автоматты түрде сақтайды. Үшінші түнде басты кейіпкердің өзі айнала бастайды: экран қараяды, интерфейс элементтері біртіндеп жоғалады, мәтіннің кейбір әріптері өшеді. Үшінші түннен кейін құтқарылғандар санына байланысты финал көрсетіледі.




Ә.12-сурет — Түн қорытындысы




Ә.13-сурет — Үшінші түн: басты кейіпкердің бүлінуі




Ә.14-сурет — Финал: төртеуі де құтқарылды

### 3 Жұмыс режимдерін таңдау және қауіпсіздік шаралары

– Толық экран және терезе режимі F11 пернесімен ауыстырылады; ойын терезеде де іске қосыла алады (--windowed).
– Дыбысты M пернесімен өшіруге және қосуға болады. Дыбыс құрылғысы жоқ болса, ойын үнсіз жұмыс істей береді.
– ESC пернесі мәзірге немесе алдыңғы экранға қайтарады; шайқастан шыққанда нәтиже сақталмайды.
– Автосақтау әр түн аяқталғанда жасалады. Файл Windows жүйесінде %APPDATA%\Kolenke\save.json жолында сақталады. Файлды қолмен өзгертпеңіз; ойынды қайта бастау үшін мәзірден «Жаңа ойын» таңдаңыз. Финалдан кейін сақтау жойылады.
– Ойында қысқа жарық жарқылдары мен экран дірілі бар. Жарыққа сезімтал адамдар ойнамас бұрын осы жайтты ескергені жөн.

### 4 Қателерді жою бойынша кеңестер

Ә.1-кесте — Жиі кездесетін жағдайлар және шешімдері

| Жағдай | Мүмкін себебі | Шешімі |
|---|---|---|
| Ойын баяу жұмыс істейді | Басқа ауыр бағдарламалар жұмыс істеп тұр немесе экран ажыратымдылығы өте жоғары | Басқа бағдарламаларды жабыңыз; F11 арқылы терезе режиміне өтіңіз |
| Дыбыс жоқ | Дыбыс өшірілген немесе құрылғы жоқ | M пернесін басыңыз; дыбыс құрылғысын қосыңыз; ойын үнсіз де жұмыс істейді |
| «Жалғастыру» пункті жоқ | Әлі сақтау жасалмаған | Алғашқы түнді аяқтаңыз: автосақтау түн соңында жасалады |
| Сақтау жүктелмейді | Файл бүлінген | %APPDATA%\Kolenke\save.json файлын жойып, «Жаңа ойын» таңдаңыз |

| Жағдай | Мүмкін себебі | Шешімі |
|---|---|---|
| Парирлеу шықпайды | L пернесі тым ерте немесе тым кеш басылған | L пернесін дайындық белгісі ақ түске боялған сәтте басыңыз (соққы тиюінен шамамен 0,13 с бұрын) |
| Дайындық белгісі көрінбейді | Диалогта сол детальді байқамағансыз | Сөйлесу кезінде кейіпкердің қолы мен қимылына қатысты сұрақтарды таңдаңыз; жазба журналда пайда болады |
| Қазақ әріптері дұрыс көрінбейді | assets/fonts қалтасы жоқ | Бағдарламаның толық қалтасын қалпына келтіріңіз немесе дайын Kolenke.exe қолданыңыз |
| Windows қорғаныс терезесі шығады | .exe файлына сандық қол қойылмаған | «Қосымша мәлімет» → «Бәрібір іске қосу» |

## ҚОСЫМША Б. БАҒДАРЛАМА ЛИСТИНГІ

Төменде бағдарламаның Python тіліндегі толық бастапқы коды берілген. Әр файлдың атауы мен қызметі көрсетілген; код түсініктемелермен (docstring және # түсініктемелер) жазылған. JSON деректері (night1.json, night2.json, night3.json, locations.json, moves.json, strings_kk.json, endings.json) мәтіндік файлдар ретінде kolenke/data қалтасында жеке сақталады.

**Файл: main.py**

*«Көлеңке» ойынының кіру нүктесі.*

```
"""«Көлеңке» ойынының кіру нүктесі.

python main.py            — толық экран
python main.py --windowed — терезеде
python main.py --selftest — бастапқы тексеру (терезесіз): ресурстар, сахналар, кадрлар
Браузерде (pygbag, WebAssembly) сол main.py asyncio арқылы іске қосылады.
"""
import argparse
import asyncio
import os
import sys
import traceback


def _selftest_body():
    from kolenke.core.game import Game
    from kolenke.scenes.fight import FightScene
    from kolenke.scenes.menu import MenuScene

    game = Game(fullscreen=False)
    game.push(MenuScene(game))
    for _ in range(30):
        game.step(1 / 60)
    game.scene.handle_action("confirm")       # «Жаңа ойын» -> ExploreScene
    for _ in range(30):
        game.step(1 / 60)
    game.push(FightScene(game, "darkhan"))
    for _ in range(60):
        game.step(1 / 60)
    return "SELFTEST OK: frames rendered, scenes=%d, audio=%s" % (len(game.scenes),
game.audio.ok)


def selftest():
    """Нәтиже консольге, ал KOLENKE_SELFTEST_OUT берілсе — файлға жазылады
    (консольсіз .exe-де тексеру үшін)."""
    os.environ.setdefault("SDL_VIDEODRIVER", "dummy")
    os.environ.setdefault("SDL_AUDIODRIVER", "dummy")
    out = os.environ.get("KOLENKE_SELFTEST_OUT")
    try:
        msg, code = _selftest_body(), 0
    except Exception:
        msg, code = "SELFTEST FAILED\n" + traceback.format_exc(), 1
    if out:
        with open(out, "w", encoding="utf-8") as f:
            f.write(msg)
    print(msg)
    return code


def main(argv=None):
```

```
    ap = argparse.ArgumentParser(description="Көлеңке")
    ap.add_argument("--windowed", action="store_true", help="терезеде іске қосу")
    ap.add_argument("--selftest", action="store_true", help="терезесіз тексеру")
    args = ap.parse_args(argv)
    if args.selftest:
        return selftest()
    from kolenke.core.game import Game
    from kolenke.scenes.menu import MenuScene

    game = Game(fullscreen=not args.windowed)
    game.push(MenuScene(game))
    game.run()
    return 0


async def main_web():
    from kolenke.core.game import Game
    from kolenke.scenes.menu import MenuScene

    game = Game(fullscreen=False)
    game.push(MenuScene(game))
    await game.run_async()


if sys.platform == "emscripten":          # pygbag: main.py модуль ретінде орындалады
    asyncio.run(main_web())
elif __name__ == "__main__":
    sys.exit(main())
```

### Файл: kolenke/core/assets.py

*Ресурстарды (шрифт, мәтін) жүктеу және кэштеу.*

```
"""Ресурстарды (шрифт, мәтін) жүктеу және кэштеу."""
import json
import sys
from pathlib import Path

import pygame

# PyInstaller (.exe) ішінде ресурстар sys._MEIPASS-та жатады
ROOT = Path(getattr(sys, "_MEIPASS", Path(__file__).resolve().parents[2]))
FONT_REGULAR = ROOT / "assets" / "fonts" / "DejaVuSans.ttf"
FONT_BOLD = ROOT / "assets" / "fonts" / "DejaVuSans-Bold.ttf"
STRINGS_PATH = ROOT / "kolenke" / "data" / "strings_kk.json"


def load_strings(path=STRINGS_PATH):
    """Интерфейс мәтіндерін JSON-нан оқиды (кодта қатар жоқ)."""
    with open(path, encoding="utf-8") as f:
        return json.load(f)


class Assets:
    """Шрифттер кэші: бір өлшем — бір Font объектісі."""

    def __init__(self):
        self._fonts = {}

    def font(self, size, bold=False):
        key = (size, bold)
        if key not in self._fonts:
            path = FONT_BOLD if bold else FONT_REGULAR
            self._fonts[key] = pygame.font.Font(str(path), size)
        return self._fonts[key]
```

**Файл: kolenke/core/audio.py**

*Дыбыс менеджері: синтезделген дыбыстар кэші, қайталанатын боран, М — өшіру.*

```
"""Дыбыс менеджері: синтезделген дыбыстар кэші, қайталанатын боран, М — өшіру.

Аудио құрылғы болмаса (немесе mixer қатесі) — ойын үнсіз жұмыс істей береді.
"""
import pygame

from kolenke.core.assets import ROOT
from kolenke.core.audio_synth import RATE, SOUNDS

AUDIO_DIR = ROOT / "assets" / "audio"   # алдын ала жасалған .wav (браузер нұсқасы үшін)

# Кейіпкер мотиві: шайқас/диалогта қайталанатын дыбыс (ТЗ: пышақ, барабан, тігін, метроном)
MOTIF = {"darkhan": "knife", "zhanel": "drum", "madina": "stitch", "bekzat": "tick"}
MOVE_SOUND = {"tock": "tock", "sweep": "tock"}   # метроном: ауыр қимыл — «так»


class Audio:
    def __init__(self, enabled=True):
        self.ok = False
        self.muted = False
        self._cache = {}
        self._loops = {}
        if not enabled:
            return
        try:
            if not pygame.mixer.get_init():
                pygame.mixer.init(frequency=RATE, size=-16, channels=1, buffer=512)
            pygame.mixer.set_num_channels(16)
            self.ok = True
        except pygame.error:
            self.ok = False

    def _sound(self, name):
        if name not in SOUNDS:
            raise KeyError(f"Белгісіз дыбыс: {name}")
        if name not in self._cache:
            ready = next((p for ext in (".ogg", ".wav") if (p := AUDIO_DIR / f"{name}
{ext}").exists()), None)
            if ready:                  # дайын файл (синтезді күтпей, жылдам жүктеледі)
                self._cache[name] = pygame.mixer.Sound(str(ready))
            else:
                self._cache[name] = pygame.mixer.Sound(buffer=SOUNDS[name]())
        return self._cache[name]

    def play(self, name, volume=1.0):
        if name not in SOUNDS:
            raise KeyError(f"Белгісіз дыбыс: {name}")
        if not self.ok or self.muted:
            return
        snd = self._sound(name)
        snd.set_volume(max(0.0, min(1.0, volume)))
        snd.play()

    def loop(self, name, volume=0.3):
        """Фондық қайталау (боран). Қайта шақырса дыбыс деңгейі ғана жаңарады."""
        if name not in SOUNDS:
            raise KeyError(f"Белгісіз дыбыс: {name}")
        if not self.ok:
            return
        snd = self._sound(name)
        snd.set_volume(0.0 if self.muted else volume)
        if name not in self._loops:
            self._loops[name] = (snd, volume)
```

```
            snd.play(loops=-1)
        else:
            self._loops[name] = (snd, volume)

    def toggle_mute(self):
        self.muted = not self.muted
        for snd, vol in self._loops.values():
            snd.set_volume(0.0 if self.muted else vol)
        return self.muted

    def motif_for(self, name, move=None):
        """Кейіпкер мотиві (метрономда қимылға қарай тік/так)."""
        if name == "bekzat" and move in MOVE_SOUND:
            return MOVE_SOUND[move]
        return MOTIF.get(name)
```

### Файл: kolenke/core/audio_synth.py

*Дыбыс синтезі: сыртқы файлсыз, математика арқылы (22050 Гц, 16 бит, моно).*

```
"""Дыбыс синтезі: сыртқы файлсыз, математика арқылы (22050 Гц, 16 бит, моно).

Әр функция bytes қайтарады — pygame.mixer.Sound(buffer=...) қабылдайды.
"""
import math
import random
from array import array

RATE = 22050
PEAK = 32767


def _pack(samples, gain=1.0):
    out = array("h")
    for v in samples:
        out.append(int(max(-1.0, min(1.0, v * gain)) * PEAK * 0.9))
    return out.tobytes()


def _n(duration):
    return int(RATE * duration)


def _noise(rng):
    return rng.uniform(-1.0, 1.0)


def hit(heavy=False, seed=1):
    rng = random.Random(seed)
    dur = 0.34 if heavy else 0.2
    f0 = 70 if heavy else 120
    return _pack((math.sin(2 * math.pi * f0 * (1 - 0.5 * i / _n(dur)) * i / RATE)
                  * math.exp(-i / RATE * (14 if heavy else 26)) * 0.9
                  + _noise(rng) * math.exp(-i / RATE * 45) * 0.7
                  for i in range(_n(dur))))


def block(seed=2):
    rng = random.Random(seed)
    return _pack((sum(math.sin(2 * math.pi * f * i / RATE) for f in (610, 905, 1230)) / 3
                  * math.exp(-i / RATE * 14) * 0.7 + _noise(rng) * math.exp(-i / RATE * 90)
* 0.5
                  for i in range(_n(0.28))))


def parry(seed=3):
    rng = random.Random(seed)
```

```
    dur = 0.8
    return _pack((sum(math.sin(2 * math.pi * f * i / RATE) * a
                      for f, a in ((1240, 1.0), (1860, 0.7), (2480, 0.5), (3100, 0.3))) /
2.5
                 * math.exp(-i / RATE * 5) * 0.8
                 + math.sin(2 * math.pi * 58 * i / RATE) * math.exp(-i / RATE * 12) * 0.7
                 + _noise(rng) * math.exp(-i / RATE * 60) * 0.4
                 for i in range(_n(dur))))


def whoosh(rise=True, dur=0.28, seed=4):
    rng = random.Random(seed)
    n = _n(dur)
    lp = 0.0
    out = []
    for i in range(n):
        k = i / n
        env = math.sin(math.pi * k) ** 2
        a = 0.05 + 0.35 * (k if rise else 1 - k)
        lp += (_noise(rng) - lp) * a
        out.append(lp * env * 1.6)
    return _pack(out)


def chime(notes=(523.25, 659.25, 783.99), seed=5):
    n = _n(1.1)
    out = []
    for i in range(n):
        t = i / RATE
        v = 0.0
        for k, f in enumerate(notes):
            tt = t - k * 0.09
            if tt > 0:
                v += math.sin(2 * math.pi * f * tt) * math.exp(-tt * 3.2)
        out.append(v / len(notes) * 1.3)
    return _pack(out)


def boom(seed=6):
    rng = random.Random(seed)
    return _pack((math.sin(2 * math.pi * 48 * (1 - 0.3 * i / _n(1.0)) * i / RATE)
                  * math.exp(-i / RATE * 4) * 0.95
                  + _noise(rng) * math.exp(-i / RATE * 10) * 0.4 for i in range(_n(1.0))))


def swell(seed=7):
    """Көлеңке жұту кезіндегі баяу төмен дыбыс."""
    rng = random.Random(seed)
    n = _n(1.4)
    lp = 0.0
    out = []
    for i in range(n):
        k = i / n
        env = math.sin(math.pi * k) ** 1.5
        lp += (_noise(rng) - lp) * 0.04
        out.append((lp * 2.2 + math.sin(2 * math.pi * 55 * i / RATE) * 0.5
                    + math.sin(2 * math.pi * 82 * i / RATE) * 0.3) * env * 0.8)
    return _pack(out)


def tick(low=False, seed=8):
    """Метроном: жоғары «тік» немесе төмен «так»."""
    rng = random.Random(seed)
    f = 1050 if low else 1750
    return _pack((math.sin(2 * math.pi * f * i / RATE) * math.exp(-i / RATE * 70) * 0.8
```

```
                + _noise(rng) * math.exp(-i / RATE * 220) * 0.5 for i in
range(_n(0.12))))


def drum(seed=9):
    rng = random.Random(seed)
    n = _n(0.4)
    return _pack((math.sin(2 * math.pi * (95 - 45 * min(1.0, i / (RATE * 0.12))) * i /
RATE)
                  * math.exp(-i / RATE * 12) * 0.95
                  + _noise(rng) * math.exp(-i / RATE * 60) * 0.35 for i in range(n)))


def knife(seed=10):
    rng = random.Random(seed)
    n = _n(0.45)
    out = []
    for i in range(n):
        t = i / RATE
        scrape = _noise(rng) * min(1.0, t * 12) * math.exp(-t * 14) * 0.5
        ping = (math.sin(2 * math.pi * 2350 * t) + 0.6 * math.sin(2 * math.pi * 3480 *
t)) / 1.6
        out.append(scrape + ping * math.exp(-t * 9) * 0.55 * (1 if t > 0.03 else 0))
    return _pack(out)


def stitch(seed=11):
    """Тігін машинасы: тез-тез шертулер."""
    rng = random.Random(seed)
    n = _n(0.55)
    out = [0.0] * n
    step = _n(0.045)
    for c in range(12):
        s = c * step
        for j in range(_n(0.02)):
            if s + j < n:
                out[s + j] += (_noise(rng) * 0.6 + math.sin(2 * math.pi * 1500 * j / RATE)
* 0.4) \
                    * math.exp(-j / RATE * 180)
    return _pack(out)


def click(seed=12):
    return _pack((math.sin(2 * math.pi * 880 * i / RATE) * math.exp(-i / RATE * 120) * 0.6
                  for i in range(_n(0.06))))


def blizzard(duration=4.0, seed=13):
    """Үзіліссіз қайталанатын боран шуы (соңы басымен біріктіріледі)."""
    rng = random.Random(seed)
    n, m = _n(duration), _n(0.25)
    raw = []
    lp1 = lp2 = 0.0
    for i in range(n + m):
        lp1 += (_noise(rng) - lp1) * 0.10
        lp2 += (_noise(rng) - lp2) * 0.012
        gust = 0.65 + 0.35 * math.sin(2 * math.pi * 2 * i / (RATE * duration))
        raw.append((lp1 * 0.9 + lp2 * 4.0) * gust * 0.55)
    out = raw[:n]
    for i in range(m):      # тігіссіз қайталау үшін кросфейд
        w = i / m
        out[i] = raw[i] * w + raw[n + i] * (1 - w)
    return _pack(out)


SOUNDS = {
```

```
    "hit": lambda: hit(False), "heavy_hit": lambda: hit(True), "block": block,
    "parry": parry, "whoosh": whoosh, "whoosh_light": lambda: whoosh(True, 0.16),
    "chime": chime, "boom": boom, "swell": swell, "tick": lambda: tick(False),
    "tock": lambda: tick(True), "drum": drum, "knife": knife, "stitch": stitch,
    "click": click, "blizzard": blizzard,
}
```

### Файл: kolenke/core/game.py

*Негізгі ойын циклі: 60 FPS, сахналар стегі, толық экран.*

```
"""Негізгі ойын циклі: 60 FPS, сахналар стегі, толық экран."""
import sys

import pygame

from kolenke.combat.fighter import load_combat_data
from kolenke.core.audio import Audio
from kolenke.core.assets import Assets, load_strings
from kolenke.core.input import InputBuffer, action_for_key
from kolenke.story.flags import Flags
from kolenke.story.catalog import load_catalog
from kolenke.story.journal import Journal

LOGICAL_SIZE = (1280, 720)  # ішкі ажыратымдылық, экранға масштабталады
FPS = 60
IS_WEB = sys.platform == "emscripten"   # pygbag: браузерде жұмыс істеп тұр
FADE_IN = 0.45


class Game:
    def __init__(self, fullscreen=True):
        pygame.init()
        self.fullscreen = fullscreen
        self.screen = self._make_screen()
        pygame.display.set_caption("Көлеңке")
        self.clock = pygame.time.Clock()
        self.assets = Assets()
        self.audio = Audio()
        self.strings = load_strings()
        self.fight_moves = load_combat_data()["moves"]
        self.input = InputBuffer()
        self.scenes = []
        self.held = set()
        self.fade_in = 0.0     # жаңа сахнаға өткенде жұмсақ ашылу (секунд)
        self._veil = pygame.Surface(LOGICAL_SIZE)
        self.running = True
        self.new_game_state()

    def start_night(self, n):
        self.night = n
        self.corruption = 0.25 if n == 3 else 0.0

    def tick_corruption(self, dt):
        """3-түнде уақыт өткен сайын кейіпкер айнала түседі."""
        if self.night == 3:
            self.corruption = min(0.92, self.corruption + 0.004 * dt)

    def new_game_state(self):
        """Жаңа ойын: флагтар мен журнал тазаланады."""
        self.flags = Flags()
        self.journal = Journal(load_catalog())
        self.reserve = 100   # естелік қоры: адамды қайтарған сайын азаяды
        self.night = 1       # қазіргі түн (1–3)
        self.corruption = 0.0  # 3-түнде басты кейіпкердің өзі айнала бастайды (0..1)

    def _make_screen(self):
```

```
    if IS_WEB:        # браузерде кенепті HTML өзі масштабтайды
        return pygame.display.set_mode(LOGICAL_SIZE)
    flags = pygame.SCALED | (pygame.FULLSCREEN if self.fullscreen else 0)
    return pygame.display.set_mode(LOGICAL_SIZE, flags)

def toggle_fullscreen(self):
    if IS_WEB:
        return
    self.fullscreen = not self.fullscreen
    pygame.display.toggle_fullscreen()

# --- сахналар стегі ---
@property
def scene(self):
    return self.scenes[-1] if self.scenes else None

def push(self, scene):
    self.scenes.append(scene)
    scene.on_enter()
    self.fade_in = FADE_IN

def pop(self):
    if self.scenes:
        self.scenes.pop().on_exit()
    if self.scene:
        self.scene.on_resume()

def pop_to_menu(self):
    """Мәзірге оралу (стектегі бірінші сахна)."""
    while len(self.scenes) > 1:
        self.scenes.pop().on_exit()
    if self.scene:
        self.scene.on_resume()

def quit(self):
    self.running = False

# --- цикл ---
def is_held(self, action):
    """Перне қазір басулы ма (үздіксіз қозғалыс үшін)."""
    return action in self.held

def process_events(self):
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            self.quit()
        elif event.type == pygame.KEYUP:
            action = action_for_key(event.key)
            if action:
                self.held.discard(action)
        elif event.type == pygame.KEYDOWN:
            if event.key == pygame.K_F11:
                self.toggle_fullscreen()
                continue
            action = action_for_key(event.key)
            if action == "mute":
                self.audio.toggle_mute()
                continue
            if action:
                self.held.add(action)
                self.input.push(action)
                if self.scene:
                    self.scene.handle_action(action)

def step(self, dt):
    """Бір кадр: оқиғалар, логика, сызу. Тестілеуге ыңғайлы бөлек."""
```

```
        self.process_events()
        if self.scene:
            self.scene.update(dt)
        self.input.tick()
        self.screen.fill((0, 0, 0))
        if self.scene:
            self.scene.draw(self.screen)
        if self.fade_in > 0:                        # қараңғыдан жұмсақ ашылу
            self.fade_in = max(0.0, self.fade_in - dt)
            self._veil.set_alpha(int(255 * (self.fade_in / FADE_IN) ** 1.5))
            self.screen.blit(self._veil, (0, 0))
        pygame.display.flip()

    def run(self):
        while self.running and self.scenes:
            dt = self.clock.tick(FPS) / 1000.0
            self.step(dt)
        pygame.quit()

    async def run_async(self):
        """Браузер нұсқасы (pygbag): әр кадрдан кейін басқаруды браузерге қайтарады."""
        import asyncio
        import time
        frames, t0 = 0, time.perf_counter()
        while self.running and self.scenes:
            dt = self.clock.tick(FPS) / 1000.0
            self.step(dt)
            frames += 1
            if time.perf_counter() - t0 >= 5.0:      # браузер консолінде FPS көрінеді
                print(f"FPS {frames / (time.perf_counter() - t0):.1f}")
                frames, t0 = 0, time.perf_counter()
            await asyncio.sleep(0)
        pygame.quit()
```

### Файл: kolenke/core/input.py

*Пернелерді әрекеттерге сәйкестендіру және енгізу буфері.*

```
"""Пернелерді әрекеттерге сәйкестендіру және енгізу буфері."""
import pygame

# ТЗ бойынша басқару: A/D/←→ қозғалыс, E әрекет, Space реплика, 1–4 таңдау,
# Tab журнал, J/K/L жекпе-жек, ESC мәзір.
KEYMAP = {
    pygame.K_a: "left", pygame.K_LEFT: "left",
    pygame.K_d: "right", pygame.K_RIGHT: "right",
    pygame.K_e: "interact",
    pygame.K_SPACE: "next",
    pygame.K_1: "choice1", pygame.K_2: "choice2",
    pygame.K_3: "choice3", pygame.K_4: "choice4",
    pygame.K_TAB: "journal",
    pygame.K_j: "light", pygame.K_k: "heavy", pygame.K_l: "block",
    pygame.K_ESCAPE: "menu",
    pygame.K_UP: "up", pygame.K_DOWN: "down",
    pygame.K_RETURN: "confirm", pygame.K_KP_ENTER: "confirm",
    pygame.K_m: "mute",
}


def action_for_key(key):
    """Перне коды бойынша әрекет атауы (жоқ болса None)."""
    return KEYMAP.get(key)


class InputBuffer:
    """Соңғы N кадр ішіндегі басылған әрекеттерді сақтайды.
```

```
    Жекпе-жекте соққыны алдын ала басуға мүмкіндік береді (буфер ~6 кадр).
    """

    def __init__(self, window=6):
        self.window = window
        self.frame = 0
        self._events = []  # (кадр, әрекет)

    def push(self, action):
        self._events.append((self.frame, action))

    def tick(self):
        """Кадр санағышын арттырып, ескірген оқиғаларды тастайды."""
        self.frame += 1
        limit = self.frame - self.window
        self._events = [e for e in self._events if e[0] >= limit]

    def consume(self, action):
        """Әрекет буферде болса — алып тастап True қайтарады."""
        for i, (_, a) in enumerate(self._events):
            if a == action:
                del self._events[i]
                return True
        return False
```

### Файл: kolenke/core/save.py

*Сақтау/жүктеу: JSON. Түн аяқталғанда автосақтау.*

```
"""Сақтау/жүктеу: JSON. Түн аяқталғанда автосақтау."""
import json
import os
from pathlib import Path

from kolenke.story.catalog import load_catalog
from kolenke.story.flags import Flags
from kolenke.story.journal import Journal

VERSION = 1


def save_path():
    """KOLENKE_SAVE_DIR немесе %APPDATA%/Kolenke (Windows) / ~/.kolenke."""
    base = os.environ.get("KOLENKE_SAVE_DIR")
    if base:
        return Path(base) / "save.json"
    root = os.environ.get("APPDATA")
    return (Path(root) / "Kolenke" if root else Path.home() / ".kolenke") / "save.json"


def has_save(path=None):
    return (path or save_path()).exists()


def save_game(game, path=None):
    path = path or save_path()
    path.parent.mkdir(parents=True, exist_ok=True)
    data = {
        "version": VERSION,
        "night": game.night,
        "flags": game.flags.to_dict(),
        "journal": game.journal.to_list(),
        "reserve": game.reserve,
        "corruption": game.corruption,
    }
    tmp = path.with_suffix(".tmp")
    with open(tmp, "w", encoding="utf-8") as f:
```

```
        json.dump(data, f, ensure_ascii=False, indent=2)
    tmp.replace(path)   # жартылай жазылған файл қалмауы үшін


def load_game(game, path=None):
    """Сақтауды оқиды. Бүлінген/жоқ файл болса False қайтарады, ойын күйі өзгермейді."""
    path = path or save_path()
    try:
        with open(path, encoding="utf-8") as f:
            data = json.load(f)
        if data.get("version") != VERSION:
            return False
        catalog = load_catalog()
        ids = [i for i in data["journal"] if i in catalog]
        night = int(data["night"])
        if night not in (1, 2, 3):
            return False
        flags = Flags.from_dict(data["flags"])
        reserve = int(data["reserve"])
        corruption = float(data["corruption"])
    except (OSError, ValueError, KeyError, TypeError):
        return False
    game.flags = flags
    game.journal = Journal.from_list(catalog, ids)
    game.reserve = reserve
    game.night = night
    game.corruption = corruption
    return True


def delete_save(path=None):
    try:
        (path or save_path()).unlink()
    except OSError:
        pass
```

### Файл: kolenke/core/scene.py

*Сахналардың базалық класы.*

```
"""Сахналардың базалық класы."""


class Scene:
    """Барлық сахналар (мәзір, диалог, жекпе-жек, финал) осыдан туады."""

    def __init__(self, game):
        self.game = game

    def on_enter(self):
        pass

    def on_exit(self):
        pass

    def on_resume(self):
        """Үстіндегі сахна жабылғанда шақырылады."""

    def handle_action(self, action):
        """Әрекет келгенде шақырылады (KEYMAP бойынша)."""

    def update(self, dt):
        """Логиканы жаңарту; dt — секунд."""

    def draw(self, surface):
        """Кадрды сызу."""
```

**Файл: kolenke/core/text.py**

*Мәтінді енге қарай жолдарға бөлу.*

```
"""Мәтінді енге қарай жолдарға бөлу."""


def wrap_text(font, text, max_width):
    lines, line = [], ""
    for word in text.split():
        candidate = f"{line} {word}".strip()
        if font.size(candidate)[0] <= max_width or not line:
            line = candidate
        else:
            lines.append(line)
            line = word
    if line:
        lines.append(line)
    return lines
```

**Файл: kolenke/story/catalog.py**

*Барлық түндердің естеліктер каталогын жинау (сақтаудан жүктегенде қажет).*

```
"""Барлық түндердің естеліктер каталогын жинау (сақтаудан жүктегенде қажет)."""
import json

from kolenke.core.assets import ROOT

DATA_DIR = ROOT / "kolenke" / "data"


def load_catalog(nights=(1, 2, 3)):
    catalog = {}
    for n in nights:
        with open(DATA_DIR / f"night{n}.json", encoding="utf-8") as f:
            catalog.update(json.load(f).get("memories", {}))
    return catalog
```

**Файл: kolenke/story/flags.py**

*Ойыншы таңдауларының флагтары: диалог пен жекпе-жекті байланыстырады.*

```
"""Ойыншы таңдауларының флагтары: диалог пен жекпе-жекті байланыстырады."""


class Flags:
    def __init__(self, data=None):
        self._data = dict(data or {})

    def set(self, name, value=True):
        self._data[name] = value

    def get(self, name, default=False):
        return self._data.get(name, default)

    def check(self, requires):
        """requires — {флаг: күтілетін мән}; бәрі сәйкес болса True."""
        return all(self.get(k) == v for k, v in (requires or {}).items())

    def to_dict(self):
        return dict(self._data)

    @classmethod
    def from_dict(cls, data):
        return cls(data)
```

**Файл: kolenke/story/graph.py**

*Диалог графы: түйіндер мен таңдау бұтақтары (JSON-нан жүктеледі).*

```
"""Диалог графы: түйіндер мен таңдау бұтақтары (JSON-нан жүктеледі).

Түйін өрістері: speaker, text, next | choices | end,
set ({флаг: мән}), memory (журналға жазба id).
Таңдау өрістері: text, next, set, memory, requires ({флаг: мән}).
"""
import json

VALID_STATES = ("human", "fading", "shadow")
VALID_BG = ("darkhan_room", "girls_room", "bekzat_room")
OFFSCREEN = ("narrator", "me")   # экранда портреті жоқ


class GraphError(Exception):
    pass


class DialogueGraph:
    def __init__(self, data):
        self.id = data.get("id", "")
        self.start = data["start"]
        self.nodes = data["nodes"]
        self.speakers = data.get("speakers", {})
        self.memories = data.get("memories", {})
        self.bg = data.get("bg", "darkhan_room")
        self.validate()

    @property
    def cast(self):
        """Экранда көрінетін кейіпкерлер (сөйлесушілер, нарратор мен басты кейіпкерден
басқа)."""
        return [k for k in self.speakers if k not in OFFSCREEN]

    @classmethod
    def load(cls, path):
        with open(path, encoding="utf-8") as f:
            return cls(json.load(f))

    def targets(self, node):
        """Түйіннен шығатын барлық өтулер."""
        out = []
        if "next" in node:
            out.append(node["next"])
        out += [c["next"] for c in node.get("choices", [])]
        return out

    def validate(self):
        if self.start not in self.nodes:
            raise GraphError(f"Бастапқы түйін жоқ: {self.start}")
        if self.bg not in VALID_BG:
            raise GraphError(f"Белгісіз фон: {self.bg}")
        for nid, node in self.nodes.items():
            ends = [k for k in ("next", "choices", "end") if k in node]
            if len(ends) != 1:
                raise GraphError(f"{nid}: next/choices/end біреуі ғана болу керек")
            for t in self.targets(node):
                if t not in self.nodes:
                    raise GraphError(f"{nid}: белгісіз түйінге өту: {t}")
            for mem in self._memory_refs(node):
                if mem not in self.memories:
                    raise GraphError(f"{nid}: белгісіз естелік: {mem}")
            for spec in [node] + list(node.get("choices", [])):
                for who, st in spec.get("states", {}).items():
```

```
                    if who not in self.speakers or who in OFFSCREEN:
                        raise GraphError(f"{nid}: күй берілген кейіпкер экранда жоқ:
{who}")
                    if st not in VALID_STATES:
                        raise GraphError(f"{nid}: белгісіз күй: {st}")
            if node.get("speaker") and node["speaker"] not in self.speakers:
                raise GraphError(f"{nid}: белгісіз кейіпкер: {node['speaker']}")
        unreachable = set(self.nodes) - self.reachable()
        if unreachable:
            raise GraphError(f"Қол жетпейтін түйіндер: {sorted(unreachable)}")

    def _memory_refs(self, node):
        refs = [node["memory"]] if "memory" in node else []
        refs += [c["memory"] for c in node.get("choices", []) if "memory" in c]
        return refs

    def reachable(self):
        seen, stack = set(), [self.start]
        while stack:
            nid = stack.pop()
            if nid in seen:
                continue
            seen.add(nid)
            stack += self.targets(self.nodes[nid])
        return seen


class DialogueRunner:
    """Графпен жүру: түйінге кіргенде флагтар мен журнал жаңарады."""

    def __init__(self, graph, flags, journal):
        self.graph = graph
        self.flags = flags
        self.journal = journal
        self.node_id = None
        self.states = {}   # кейіпкер -> human | fading | shadow (түйіндер ауыстырады)
        self._enter(graph.start)

    @property
    def node(self):
        return self.graph.nodes[self.node_id]

    @property
    def finished(self):
        return "end" in self.node

    def _apply(self, spec):
        self.states.update(spec.get("states", {}))
        for k, v in spec.get("set", {}).items():
            self.flags.set(k, v)
        if "memory" in spec:
            self.journal.add(spec["memory"])

    def _enter(self, node_id):
        self.node_id = node_id
        self._apply(self.node)

    def choices(self):
        """Қазіргі шарттарға сай көрінетін таңдаулар (түпнұсқа индекс, мәтін)."""
        return [
            (i, c) for i, c in enumerate(self.node.get("choices", []))
            if self.flags.check(c.get("requires"))
        ]

    @property
    def has_choices(self):
```

```
        return "choices" in self.node

    def advance(self):
        """Таңдауы жоқ түйіннен келесіге өту."""
        if self.finished or self.has_choices:
            return False
        self._enter(self.node["next"])
        return True

    def choose(self, visible_index):
        """visible_index — көрінетін тізімдегі 0-негізді нөмір."""
        visible = self.choices()
        if not self.has_choices or not 0 <= visible_index < len(visible):
            return False
        choice = visible[visible_index][1]
        self._apply(choice)
        self._enter(choice["next"])
        return True
```

### Файл: kolenke/story/journal.py

*Естеліктер журналы (Tab): диалогта табылған детальдар.*

```
"""Естеліктер журналы (Tab): диалогта табылған детальдар."""


class Journal:
    def __init__(self, catalog):
        """catalog — {id: {"who": кейіпкер, "text": мәтін}} (JSON-нан)."""
        self.catalog = catalog
        self._ids = []

    def add(self, memory_id):
        """Жазба қосады; қайта қосылмайды. Жаңа болса True қайтарады."""
        if memory_id not in self.catalog:
            raise KeyError(f"Белгісіз естелік: {memory_id}")
        if memory_id in self._ids:
            return False
        self._ids.append(memory_id)
        return True

    def has(self, memory_id):
        return memory_id in self._ids

    def entries(self):
        return [(i, self.catalog[i]) for i in self._ids]

    def __len__(self):
        return len(self._ids)

    def to_list(self):
        return list(self._ids)

    @classmethod
    def from_list(cls, catalog, ids):
        j = cls(catalog)
        for i in ids:
            j.add(i)
        return j
```

### Файл: kolenke/explore/locations.py

*Жатақхана локациялары: JSON-нан жүктеледі және тексеріледі.*

```
"""Жатақхана локациялары: JSON-нан жүктеледі және тексеріледі."""
import json
from dataclasses import dataclass, field
```

```
class LocationError(Exception):
    pass


@dataclass
class Obj:
    id: str
    kind: str          # "door" | "npc"
    x: int
    w: int
    label: str         # strings_kk.json кілті
    to: str = ""
    spawn: int = 0
    dialogue: str = ""
    night: int = 0          # 0 — әрқашан белсенді; N — тек N-түнде
    who: str = ""           # NPC кейіпкер id (флагтар: <who>_saved/_killed)
    fight: str = ""         # "darkhan" немесе "@флаг1,флаг2" (ретпен шайқас)
    accent: tuple = (230, 230, 230)

    @property
    def done_flag(self):
        """Диалог аяқталғанда қойылатын флаг атауы."""
        return self.dialogue.replace(".json", "") + "_talk_done"

    def active(self, night):
        return self.night in (0, night)

    def enemies(self, flags):
        """Осы NPC-мен өтетін шайқастар тізімі (диалогтан кейін анықталады)."""
        if not self.fight:
            return []
        if self.fight.startswith("@"):
            names = [flags.get(k, "") for k in self.fight[1:].split(",")]
            return [n for n in names if n]
        return [self.fight]


@dataclass
class Location:
    id: str
    width: int
    name: str
    start: int
    objects: list = field(default_factory=list)


def load_locations(path):
    with open(path, encoding="utf-8") as f:
        raw = json.load(f)
    locs = {}
    for lid, d in raw.items():
        objs = []
        for o in d["objects"]:
            o = dict(o)
            if "accent" in o:
                o["accent"] = tuple(o["accent"])
            objs.append(Obj(**o))
        locs[lid] = Location(lid, d["width"], d["name"], d["start"], objs)
    _validate(locs)
    return locs


def _validate(locs):
    for loc in locs.values():
        for o in loc.objects:
```

```
            if not 0 <= o.x <= loc.width:
                raise LocationError(f"{loc.id}/{o.id}: x локациядан тыс")
            if o.kind == "door":
                if o.to not in locs:
                    raise LocationError(f"{loc.id}/{o.id}: белгісіз есік: {o.to}")
                if not 0 <= o.spawn <= locs[o.to].width:
                    raise LocationError(f"{loc.id}/{o.id}: spawn локациядан тыс")
            elif o.kind == "npc":
                if not o.dialogue:
                    raise LocationError(f"{loc.id}/{o.id}: диалог файлы жоқ")
            else:
                raise LocationError(f"{loc.id}/{o.id}: белгісіз түр: {o.kind}")
```

### Файл: kolenke/explore/player.py

*Басты кейіпкердің қозғалысы (A/D немесе ← →) жылдамдану мен тежелумен.*

```
"""Басты кейіпкердің қозғалысы (A/D немесе ←→) жылдамдану мен тежелумен."""
import math

SPEED = 300          # px/сек, ең жоғары жылдамдық
ACCEL = 11.0         # жылдамдану/тежелу қарқыны (1/сек)
MARGIN = 50          # локация шетінен қашықтық


class Player:
    def __init__(self, x):
        self.x = float(x)
        self.vx = 0.0
        self.facing = 1       # 1 — оңға, -1 — солға
        self.walk_phase = 0.0

    @property
    def moving(self):
        return abs(self.vx) > 25

    def update(self, dt, left, right, width):
        """left/right — қазір басулы ма. Локация шегінде қалады."""
        direction = (1 if right else 0) - (1 if left else 0)
        if direction:
            self.facing = direction
        target = direction * SPEED
        self.vx += (target - self.vx) * min(1.0, ACCEL * dt)
        self.x += self.vx * dt
        if self.moving:
            self.walk_phase += dt * 10 * abs(self.vx) / SPEED
        else:
            self.walk_phase = 0.0
        if self.x < MARGIN or self.x > width - MARGIN:
            self.x = max(MARGIN, min(width - MARGIN, self.x))
            self.vx = 0.0

    @property
    def swing(self):
        """Аяқ ұлғаюы -1..1 (анимация үшін)."""
        return math.sin(self.walk_phase) if self.moving else 0.0
```

### Файл: kolenke/combat/ai_scripted.py

*Қарапайым скриптті қарсылас (5-кезеңде бейімделетін ИИ-мен ауысады).*

```
"""Қарапайым скриптті қарсылас (5-кезеңде бейімделетін ИИ-мен ауысады)."""
import random


class ScriptedAI:
    """Жақындайды, қашықтықта соғады. Соққы аралығы — кездейсоқ күту."""
```

```
    def __init__(self, seed=1, wait=(35, 80)):
        self.rng = random.Random(seed)
        self.wait_range = wait
        self.wait = self.rng.randint(*wait)

    def decide(self, fight, me, opp):
        dist = abs(opp.x - me.x)
        walk = 0
        cmd = None
        if me.can_act:
            reach = max(m["reach"][0] + m["reach"][1] for m in me.moves.values())
            if dist > reach - 15:
                walk = 1 if opp.x > me.x else -1
            elif self.wait > 0:
                self.wait -= 1
            else:
                cmd = self.rng.choice(list(me.moves))
                self.wait = self.rng.randint(*self.wait_range)
        return walk, cmd, False
```

### Файл: kolenke/combat/fight.py

*Жекпе-жек ядросы: кадр дәлдігіндегі есептеу, hitstop, парирлеу, буфер, естеліктер.*

```
"""Жекпе-жек ядросы: кадр дәлдігіндегі есептеу, hitstop, парирлеу, буфер, естеліктер.

Fight.tick() — бір логикалық кадр. Сызу мен баяулау (slow-mo) сахнада.
"""
from dataclasses import dataclass

from kolenke.combat.fighter import Fighter, load_combat_data
from kolenke.combat.hitbox import GROUND, hitbox, hurtbox
from kolenke.core.input import InputBuffer

STAGGER_BONUS = 1.5       # парирленген жауға қосымша зиян
PARRY_MEMORY_GAIN = 15    # сәтті парирлеу -> естелік жолағы
MEMORY_COST = 33          # бір естелік слотының бағасы
HEAL_AMOUNT = 25
FOCUS_BASE = 180          # кадр (3 с); әр естелік жазбасы +60
FOCUS_PARRY_BONUS = 6     # парирлеу терезесіне қосымша кадр
RESTORE_HP_FRACTION = 0.35
UNKNOWN_TELL_SPEEDUP = 0.7   # белгісіз дайындық қимылы 30% жылдамырақ


@dataclass
class Event:
    type: str            # hit | block | parry | ko | mem_heal | mem_focus | restore
    attacker: str
    defender: str
    x: float
    y: float
    damage: int = 0
    move: str = ""
    heavy: bool = False


class PlayerController:
    """Ойыншы енгізуі: буфер (6 кадр) + басулы блок. Тек әрекет ете алғанда оқиды."""

    ORDER = ("mem3", "mem2", "mem1", "block", "heavy", "light")

    def __init__(self, buffer, is_held):
        self.buffer = buffer
        self.is_held = is_held

    def decide(self, fight, me, opp):
        walk = (1 if self.is_held("right") else 0) - (1 if self.is_held("left") else 0)
```

```
        cmd = None
        if me.can_act:
            for c in self.ORDER:
                if self.buffer.consume(c):
                    cmd = c
                    break
        return walk, cmd, self.is_held("block")


class Fight:
    def __init__(self, enemy_name="darkhan", enemy_controller=None, data=None,
                 player_controller=None, known_tells=None, start_memory=0,
                 enemy_hp_mult=1.0, entries=0):
        """known_tells: жау қимылдарының танылған id-лері (None — барлығы белгілі).
        entries — журналдағы осы кейіпкер туралы жазба саны (естеліктерге әсер етеді)."""
        self.data = data or load_combat_data()
        self.player = Fighter("player", 420, 1, self.data)
        self.enemy = Fighter(enemy_name, 860, -1, self.data)
        self.entries = entries
        self.player.memory = max(0, min(self.player.max_memory, int(start_memory)))
        self.enemy.max_hp = self.enemy.hp = int(self.enemy.max_hp * enemy_hp_mult)
        if known_tells is not None:
            for mid, m in list(self.enemy.moves.items()):
                if m.get("tell_flag") and mid not in known_tells:
                    m = dict(m)
                    m["startup"] = max(10, int(m["startup"] * UNKNOWN_TELL_SPEEDUP))
                    self.enemy.moves[mid] = m
        self.buffer = InputBuffer(window=6)
        self.controllers = {
            self.player: player_controller or PlayerController(self.buffer, lambda a:
False),
            self.enemy: enemy_controller,
        }
        self.hitstop = 0
        self.frame_no = 0
        self.focus_left = 0
        self.over = False
        self.winner = None
        self.outcome = None   # "kill" | "restore" | "lose"

    def push_input(self, action):
        self.buffer.push(action)

    # --- естеліктер ---
    @property
    def can_restore(self):
        e = self.enemy
        low = e.hp <= e.max_hp * RESTORE_HP_FRACTION or e.state == "stagger"
        return (self.entries >= 1 and low and e.alive and not self.over
                and self.player.memory >= MEMORY_COST)

    def usable(self, slot):
        p = self.player
        if self.over or p.memory < MEMORY_COST:
            return False
        if slot == 1:
            return p.hp < p.max_hp
        if slot == 2:
            return self.focus_left == 0
        return self.can_restore

    def _use_memory(self, slot):
        p, e = self.player, self.enemy
        if not self.usable(slot):
            return None
        p.memory -= MEMORY_COST
```

```
        if slot == 1:
            p.hp = min(p.max_hp, p.hp + HEAL_AMOUNT)
            return Event("mem_heal", p.name, p.name, p.x, GROUND - 100, HEAL_AMOUNT)
        if slot == 2:
            self.focus_left = FOCUS_BASE + 60 * self.entries
            p.parry_frames += FOCUS_PARRY_BONUS
            return Event("mem_focus", p.name, p.name, p.x, GROUND - 100)
        self.over, self.winner, self.outcome = True, p, "restore"
        return Event("restore", p.name, e.name, e.x, GROUND - 100)

    # --- цикл ---
    def tick(self):
        """Бір логикалық кадр. Қайтарады: осы кадрдағы оқиғалар тізімі."""
        if self.over:
            return []
        if self.hitstop > 0:           # hitstop: уақыт қатады, енгізу буфері да
            self.hitstop -= 1
            return []
        self.frame_no += 1
        self.buffer.tick()
        events = []
        if self.focus_left:
            self.focus_left -= 1
            if not self.focus_left:
                self.player.parry_frames -= FOCUS_PARRY_BONUS
        for me, opp in ((self.player, self.enemy), (self.enemy, self.player)):
            ctrl = self.controllers.get(me)
            walk, cmd, hold = ctrl.decide(self, me, opp) if ctrl else (0, None, False)
            if cmd and cmd.startswith("mem"):
                ev = self._use_memory(int(cmd[3]))
                if ev:
                    events.append(ev)
                cmd = None
            if self.over:
                return events
            if me.alive:
                me.update(walk, cmd, hold, opp.x)
        self._separate()
        hits = [ev for ev in (self._resolve(self.player, self.enemy),
                              self._resolve(self.enemy, self.player)) if ev]
        events += hits
        if hits:
            self.hitstop = max(self._stop_for(e) for e in hits)
        for e in (self.player, self.enemy):
            if not e.alive and not self.over:
                self.over = True
                self.winner = self.player if e is self.enemy else self.enemy
                self.outcome = "kill" if e is self.enemy else "lose"
                events.append(Event("ko", self.winner.name, e.name, e.x, GROUND - 90))
        return events

    def _stop_for(self, ev):
        if ev.type == "parry":
            return 10
        if ev.type == "block":
            return 3
        return self.data["moves"][ev.move]["hitstop"]

    def _separate(self):
        gap = 56
        d = self.enemy.x - self.player.x
        if abs(d) < gap:
            push = (gap - abs(d)) / 2
            sgn = 1 if d >= 0 else -1
            self.player.x -= sgn * push
            self.enemy.x += sgn * push
```

```
    def _resolve(self, atk, dfn):
        m = atk.move
        if not m or atk.phase != "active" or atk.has_hit or not dfn.alive:
            return None
        box = hitbox(atk.x, atk.facing, m)
        hurt = hurtbox(dfn.x)
        if not box.colliderect(hurt):
            return None
        atk.has_hit = True
        inter = box.clip(hurt)
        ix, iy = inter.centerx, inter.centery
        heavy = m["damage"] >= 18
        kdir = atk.facing
        if dfn.blocking and dfn.facing == -atk.facing:
            if dfn.parry_window:
                atk.get_staggered()
                dfn.memory = min(dfn.max_memory, dfn.memory + PARRY_MEMORY_GAIN)
                dfn.stun = 0
                return Event("parry", atk.name, dfn.name, ix, iy, 0, atk.state, heavy)
            dmg = int(m["damage"] * 0.15)
            dfn.take_block(dmg, kdir, m["knock"])
            return Event("block", atk.name, dfn.name, ix, iy, dmg, atk.state, heavy)
        dmg = m["damage"]
        if dfn.state == "stagger":
            dmg = int(dmg * STAGGER_BONUS)
        move_id = atk.state
        dfn.take_hit(dmg, m["hitstun"], kdir, m["knock"])
        return Event("hit", atk.name, dfn.name, ix, iy, dmg, move_id, heavy)
```

### Файл: kolenke/combat/fighter.py

*Жауынгер: кадрлық ақырлы автомат (FSM).*

```
"""Жауынгер: кадрлық ақырлы автомат (FSM).

Күйлер: idle, block, <move id> (соққы), hurt, stagger, dead.
Соққы фазалары: startup -> active (hitbox бар) -> recovery.
Блок: алғашқы PARRY_FRAMES кадр — парирлеу терезесі.
"""
import json

from kolenke.core.assets import ROOT

DATA_PATH = ROOT / "kolenke" / "data" / "moves.json"
ARENA = (150, 1130)


def load_combat_data(path=DATA_PATH):
    with open(path, encoding="utf-8") as f:
        return json.load(f)


class Fighter:
    def __init__(self, name, x, facing, data, spec=None):
        self.name = name
        self.data = data
        self.spec = spec or data["fighters"][name]
        self.moves = {m: data["moves"][m] for m in self.spec["moves"]}
        self.parry_frames = data["parry_frames"]
        self.block_cooldown_len = data["block_cooldown"]
        self.x = float(x)
        self.facing = facing
        self.max_hp = self.hp = self.spec["hp"]
        self.max_memory = 100
        self.memory = 0
        self.speed = self.spec["speed"]
```

```
        self.state = "idle"
        self.frame = 0
        self.stun = 0            # hurt/stagger/blockstun ұзақтығы
        self.block_cooldown = 0
        self.has_hit = False
        self.knock = 0.0
        self.flash = 0           # соққы вспышкасы (кадр)
        self.walk_phase = 0.0
        self.walking = False

    # --- сұраныстар ---
    @property
    def move(self):
        return self.moves.get(self.state)

    @property
    def can_act(self):
        return self.state == "idle"

    @property
    def alive(self):
        return self.state != "dead"

    @property
    def phase(self):
        m = self.move
        if not m:
            return None
        if self.frame < m["startup"]:
            return "startup"
        if self.frame < m["startup"] + m["active"]:
            return "active"
        return "recovery"

    @property
    def parry_window(self):
        return self.state == "block" and self.frame < self.parry_frames

    @property
    def blocking(self):
        return self.state == "block"

    # --- логика ---
    def start_move(self, move_id):
        self.state, self.frame, self.has_hit = move_id, 0, False

    def face(self, opp_x):
        self.facing = 1 if opp_x >= self.x else -1

    def update(self, walk, cmd, hold_block, opp_x):
        """Бір логикалық кадр (1/60 с)."""
        if self.flash:
            self.flash -= 1
        if self.block_cooldown:
            self.block_cooldown -= 1
        self.x += self.knock
        self.knock *= 0.78
        self.walking = False
        s = self.state
        if s == "idle":
            self.face(opp_x)
            if walk:
                self.x += walk * self.speed
                self.walking = True
                self.walk_phase += 0.2
            if cmd == "block" and self.block_cooldown == 0:
```

```
                self.state, self.frame = "block", 0
            elif cmd in self.moves:
                self.start_move(cmd)
        elif s == "block":
            self.frame += 1
            if self.frame >= self.parry_frames and not hold_block and self.stun == 0:
                self.state = "idle"
                self.block_cooldown = self.block_cooldown_len
            if self.stun:
                self.stun -= 1
        elif s in self.moves:
            if cmd == "cancel" and self.phase == "startup":   # фінт: қимыл тоқтатылады
                self.state, self.frame = "idle", 0
                return
            self.frame += 1
            m = self.moves[s]
            if self.phase in ("startup", "active"):
                self.x += self.facing * m["step"] * (0.5 if self.phase == "startup" else
1.0)
            if self.frame >= m["startup"] + m["active"] + m["recovery"]:
                self.state = "idle"
        elif s in ("hurt", "stagger"):
            self.frame += 1
            if self.frame >= self.stun:
                self.state = "idle"
        self.x = max(ARENA[0], min(ARENA[1], self.x))

    # --- соққы алу ---
    def take_hit(self, damage, hitstun, knock_dir, knock):
        self.hp = max(0, self.hp - damage)
        self.flash = 4
        self.knock = knock_dir * knock
        if self.hp == 0:
            self.state, self.frame = "dead", 0
        else:
            self.state, self.frame, self.stun = "hurt", 0, hitstun

    def take_block(self, damage, knock_dir, knock):
        self.hp = max(1, self.hp - max(1, damage))
        self.knock = knock_dir * knock * 0.5
        self.stun = 8  # blockstun: блоктан тез шыға алмайды

    def get_staggered(self, frames=55):
        self.state, self.frame, self.stun = "stagger", 0, frames
        self.has_hit = True
```

### Файл: kolenke/combat/hitbox.py

*Hitbox / hurtbox: тік жазықтықтағы тіктөртбұрыштар (әлем координатасы).*

```
"""Hitbox / hurtbox: тік жазықтықтағы тіктөртбұрыштар (әлем координатасы)."""
import pygame

GROUND = 640
HURT_W = 50
HURT_H = 165


def hurtbox(x):
    """Кейіпкер денесі (қорғалатын аймақ)."""
    return pygame.Rect(int(x - HURT_W / 2), GROUND - HURT_H, HURT_W, HURT_H)


def hitbox(x, facing, move):
    """Соққы аймағы: move['reach']=[алыстық басы, ұзындығы], hit_y=[төмен, жоғары
биіктік]."""
    start, length = move["reach"]
```

```
    y_lo, y_hi = move["hit_y"]
    near = x + facing * start
    far = near + facing * length
    left, right = min(near, far), max(near, far)
    return pygame.Rect(int(left), GROUND - y_hi, int(right - left), y_hi - y_lo)
```

### Файл: kolenke/combat/setup.py

*Диалог таңдауларын (флагтар, журнал) жекпе-жек параметрлеріне айналдыру.*

```
"""Диалог таңдауларын (флагтар, журнал) жекпе-жек параметрлеріне айналдыру."""
from kolenke.combat.fighter import load_combat_data

START_MEMORY_FRACTION = 0.5   # жекпе-жек басында естелік жолағы = қордың жартысы
CALLED_NAME_HP_MULT = 0.85    # атын атаған жаудың күші азаяды
RESTORE_RESERVE_COST = 25     # адамды қайтару келесі түндегі қорды азайтады


def known_tells(data, enemy, flags):
    """Қарсылас қимылдарының ішінде ойыншы танитындарының id-лері."""
    moves = data["fighters"][enemy]["moves"]
    return {m for m in moves
            if not data["moves"][m].get("tell_flag") or flags.get(data["moves"][m]
["tell_flag"])}


def journal_entries_for(journal, who):
    return sum(1 for _, mem in journal.entries() if mem["who"] == who)


def fight_params(flags, journal, reserve, enemy, data=None):
    """Fight(...) конструкторына берілетін параметрлер сөздігі."""
    data = data or load_combat_data()
    return dict(
        known_tells=known_tells(data, enemy, flags),
        start_memory=reserve * START_MEMORY_FRACTION,
        enemy_hp_mult=CALLED_NAME_HP_MULT if flags.get(enemy + "_called_name") else 1.0,
        entries=journal_entries_for(journal, enemy),
    )
```

### Файл: kolenke/combat/ai/adaptive.py

*Бейімделетін қарсылас: ойыншының әдеттерін үйреніп, қарсы тактика таңдайды.*

```
"""Бейімделетін қарсылас: ойыншының әдеттерін үйреніп, қарсы тактика таңдайды.

1. Қорғаныс. Ойыншы соққы бастағанда болжам дұрыс болса (conf >= threshold),
   қарсылас реакцияны 0–3 кадрда береді -> парирлеу терезесіне түседі.
   Болжам қате болса, реакция кешеді (адам тәрізді 16–24 кадр).
2. Фінт. «Блок» болжанса: соққыны бастап, ортасында тоқтатады, ойыншының блок
   кулдауынан кейін соғады.
3. Жазалау. Парирленген ойыншыға (stagger) бірден шабуыл жасайды.
Ойыншы әдеттерін өзгертсе, болжам нашарлайды -> қарсылас осалданады.
"""
import random

from kolenke.combat.ai.markov import MarkovPredictor

NIGHT_THRESHOLD = {1: 0.70, 2: 0.55, 3: 0.45}   # 1-түн — оқыту: баяу үйренеді
OBSERVED = ("light", "heavy", "block")
ANTICIPATED_DELAY = (0, 3)     # болжам дұрыс: дереу қорғанады
UNANTICIPATED_DELAY = (16, 24)  # болжам қате: кеш реакция
DEFEND_HOLD = 14               # қорғаныс блогын ұстау (кадр)
DEFEND_TIMEOUT = 20            # қорғанысқа кеш қалса бас тарту


class AdaptiveAI:
    def __init__(self, seed=1, predictor=None, threshold=0.55, adapt=True,
```

```
                 wait=(30, 70)):
        self.rng = random.Random(seed)
        self.predictor = predictor or MarkovPredictor(order=2)
        self.threshold = threshold
        self.adapt = adapt
        self.wait_range = wait
        self.wait = self.rng.randint(*wait)
        self.feint_cancel_at = None
        self.punish_wait = 0
        self.defend_at = None
        self.hold_left = 0
        self.last_logged_tick = -1
        self.stats = {"feints": 0, "defends": 0, "anticipated": 0, "punishes": 0,
                      "plain": 0}

    @classmethod
    def for_night(cls, night, seed=1):
        return cls(seed=seed, threshold=NIGHT_THRESHOLD.get(night, 0.55))

    # --- бақылау ---
    def _observe(self, fight, player):
        if not (player.state in OBSERVED and player.frame == 0
                and self.last_logged_tick != fight.frame_no):
            return
        self.last_logged_tick = fight.frame_no
        action = player.state
        pred, conf = self.predictor.predict()
        if self.adapt and action in ("light", "heavy"):
            hit = pred == action and conf >= self.threshold
            delay = self.rng.randint(*(ANTICIPATED_DELAY if hit else UNANTICIPATED_DELAY))
            self.defend_at = fight.frame_no + delay
            self.stats["defends"] += 1
            self.stats["anticipated"] += hit
        self.predictor.observe(action)

    # --- шешім ---
    def decide(self, fight, me, opp):
        self._observe(fight, opp)
        hold = self.hold_left > 0
        if hold:
            self.hold_left -= 1
        if self.defend_at is not None:                      # қорғаныс жоспарланған
            if fight.frame_no > self.defend_at + DEFEND_TIMEOUT:
                self.defend_at = None
            elif fight.frame_no >= self.defend_at:
                if me.can_act:
                    self.defend_at, self.hold_left = None, DEFEND_HOLD
                    return 0, "block", True
                if me.move and me.phase == "startup":
                    return 0, "cancel", False               # өз шабуылын тоқтатып қорғану
        if not me.can_act:
            if (self.feint_cancel_at is not None and me.move
                    and me.phase == "startup" and me.frame >= self.feint_cancel_at):
                self.feint_cancel_at = None
                return 0, "cancel", False
            return 0, None, hold
        toward = 1 if opp.x > me.x else -1
        dist = abs(opp.x - me.x)
        reach = max(m["reach"][0] + m["reach"][1] for m in me.moves.values())
        if self.adapt and opp.state == "stagger" and dist <= reach:   # жазалау
            self.stats["punishes"] += 1
            return 0, self._fastest(me), False
        if self.punish_wait > 0:                    # фінттен кейін: ойыншы блоктан
шыққанша
            self.punish_wait -= 1
            if opp.state == "idle" and opp.block_cooldown > 0 or self.punish_wait == 0:
```

```
                self.punish_wait = 0
                self.stats["plain"] += 1
                return 0, self._fastest(me), False
            return 0, None, hold
        if dist > reach - 15:
            return toward, None, hold
        if self.wait > 0:
            self.wait -= 1
            return 0, None, hold
        self.wait = self.rng.randint(*self.wait_range)
        pred, conf = self.predictor.predict()
        if self.adapt and pred == "block" and conf >= self.threshold:
            move = self.rng.choice(list(me.moves))
            self.feint_cancel_at = max(4, int(me.moves[move]["startup"] * 0.55))
            self.punish_wait = 50
            self.stats["feints"] += 1
            return 0, move, False
        self.stats["plain"] += 1
        return 0, self.rng.choice(list(me.moves)), False

    @staticmethod
    def _fastest(me):
        return min(me.moves, key=lambda k: me.moves[k]["startup"])
```

### Файл: kolenke/combat/ai/bots.py

*Тестілеуге арналған «ойыншы-боттар»: әртүрлі ойын стильдері (эксперимент үшін).*

```
"""Тестілеуге арналған «ойыншы-боттар»: әртүрлі ойын стильдері (эксперимент үшін)."""
import random

APPROACH = 95   # соғатын қашықтық


class Bot:
    """Базалық бот: жақындайды, стиль бойынша әрекет таңдайды."""

    name = "bot"

    def __init__(self, seed=1):
        self.rng = random.Random(seed)
        self.wait = self.rng.randint(10, 30)
        self.hold_left = 0

    def choose(self, fight, me, opp):
        raise NotImplementedError

    def decide(self, fight, me, opp):
        if self.hold_left > 0:
            self.hold_left -= 1
        hold = self.hold_left > 0
        if not me.can_act:
            return 0, None, hold
        toward = 1 if opp.x > me.x else -1
        if abs(opp.x - me.x) > APPROACH:
            return toward, None, hold
        cmd = self.choose(fight, me, opp)
        if cmd == "block":
            self.hold_left = 12
        return 0, cmd, self.hold_left > 0


class Aggressor(Bot):
    name = "aggressor"

    def choose(self, fight, me, opp):
        if self.wait > 0:
```

```
            self.wait -= 1
            return None
        self.wait = self.rng.randint(18, 40)
        return self.rng.choices(("light", "heavy", "block"), (0.65, 0.3, 0.05))[0]


class Pattern(Bot):
    """Қатаң қайталанатын әдет: L, L, H, B."""

    name = "pattern"
    SEQ = ("light", "light", "heavy", "block")

    def __init__(self, seed=1):
        super().__init__(seed)
        self.i = 0

    def choose(self, fight, me, opp):
        if self.wait > 0:
            self.wait -= 1
            return None
        self.wait = 24
        cmd = self.SEQ[self.i % len(self.SEQ)]
        self.i += 1
        return cmd


class RandomBot(Bot):
    name = "random"

    def choose(self, fight, me, opp):
        if self.wait > 0:
            self.wait -= 1
            return None
        self.wait = self.rng.randint(18, 40)
        return self.rng.choice(("light", "heavy", "block"))


class Reactive(Bot):
    """Қарсыластың дайындық қимылына жауап береді (парирлеуге тырысады), кейін соғады."""

    name = "reactive"

    def __init__(self, seed=1, jitter=(0, 5)):
        super().__init__(seed)
        self.jitter = jitter
        self.delay = None

    def choose(self, fight, me, opp):
        m = opp.move
        if m and opp.phase == "startup":
            left = m["startup"] - opp.frame
            if self.delay is None:
                self.delay = self.rng.randint(*self.jitter)
            if left <= me.parry_frames - 2 - self.delay:
                self.delay = None
                return "block"
            return None
        self.delay = None
        if opp.state in ("stagger", "hurt") or (opp.phase == "recovery"):
            return self.rng.choice(("light", "heavy"))
        if self.wait > 0:
            self.wait -= 1
            return None
        self.wait = self.rng.randint(25, 60)
        return "light"
```

```
BOTS = {c.name: c for c in (Aggressor, Pattern, RandomBot, Reactive)}
```

### Файл: kolenke/combat/ai/markov.py

*Ойыншының әрекеттерін жиілік кестесіне жазып, келесісін болжау (Марков тізбегі).*

```
"""Ойыншының әрекеттерін жиілік кестесіне жазып, келесісін болжау (Марков тізбегі).

Күй — соңғы k әрекет (k = order). Кесте counts[күй][келесі әрекет].
Болжау: ең ұзын контексттен бастап (backoff) жиілігі бар алғашқысы алынады.
Сглаживание Лапласа: p = (n + a) / (N + a*|A|).
"""
from collections import defaultdict

ACTIONS = ("light", "heavy", "block")


class MarkovPredictor:
    def __init__(self, order=2, actions=ACTIONS, alpha=0.5):
        self.order = order
        self.actions = tuple(actions)
        self.alpha = alpha
        self.counts = defaultdict(lambda: defaultdict(int))  # контекст -> әрекет -> n
        self.history = []
        self.log = []   # (болжам, шын әрекет) жұптары — дәлдікті өлшеу үшін

    def _context(self, k):
        return tuple(self.history[-k:]) if k else ()

    def predict(self):
        """(әрекет, ықтималдық). Деректер жоқ болса (None, 0.0)."""
        for k in range(min(self.order, len(self.history)), -1, -1):
            table = self.counts.get(self._context(k))
            if not table:
                continue
            total = sum(table.values())
            denom = total + self.alpha * len(self.actions)
            probs = {a: (table.get(a, 0) + self.alpha) / denom for a in self.actions}
            best = max(self.actions, key=lambda a: (probs[a], -self.actions.index(a)))
            return best, probs[best]
        return None, 0.0

    def observe(self, action):
        """Шын әрекетті жазады; алдымен болжамның дұрыстығын журналға түсіреді."""
        pred, _ = self.predict()
        if pred is not None:
            self.log.append((pred, action))
        for k in range(0, self.order + 1):
            if k <= len(self.history):
                self.counts[self._context(k)][action] += 1
        self.history.append(action)

    @property
    def accuracy(self):
        if not self.log:
            return 0.0
        return sum(p == a for p, a in self.log) / len(self.log)

    def reset(self):
        self.counts.clear()
        self.history.clear()
        self.log.clear()
```

**Файл: kolenke/combat/ai/metrics.py**

*Болжау дәлдігін өлшеу және базалық әдістермен салыстыру.*

```
"""Болжау дәлдігін өлшеу және базалық әдістермен салыстыру."""
import random
from collections import Counter

from kolenke.combat.ai.markov import ACTIONS, MarkovPredictor


def online_accuracy(sequence, predictor):
    """Тізбекті бір-бірлеп беріп, әр қадамдағы болжам дәлдігін есептейді."""
    hits = total = 0
    curve = []
    for a in sequence:
        pred, _ = predictor.predict()
        if pred is not None:
            total += 1
            hits += pred == a
            curve.append(hits / total)
        predictor.observe(a)
    return (hits / total if total else 0.0), curve


def baseline_random(sequence, seed=0):
    rng = random.Random(seed)
    if len(sequence) < 2:
        return 0.0
    return sum(rng.choice(ACTIONS) == a for a in sequence[1:]) / (len(sequence) - 1)


def baseline_last(sequence):
    """Алдыңғы әрекет қайталанады деп болжау."""
    if len(sequence) < 2:
        return 0.0
    return sum(sequence[i - 1] == sequence[i] for i in range(1, len(sequence))) /
(len(sequence) - 1)


def baseline_frequency(sequence):
    """Әрқашан бұрынғы ең жиі әрекетті болжау (order-0)."""
    seen, hits, total = Counter(), 0, 0
    for a in sequence:
        if seen:
            total += 1
            hits += seen.most_common(1)[0][0] == a
        seen[a] += 1
    return hits / total if total else 0.0


def compare(sequence):
    """Барлық әдістің дәлдігі бір сөздікте."""
    return {
        "random": baseline_random(sequence),
        "last": baseline_last(sequence),
        "frequency": baseline_frequency(sequence),
        "markov1": online_accuracy(sequence, MarkovPredictor(order=1))[0],
        "markov2": online_accuracy(sequence, MarkovPredictor(order=2))[0],
        "markov3": online_accuracy(sequence, MarkovPredictor(order=3))[0],
    }
```

**Файл: kolenke/render/corruption.py**

*3-түн: басты кейіпкердің өзі айнала бастайды — экран қараяды, интерфейс жоғалады.*

```
"""3-түн: басты кейіпкердің өзі айнала бастайды — экран қараяды, интерфейс жоғалады."""
import random
```

```
import pygame

_cache = {}


def hidden(level, threshold):
    """Интерфейс элементі осы дәрежеде жоғалады ма."""
    return level >= threshold


def protagonist_detail(level):
    """Басты кейіпкердің акценті мен бет белгілері тозуы: 1.0 -> 0.0."""
    return max(0.0, min(1.0, 1.0 - (level - 0.3) / 0.7))


def draw_corruption(surface, level, t):
    """Қараңғы перде + кездейсоқ «жоғалу» жолақтары."""
    if level <= 0.04:
        return
    size = surface.get_size()
    if size not in _cache:
        dark = pygame.Surface(size)
        dark.fill((0, 0, 0))
        _cache[size] = dark
    dark = _cache[size]
    dark.set_alpha(int(min(0.85, level) * 200))
    surface.blit(dark, (0, 0))
    rng = random.Random(int(t * 6))
    w, h = size
    for _ in range(int(level * 7)):
        y = rng.randrange(0, h)
        bar = pygame.Rect(0, y, w, rng.choice((2, 3, 5, 8)))
        surface.fill((0, 0, 0), bar)
```

### Файл: kolenke/render/effects.py

*Экран эффектілері: қараңғыға өту (fade), экран дірілі.*

```
"""Экран эффектілері: қараңғыға өту (fade), экран дірілі."""
import random

import pygame


class Fade:
    """Қараңғыға өшу -> callback -> қайта ашылу. Есік ауысуына қолданылады."""

    def __init__(self, size):
        self.overlay = pygame.Surface(size)
        self.overlay.fill((0, 0, 0))
        self.alpha = 0.0
        self.state = "idle"
        self.duration = 0.25
        self.callback = None

    @property
    def active(self):
        return self.state != "idle"

    def start(self, callback, duration=0.25):
        if self.active:
            return
        self.state, self.duration, self.callback = "out", duration, callback

    def update(self, dt):
        if self.state == "out":
```

```
            self.alpha = min(255.0, self.alpha + 255 * dt / self.duration)
            if self.alpha >= 255:
                if self.callback:
                    self.callback()
                self.state = "in"
        elif self.state == "in":
            self.alpha = max(0.0, self.alpha - 255 * dt / self.duration)
            if self.alpha <= 0:
                self.state = "idle"

    def draw(self, surface):
        if self.alpha > 0:
            self.overlay.set_alpha(int(self.alpha))
            surface.blit(self.overlay, (0, 0))


class ScreenShake:
    """Trauma-негізді діріл: соққы/парирлеуде add() шақырылады."""

    def __init__(self, max_px=18, decay=2.2):
        self.trauma = 0.0
        self.max_px = max_px
        self.decay = decay

    def add(self, amount):
        self.trauma = min(1.0, self.trauma + amount)

    def update(self, dt):
        self.trauma = max(0.0, self.trauma - self.decay * dt)

    def offset(self):
        k = self.trauma ** 2 * self.max_px
        return (int(random.uniform(-k, k)), int(random.uniform(-k, k)))
```

### Файл: kolenke/render/fighter_draw.py

*Жауынгерді күйі мен кадрына қарай сызу (поза — moves.json ішіндегі 'arm' кілттері).*

```
"""Жауынгерді күйі мен кадрына қарай сызу (поза — moves.json ішіндегі 'arm' кілттері)."""
import math

from kolenke.combat.hitbox import GROUND
from kolenke.render.silhouette import draw_silhouette

FLASH_COLOR = (235, 235, 248)


def _lerp(a, b, t):
    return a + (b - a) * t


def pose(f):
    """(arm_angle, lean, weapon_len) — жауынгердің қазіргі позасы."""
    weapon = 28.0 if f.spec.get("weapon") else 0.0
    base_arm = 0.45 if weapon else None
    s = f.state
    m = f.move
    if m:
        a0, a1, a2, a3 = m["arm"]
        su, ac, rec = m["startup"], m["active"], m["recovery"]
        lean_wind = m.get("lean", -8)
        weapon = m.get("weapon_len", weapon)
        if f.frame < su:
            t = f.frame / su
            t = t * t * (3 - 2 * t)      # ease: қозғалыс «дайындалады»
            return _lerp(a0, a1, t), lean_wind * t, weapon
        if f.frame < su + ac:
```

```
            return a2, 14.0, weapon
        t = (f.frame - su - ac) / max(1, rec)
        return _lerp(a2, a3, t), 14.0 * (1 - t), weapon
    if s == "block":
        return 2.3, -4.0, weapon
    if s == "hurt":
        return base_arm, -16.0, weapon
    if s == "stagger":
        return 0.2, -22.0, weapon
    if s == "dead":
        return base_arm, -48.0, 0.0
    return base_arm, 0.0, weapon


def draw_fighter(surface, f, accent, t, part="all", detail=1.0):
    """Қайтарады: (қол, ұш) — соққы ізі (trail) үшін."""
    arm, lean, weapon = pose(f)
    lean += math.sin(t * 2.2 + (0 if f.facing > 0 else 1.3)) * 1.5
    swing = math.sin(f.walk_phase * 2) * 0.8 if f.walking else 0.0
    return draw_silhouette(
        surface, f.x, GROUND, accent, f.facing, swing, t=t, part=part,
        detail=0.0 if f.state == "dead" else detail,
        arm_angle=arm, lean=lean, weapon=weapon,
        body_color=FLASH_COLOR if f.flash > 0 else None)
```

### Файл: kolenke/render/lighting.py

*Жарықтандыру (multiply-маска), виньетка. Қараңғы түн атмосферасы үшін.*

```
"""Жарықтандыру (multiply-маска), виньетка. Қараңғы түн атмосферасы үшін."""
import math

import pygame


class Lighting:
    """Қараңғы қабат: ambient + қосылатын жарық дақтары, сосын сахнаға көбейтіледі."""

    def __init__(self, size, ambient=(16, 18, 28)):
        self.size = size
        self.ambient = ambient
        self.dark = pygame.Surface(size)
        self._cache = {}

    def _light_surface(self, radius, color, intensity):
        radius = max(16, int(radius) // 8 * 8)
        level = int(intensity * 8)
        key = (radius, color, level)
        if key not in self._cache:
            base = pygame.Surface((64, 64))
            k = level / 8.0
            for i in range(32, 0, -1):
                f = (1 - i / 32) ** 1.5 * k
                pygame.draw.circle(base, tuple(min(255, int(c * f)) for c in color),
                                   (32, 32), i)
            self._cache[key] = pygame.transform.smoothscale(base, (radius * 2, radius * 2))
        return self._cache[key], radius

    def render(self, screen, lights, cam_x, t=0.0, offset=(0, 0)):
        """lights: dict(x, y, r, color, i, flicker, seed) тізімі."""
        self.dark.fill(self.ambient)
        w = self.size[0]
        for L in lights:
            sx = L["x"] - cam_x
            if sx + L["r"] < 0 or sx - L["r"] > w:
                continue
            inten = L["i"] * (1 - L.get("flicker", 0) * self.noise(t, L.get("seed", 0)))
```

```
            surf, r = self._light_surface(L["r"], L["color"], max(0.05, inten))
            self.dark.blit(surf, (sx - r, L["y"] - r), special_flags=pygame.BLEND_RGB_ADD)
        screen.blit(self.dark, offset, special_flags=pygame.BLEND_RGB_MULT)

    @staticmethod
    def noise(t, seed):
        """0..1 мерцание: көбіне тыныш, кейде күрт сөнеді."""
        a = math.sin(t * 13 + seed) * math.sin(t * 7.3 + seed * 2.1)
        return max(0.0, abs(a) - 0.35) / 0.65


def make_vignette(size, strength=120):
    small = pygame.Surface((64, 36), pygame.SRCALPHA)
    for y in range(36):
        for x in range(64):
            dx, dy = (x / 63 - 0.5) * 2, (y / 35 - 0.5) * 2
            r = min(1.0, math.hypot(dx, dy) / 1.38)
            a = int(strength * (max(0.0, r - 0.42) / 0.58) ** 2)
            small.set_at((x, y), (0, 0, 0, a))
    return pygame.transform.smoothscale(small, size)
```

### Файл: kolenke/render/particles.py

*Бөлшектер жүйесі: жалпы (шаң, ұшқын, көлеңке) және боран.*

```
"""Бөлшектер жүйесі: жалпы (шаң, ұшқын, көлеңке) және боран."""
import math
import random


class ParticleSystem:
    """Қарапайым бөлшектер: орны, жылдамдығы, өмірі, өлшемі, түсі."""

    def __init__(self, limit=400):
        self.limit = limit
        self.items = []  # [x, y, vx, vy, life, max_life, size, color, gravity]

    def emit(self, x, y, vx, vy, life, size, color, gravity=0.0):
        if len(self.items) < self.limit:
            self.items.append([x, y, vx, vy, life, life, size, color, gravity])

    def update(self, dt):
        alive = []
        for p in self.items:
            p[4] -= dt
            if p[4] > 0:
                p[3] += p[8] * dt
                p[0] += p[2] * dt
                p[1] += p[3] * dt
                alive.append(p)
        self.items = alive

    def draw(self, surface, cam_x=0.0):
        for x, y, _, _, life, max_life, size, color, _ in self.items:
            k = life / max_life  # өшу: түсті қараңғыға қарай азайту
            c = (int(color[0] * k), int(color[1] * k), int(color[2] * k))
            r = max(1, int(size * (0.4 + 0.6 * k)))
            surface.fill(c, (int(x - cam_x), int(y), r, r))


class Blizzard:
    """Экран кеңістігіндегі боран: терезе артында немесе фонда."""

    def __init__(self, width, height, count=200, wind=-140, seed=7):
        rng = random.Random(seed)
        self.w, self.h, self.wind = width, height, wind
        self.flakes = [[rng.uniform(0, width), rng.uniform(0, height),
```

```
                        rng.uniform(90, 260), rng.choice((1, 2, 2, 3, 4)),
                        rng.uniform(0, 6.28)] for _ in range(count)]
        self.t = 0.0

    def update(self, dt):
        self.t += dt
        for f in self.flakes:
            f[1] += f[2] * dt
            f[0] += (self.wind + math.sin(self.t * 1.3 + f[4]) * 45) * dt * (f[3] / 2.5)
            if f[1] > self.h:
                f[1] -= self.h
            if f[0] < 0:
                f[0] += self.w
            elif f[0] > self.w:
                f[0] -= self.w

    def draw(self, surface, brightness=1.0, offset=(0, 0)):
        ox, oy = offset
        for x, y, _, size, _ in self.flakes:
            v = min(255, int((110 + 36 * size) * brightness))
            surface.fill((v, v, min(255, v + 12)), (int(x + ox), int(y + oy), size, size))
```

### Файл: kolenke/render/portrait.py

*Диалогтағы үлкен силуэт-портреттер: жұмсартылған пішін, контр-жарық, 3 күй.*

```
"""Диалогтағы үлкен силуэт-портреттер: жұмсартылған пішін, контр-жарық, 3 күй.

Күйлер (көлеңке жұтқан сайын детальдар жоғалады — ТЗ):
  human  — бет белгілері, киім бөлшектері, акцент жарқырайды;
  fading — белгілер өшеді, шеті бұзылады, акцент солғын;
  shadow — таза қара силуэт, тек жұқа акцентті жиек.
Портрет 2x масштабта сызылып, кішірейтіледі (anti-aliasing) және кэштеледі.
"""
import math
import random

import pygame

W, H = 520, 700            # соңғы өлшем
S = 2                      # супер-семплинг
BODY = (10, 10, 16)
RIM_DIM = {"human": 0.9, "fading": 0.5, "shadow": 0.7}
CHARACTERS = {
    "darkhan": (240, 160, 60),
    "zhanel": (90, 215, 190),
    "madina": (215, 125, 225),
    "bekzat": (245, 220, 125),
    "me": (120, 180, 240),
}
# Кейіпкер келбеті: тері, шаш, киім, шаш үлгісі
LOOK = {
    "darkhan": dict(skin=(236, 192, 152), hair=(46, 30, 24), cloth=(214, 204, 192),
cloth2=(160, 148, 138), style="hat"),
    "zhanel": dict(skin=(228, 182, 146), hair=(24, 22, 30), cloth=(52, 108, 110),
cloth2=(34, 76, 80), style="long"),
    "madina": dict(skin=(238, 198, 168), hair=(70, 40, 34), cloth=(132, 78, 142),
cloth2=(96, 54, 106), style="bun"),
    "bekzat": dict(skin=(220, 178, 142), hair=(30, 24, 26), cloth=(226, 218, 196),
cloth2=(52, 56, 78), style="messy"),
    "me": dict(skin=(230, 188, 152), hair=(54, 38, 32), cloth=(66, 98, 150), cloth2=(44,
68, 108), style="short"),
}
_cache = {}
_glow_cache = {}
```

```
def _mix(c, k):
    return tuple(max(0, min(255, int(v * k))) for v in c)


def _lerp(a, b, k):
    return tuple(int(a[i] + (b[i] - a[i]) * k) for i in range(3))


class _Pen:
    """Күйге қарай түсті өзгертетін қылқалам: human — толық түс, fading — сөніп бара
жатқан,
    shadow — таза қара."""

    def __init__(self, surf, state):
        self.s, self.state = surf, state

    def c(self, col):
        if self.state == "human":
            return col
        if self.state == "fading":
            g = sum(col) // 3
            return _lerp(_lerp(col, (g, g, g), 0.7), (14, 14, 22), 0.55)
        return BODY

    def ellipse(self, col, rect, width=0):
        pygame.draw.ellipse(self.s, self.c(col), tuple(int(v * S) for v in rect), width *
S)

    def poly(self, col, pts):
        pygame.draw.polygon(self.s, self.c(col), [(int(x * S), int(y * S)) for x, y in
pts])

    def rect(self, col, rect, radius=0):
        pygame.draw.rect(self.s, self.c(col), tuple(int(v * S) for v in rect),
border_radius=radius * S)

    def circle(self, col, pos, r, width=0):
        pygame.draw.circle(self.s, self.c(col), (int(pos[0] * S), int(pos[1] * S)), int(r *
S), width * S)

    def line(self, col, a, b, w=2):
        pygame.draw.line(self.s, self.c(col), (int(a[0] * S), int(a[1] * S)),
                         (int(b[0] * S), int(b[1] * S)), max(1, int(w * S)))


CX = W // 2


def _torso(p, look, accent, dx=0):
    cloth, cloth2 = look["cloth"], look["cloth2"]
    body = [(CX - 120 + dx, 405), (CX + 120 + dx, 405), (CX + 300 + dx, 520), (CX + 345 +
dx, H),
            (CX - 345 + dx, H), (CX - 300 + dx, 520)]
    p.poly(cloth2, body)
    for sx in (-1, 1):
        p.circle(cloth2, (CX + sx * 262 + dx, 545), 92)
    # жарық жағы (сол жақ) — ашығырақ
    p.poly(cloth, [(CX - 118 + dx, 405), (CX + 30 + dx, 405), (CX + 40 + dx, H), (CX - 345
+ dx, H),
                   (CX - 300 + dx, 520)])
    p.circle(cloth, (CX - 262 + dx, 545), 90)
    # мойын + көлеңке
    p.rect(look["skin"], (CX - 38 + dx, 330, 76, 100))


def _neck_shadow(p, look):
```

```
    p.ellipse(_mix(look["skin"], 0.72), (CX - 44, 376, 88, 56))
    p.rect(look["skin"], (CX - 38, 330, 40, 60))


def _hair_back(p, look):
    st, hc = look["style"], look["hair"]
    if st == "long":
        p.poly(hc, [(CX - 112, 220), (CX + 112, 220), (CX + 150, 520), (CX + 60, 560), (CX
- 60, 560),
                    (CX - 150, 520)])
        p.ellipse(hc, (CX - 112, 120, 224, 240))
    elif st == "bun":
        p.circle(hc, (CX, 96), 52)
        p.circle(_mix(hc, 1.25), (CX - 12, 86), 22)
    elif st == "short":
        p.ellipse(hc, (CX - 104, 134, 208, 150))
    elif st == "messy":
        p.ellipse(hc, (CX - 106, 128, 212, 160))


def _head(p, look):
    skin = look["skin"]
    p.ellipse(_mix(skin, 0.8), (CX - 94, 150, 188, 232))              # көлеңкелі бет
негізі
    p.ellipse(skin, (CX - 94, 150, 168, 232))                           # жарық жағы
    for sx in (-1, 1):                                                  # құлақтар
        p.ellipse(_mix(skin, 0.9 if sx < 0 else 0.72), (CX + sx * 92 - 14, 262, 28, 50))
    p.ellipse(_mix(skin, 1.04), (CX - 22, 276, 44, 58))                 # мұрын көпірі


def _hair_front(p, look):
    st, hc = look["style"], look["hair"]
    hi = _mix(hc, 1.7)
    if st == "hat":
        p.rect((246, 244, 236), (CX - 104, 120, 208, 62), 10)           # қалпақ белдігі
        p.ellipse((250, 248, 240), (CX - 126, 30, 252, 130))             # қалпақ төбесі
        p.ellipse((226, 222, 212), (CX - 60, 52, 190, 96))
        p.ellipse((250, 248, 240), (CX - 126, 36, 160, 120))
        p.rect((208, 200, 188), (CX - 104, 168, 208, 12), 6)
        p.poly(hc, [(CX - 92, 186), (CX - 58, 176), (CX - 80, 232)])    # самай
        p.poly(hc, [(CX + 92, 186), (CX + 58, 176), (CX + 80, 232)])
    elif st == "long":
        p.ellipse(hc, (CX - 104, 130, 208, 120))
        p.poly(hc, [(CX - 100, 200), (CX - 20, 178), (CX + 80, 190), (CX + 100, 250), (CX +
60, 214),
                    (CX - 30, 200), (CX - 90, 256)])
        p.line(hi, (CX - 60, 150), (CX - 20, 142), 6)
    elif st == "bun":
        p.ellipse(hc, (CX - 102, 132, 204, 120))
        p.poly(hc, [(CX - 98, 204), (CX - 10, 170), (CX + 98, 204), (CX + 70, 214), (CX,
190), (CX - 70, 220)])
        p.line(hi, (CX - 54, 150), (CX - 10, 142), 6)
    elif st == "short":
        p.ellipse(hc, (CX - 104, 130, 208, 114))
        p.poly(hc, [(CX - 100, 190), (CX - 30, 160), (CX + 60, 168), (CX + 102, 200), (CX +
70, 196),
                    (CX + 10, 180), (CX - 60, 196)])
        p.line(hi, (CX - 62, 146), (CX - 16, 138), 6)
    else:  # messy
        p.ellipse(hc, (CX - 106, 124, 212, 120))
        for i, ox in enumerate((-90, -46, 0, 46, 90)):
            p.poly(hc, [(CX + ox - 26, 160), (CX + ox + 6, 108 - (i % 2) * 14), (CX + ox +
34, 166)])
        p.poly(hc, [(CX - 100, 196), (CX - 20, 170), (CX + 100, 200), (CX + 40, 190), (CX -
70, 216)])
        p.line(hi, (CX - 52, 140), (CX - 6, 124), 6)
```

```
def _face(p, look, accent, name):
    """Бет белгілері: қас, көз (ақ + нұрлы мөлдір), мұрын, ауыз, ұялу."""
    skin = look["skin"]
    brow = _mix(look["hair"], 1.1)
    for sx in (-1, 1):
        ex = CX + sx * 40
        p.ellipse(_mix(skin, 0.86), (ex - 24, 247, 48, 30))              # қабақ көлеңкесі
        p.ellipse((250, 250, 252), (ex - 19, 250, 38, 24))                # көз қарашығы ақ
        p.circle(_mix(accent, 0.75), (ex + sx * -1, 262), 11)             # нұрлы
        p.circle((12, 12, 18), (ex + sx * -1, 262), 6)
        p.circle((255, 255, 255), (ex - 4, 258), 3)
        p.line((20, 16, 20), (ex - 20, 255), (ex + 20, 253), 3)           # кірпік
        p.line(brow, (ex - sx * 20, 230), (ex + sx * 26, 224 + (4 if sx > 0 else 0)), 6)
        p.ellipse(_lerp(skin, (240, 130, 120), 0.35), (ex - 18, 296, 36, 20))   # ұялу
    p.line(_mix(skin, 0.6), (CX - 4, 286), (CX - 10, 304), 3)
    p.line(_mix(skin, 0.6), (CX - 10, 304), (CX + 2, 308), 3)
    p.line((150, 70, 70), (CX - 24, 332), (CX + 24, 332), 4)
    p.line((150, 70, 70), (CX - 24, 332), (CX - 30, 326), 3)
    p.line((150, 70, 70), (CX + 24, 332), (CX + 30, 328), 3)


def _outfit(p, look, accent, name):
    """Киім бөлшектері мен кейіпкерге тән аксессуар."""
    cloth2 = look["cloth2"]
    if name == "darkhan":                                          # наубайшы алжапқышы + бау
        p.poly(accent, [(CX - 70, 430), (CX + 70, 430), (CX + 90, H), (CX - 90, H)])
        p.poly(_mix(accent, 0.7), [(CX + 20, 430), (CX + 70, 430), (CX + 90, H), (CX + 40,
H)])
        p.line((240, 232, 222), (CX - 70, 430), (CX - 30, 405), 10)
        p.line((240, 232, 222), (CX + 70, 430), (CX + 30, 405), 10)
        for i in range(6):                                         # ұн дақтары
            p.circle((246, 244, 238), (CX - 190 + i * 38, 520 + (i * 37) % 60), 5 + i % 3)
    elif name == "zhanel":                                         # құлаққап + капюшон бауы
        p.line((26, 28, 36), (CX - 112, 220), (CX - 98, 150), 14)
        pygame.draw.arc(p.s, p.c((26, 28, 36)), (int((CX - 120) * S), int(70 * S), int(240
* S), int(250 * S)),
                        0.15, math.pi - 0.15, 14 * S)
        for sx in (-1, 1):
            p.ellipse((30, 32, 42), (CX + sx * 112 - 26, 232, 52, 86))
            p.ellipse(accent, (CX + sx * 112 - 26, 232, 52, 86), 4)
            p.line(accent, (CX + sx * 62, 410), (CX + sx * 70, 520), 5)
        p.circle(accent, (CX + 330, 340), 9)
        p.line((40, 40, 50), (CX + 200, 520), (CX + 330, 340), 8)
    elif name == "madina":                                         # кардиган + өлшеу таспасы
        p.poly(look["cloth"], [(CX - 60, 410), (CX + 60, 410), (CX + 36, H), (CX - 36, H)])
        for sx in (-1, 1):
            p.line(accent, (CX + sx * 34, 408), (CX + sx * 76, H), 10)
        p.line(accent, (CX - 40, 414), (CX + 40, 414), 9)
        for k in range(9):
            p.line((245, 240, 230), (CX - 62 + k * 15, 420 + k * 0), (CX - 62 + k * 15,
432), 2)
        p.circle((240, 220, 150), (CX + 12, 560), 7)                # түйме
        p.circle((240, 220, 150), (CX + 12, 610), 7)
    elif name == "bekzat":                                         # жилет + галстук + көзәйнек
        p.poly(cloth2, [(CX - 70, 410), (CX, 520), (CX - 14, H), (CX - 118, H), (CX - 150,
470)])
        p.poly(cloth2, [(CX + 70, 410), (CX, 520), (CX + 14, H), (CX + 118, H), (CX + 150,
470)])
        p.poly((238, 232, 214), [(CX - 40, 408), (CX, 470), (CX + 40, 408)])
        p.poly(accent, [(CX - 14, 440), (CX + 14, 440), (CX + 22, 560), (CX, 590), (CX -
22, 560)])
        for sx in (-1, 1):
            p.circle((34, 34, 46), (CX + sx * 42, 262), 36, 5)
        p.line((34, 34, 46), (CX - 8, 262), (CX + 8, 262), 5)
```

```
        p.line(accent, (CX - 20, 244), (CX - 6, 232), 4)
    else:                                                # me: мойынорағыш
        p.rect(accent, (CX - 74, 396, 148, 44), 18)
        p.rect(_mix(accent, 0.8), (CX - 74, 416, 148, 24), 12)
        p.poly(accent, [(CX + 28, 430), (CX + 106, 430), (CX + 124, 640), (CX + 62, 612)])
        p.line(_mix(accent, 0.7), (CX + 60, 450), (CX + 86, 620), 3)
        for sx in (-1, 1):                               # жаға
            p.poly(cloth2, [(CX + sx * 74, 405), (CX + sx * 150, 430), (CX + sx * 90,
470)])


def _folds(p, look):
    """Киім қыртыстары — көлемдік әсер."""
    dark = _mix(look["cloth2"], 0.7)
    for x0, x1 in ((-230, -200), (-160, -150), (180, 210), (250, 290)):
        p.line(dark, (CX + x0, 520), (CX + x1, H), 4)


def _erode(surf, amount, seed):
    """Шеттері бұзылу: кездейсоқ дөңгелектермен альфаны азайту."""
    rng = random.Random(seed)
    mask = pygame.Surface(surf.get_size(), pygame.SRCALPHA)
    for _ in range(int(70 * amount)):
        x = rng.randint(0, W * S)
        y = rng.randint(30 * S, H * S)
        if rng.random() < 0.65:   # көбіне шетке жақын
            x = W * S // 2 + rng.choice((-1, 1)) * rng.randint(70 * S, 340 * S)
        pygame.draw.circle(mask, (255, 255, 255, 255), (x, y), rng.randint(8 * S, 34 * S))
    surf.blit(mask, (0, 0), special_flags=pygame.BLEND_RGBA_SUB)


def _draw_character(surf, name, state, accent, dx=0, dy=0):
    look = LOOK[name]
    p = _Pen(surf, state)
    _hair_back(p, look)
    _torso(p, look, accent, dx)
    if state != "shadow":
        _folds(p, look)
    _head(p, look)
    _neck_shadow(p, look)
    _hair_front(p, look)
    if state == "human":
        _face(p, look, accent, name)
    elif state == "fading":
        ex = CX - 40
        p.ellipse(_mix(accent, 0.5), (ex - 14, 252, 28, 18))        # бір көз ғана солғын
    _outfit(p, look, accent, name)


def build_portrait(name, state):
    """Портрет бетін жасайды (кэштеледі). Қайтарады: W×H SRCALPHA."""
    key = (name, state)
    if key in _cache:
        return _cache[key]
    accent = CHARACTERS[name]
    big = pygame.Surface((W * S, H * S), pygame.SRCALPHA)
    # контр-жарық жиегі: бейненің жылжытылған көшірмесі акцент түсімен
    rim = pygame.Surface((W * S, H * S), pygame.SRCALPHA)
    _draw_character(rim, name, "shadow", accent)
    mask = pygame.mask.from_surface(rim)
    rim_col = _mix(accent, RIM_DIM[state])
    rim_img = mask.to_surface(setcolor=rim_col + (255,), unsetcolor=(0, 0, 0, 0))
    big.blit(rim_img, (8 * S, -3 * S))
    _draw_character(big, name, state, accent)
    if state == "human":                    # сол жақтан жарық: оң жақты көлеңкелеу
        grad = pygame.Surface((W * S, 1))
```

```
        for x in range(W * S):
            k = x / (W * S)
            v = int(255 - 70 * max(0.0, k - 0.35) / 0.65)
            grad.set_at((x, 0), (v, v, v))
        big.blit(pygame.transform.scale(grad, (W * S, H * S)), (0, 0),
special_flags=pygame.BLEND_RGB_MULT)
    if state == "fading":
        _erode(big, 1.0, seed=hash(name) & 0xFFFF)
    elif state == "shadow":
        _erode(big, 0.25, seed=(hash(name) + 1) & 0xFFFF)
    surf = pygame.transform.smoothscale(big, (W, H))
    _cache[key] = surf
    return surf


def glow(accent, radius, strength=1.0):
    key = (accent, radius, int(strength * 10))
    if key not in _glow_cache:
        base = pygame.Surface((64, 64))
        for i in range(32, 0, -1):
            f = (1 - i / 32) ** 2 * strength * 0.55
            pygame.draw.circle(base, _mix(accent, f), (32, 32), i)
        _glow_cache[key] = pygame.transform.smoothscale(base, (radius * 2, radius * 2))
    return _glow_cache[key]


def draw_portrait(surface, name, state, cx, bottom, t, active=True, scale=1.0):
    """Портретті экранға салу: артқы жарқыл + тыныс алу қозғалысы.

    cx — ортасы, bottom — төменгі шеті. active=False болса күңгірттеледі."""
    img = build_portrait(name, state)
    accent = CHARACTERS[name]
    breathe = math.sin(t * 1.8 + hash(name) % 7) * 4
    w, h = int(W * scale), int(H * scale)
    if scale != 1.0:
        img = pygame.transform.smoothscale(img, (w, h))
    pos = img.get_rect(midbottom=(cx, bottom + breathe))
    g = glow(accent, int(330 * scale), 0.9 if state == "human" else 0.55 if state ==
"fading" else 0.3)
    surface.blit(g, g.get_rect(center=(cx, pos.top + 220 * scale)),
special_flags=pygame.BLEND_RGB_ADD)
    surface.blit(img, pos)
    if not active:
        dim = pygame.Surface((w, h), pygame.SRCALPHA)
        dim.fill((0, 0, 0, 120))
        surface.blit(dim, pos, special_flags=0)
    return pos
```

### Файл: kolenke/render/scenery.py

*Локация декорациялары: бір рет алдын ала сызылады (60 FPS үшін).*

```
"""Локация декорациялары: бір рет алдын ала сызылады (60 FPS үшін)."""
from dataclasses import dataclass, field

import pygame

H = 720
GROUND = 640       # кейіпкер табаны тұратын сызық
WALL_BOTTOM = 560


@dataclass
class Scenery:
    surface: pygame.Surface
    windows: list = field(default_factory=list)   # әлем координатасындағы Rect
    lights: list = field(default_factory=list)    # Lighting.render үшін
```

```
    pillars: list = field(default_factory=list)   # алдыңғы план бағандары (x)
    ambient: tuple = (16, 18, 28)


def _gradient(width, y0, y1, top, bottom, surf):
    for y in range(y0, y1):
        k = (y - y0) / max(1, y1 - y0 - 1)
        c = tuple(int(top[i] + (bottom[i] - top[i]) * k) for i in range(3))
        pygame.draw.line(surf, c, (0, y), (width, y))


def _floor(surf, width):
    _gradient(width, WALL_BOTTOM, H, (84, 66, 58), (36, 28, 26), surf)
    y, step = WALL_BOTTOM + 8, 10
    while y < H:
        pygame.draw.line(surf, (10, 8, 8), (0, y), (width, y))
        y += step
        step += 6
    for x in range(0, width, 170):
        pygame.draw.line(surf, (12, 10, 9), (x, WALL_BOTTOM), (x - 60, H))
    pygame.draw.line(surf, (60, 50, 46), (0, WALL_BOTTOM), (width, WALL_BOTTOM), 3)


def _window(surf, rect, windows):
    glass = pygame.Rect(rect)
    pygame.draw.rect(surf, (8, 10, 14), glass.inflate(16, 16), border_radius=4)
    for y in range(glass.top, glass.bottom):
        k = (y - glass.top) / glass.height
        c = (int(22 + 34 * k), int(38 + 44 * k), int(66 + 52 * k))
        pygame.draw.line(surf, c, (glass.left, y), (glass.right - 1, y))
    windows.append(glass)


def _door(surf, obj):
    x, w = obj.x, obj.w
    rect = pygame.Rect(0, 0, w, 330)
    rect.midbottom = (x, WALL_BOTTOM)
    pygame.draw.rect(surf, (12, 9, 8), rect.inflate(18, 14), border_radius=3)
    pygame.draw.rect(surf, (62, 42, 33), rect)
    for panel in (rect.inflate(-34, -190).move(0, -80), rect.inflate(-34, -190).move(0,
85)):
        pygame.draw.rect(surf, (48, 32, 26), panel)
        pygame.draw.rect(surf, (74, 52, 42), panel, 2)
    pygame.draw.circle(surf, (170, 140, 80), (rect.right - 16, rect.centery + 10), 6)
    plate = pygame.Rect(0, 0, 38, 20)
    plate.center = (rect.centerx, rect.top + 26)
    pygame.draw.rect(surf, (150, 130, 90), plate)


def _lamp_fixture(surf, x, y=64):
    pygame.draw.line(surf, (10, 10, 12), (x, 40), (x, y), 3)
    pygame.draw.rect(surf, (200, 190, 150), pygame.Rect(x - 22, y, 44, 10),
border_radius=4)


def build_corridor(loc, objects, night=0):
    w = loc.width
    surf = pygame.Surface((w, H))
    _gradient(w, 0, WALL_BOTTOM, (78, 92, 116), (48, 58, 76), surf)
    pygame.draw.rect(surf, (12, 14, 18), (0, 0, w, 60))
    for x in range(0, w, 300):
        pygame.draw.rect(surf, (18, 20, 25), (x, 0, 14, 60))
    pygame.draw.rect(surf, (30, 36, 45), (0, 390, w, WALL_BOTTOM - 390))
    pygame.draw.line(surf, (70, 80, 92), (0, 390), (w, 390), 3)
    _floor(surf, w)
    windows = []
```

```
    for x in (700, 1600):
        _window(surf, pygame.Rect(x - 90, 130, 180, 220), windows)
    lights, pillars = [], [1350]
    for x, flick, power in ((450, 0.8, 1), (1100, 0, 0), (1700, 0.35, 1), (2200, 0.1, 1)):
        _lamp_fixture(surf, x)
        if power:
            lights.append(dict(x=x, y=100, r=380, color=(255, 214, 160),
                               i=1.0, flicker=flick, seed=x))
    for o in objects:
        if o.kind == "door":
            _door(surf, o)
            if o.active(night):   # тек белсенді есіктің астынан жарық түседі
                lights.append(dict(x=o.x, y=WALL_BOTTOM - 20, r=190, color=(255, 190, 110),
                                   i=0.8, flicker=0.15, seed=o.x))
    for wnd in windows:  # терезеден түсетін суық жарық
        lights.append(dict(x=wnd.centerx, y=wnd.centery + 60, r=300,
                           color=(110, 150, 220), i=0.75, flicker=0, seed=0))
    return Scenery(surf, windows, lights, pillars, ambient=(120, 130, 168))


def _room_darkhan(surf, lights):
    pygame.draw.rect(surf, (20, 18, 24), (300, 470, 230, 90))      # кереует
    pygame.draw.rect(surf, (36, 40, 62), (306, 464, 218, 40), border_radius=6)
    pygame.draw.rect(surf, (14, 12, 16), (292, 420, 16, 140))
    pygame.draw.rect(surf, (28, 22, 20), (1010, 500, 160, 14))     # үстел мен шам
    pygame.draw.rect(surf, (20, 16, 14), (1020, 514, 12, 46))
    pygame.draw.rect(surf, (20, 16, 14), (1148, 514, 12, 46))
    pygame.draw.rect(surf, (230, 200, 120), (1080, 470, 14, 30), border_radius=3)
    for i in range(3):                                              # сөре
        pygame.draw.rect(surf, (22, 18, 18), (860, 250 + i * 58, 200, 8))
        for j in range(7):
            pygame.draw.rect(surf, [(80, 50, 46), (50, 64, 80), (90, 80, 52)][(i + j) % 3],
                             (868 + j * 26, 214 + i * 58, 18, 36))
    lights.append(dict(x=1087, y=470, r=420, color=(255, 190, 110), i=1.0, flicker=0.25,
seed=3))


def _room_girls(surf, lights):
    pygame.draw.ellipse(surf, (18, 14, 14), (260, 470, 150, 110))   # барабан
    pygame.draw.ellipse(surf, (60, 64, 70), (260, 462, 150, 40))
    pygame.draw.line(surf, (30, 24, 22), (290, 560), (270, 600), 5)
    pygame.draw.line(surf, (30, 24, 22), (380, 560), (400, 600), 5)
    pygame.draw.rect(surf, (28, 22, 20), (1000, 500, 200, 14))      # тігін машинасы үстелі
    pygame.draw.rect(surf, (20, 16, 14), (1015, 514, 12, 46))
    pygame.draw.rect(surf, (20, 16, 14), (1173, 514, 12, 46))
    pygame.draw.rect(surf, (50, 52, 60), (1060, 456, 80, 44), border_radius=6)
    pygame.draw.rect(surf, (70, 72, 82), (1110, 436, 12, 24))
    pygame.draw.circle(surf, (150, 60, 90), (1075, 440), 10)         # катушка
    for i in range(4):                                               # маталар
        pygame.draw.rect(surf, [(110, 60, 90), (60, 90, 100), (100, 90, 50), (70, 60, 110)]
[i],
                         (860 + i * 28, 300, 22, 120))
    lights.append(dict(x=1100, y=470, r=420, color=(255, 190, 120), i=1.0, flicker=0.2,
seed=5))


def _room_bekzat(surf, lights):
    pygame.draw.rect(surf, (28, 22, 20), (980, 500, 200, 14))       # үстел
    pygame.draw.rect(surf, (20, 16, 14), (995, 514, 12, 46))
    pygame.draw.rect(surf, (20, 16, 14), (1153, 514, 12, 46))
    pygame.draw.polygon(surf, (36, 28, 24), [(1060, 500), (1120, 500), (1105, 420), (1075,
420)])  # метроном
    pygame.draw.line(surf, (190, 180, 150), (1090, 500), (1100, 428), 3)
    for k in range(6):                                               # ноталар/қағаздар
        pygame.draw.rect(surf, (150, 148, 138), (860 + k * 22, 520 - (k % 2) * 6, 18, 24))
    pygame.draw.rect(surf, (20, 18, 24), (300, 470, 230, 90))        # кереует
```

```
    pygame.draw.rect(surf, (52, 40, 40), (306, 464, 218, 40), border_radius=6)
    lights.append(dict(x=1090, y=450, r=300, color=(255, 215, 150), i=0.8, flicker=0.55,
seed=9))


ROOM_VARIANTS = {"darkhan_room": _room_darkhan, "girls_room": _room_girls,
                 "bekzat_room": _room_bekzat}
ROOM_AMBIENT = {"darkhan_room": (140, 124, 148), "girls_room": (142, 126, 150),
                "bekzat_room": (122, 118, 150)}


def build_room(loc, objects, night=0):
    w = loc.width
    surf = pygame.Surface((w, H))
    _gradient(w, 0, WALL_BOTTOM, (104, 84, 92), (66, 54, 64), surf)
    pygame.draw.rect(surf, (14, 12, 16), (0, 0, w, 40))
    _floor(surf, w)
    windows = []
    _window(surf, pygame.Rect(540, 120, 200, 240), windows)
    lights = [dict(x=640, y=330, r=320, color=(110, 150, 220), i=0.7, flicker=0, seed=0)]
    ROOM_VARIANTS.get(loc.id, _room_darkhan)(surf, lights)
    for o in objects:
        if o.kind == "door":
            _door(surf, o)
    return Scenery(surf, windows, lights, [], ambient=ROOM_AMBIENT.get(loc.id, (140, 124,
148)))


def build_scenery(loc, night=0):
    builder = build_corridor if loc.id == "corridor" else build_room
    return builder(loc, loc.objects, night)
```

### Файл: kolenke/render/silhouette.py

*Силуэт + бір акцентті түс: кейіпкерлерді код арқылы сызу.*

```
"""Силуэт + бір акцентті түс: кейіпкерлерді код арқылы сызу.

Дене қара, акцент (мойынорағыш пен көз) — жарықтан тыс «жарқырайды».
detail 1.0 → толық, 0.0 → детальсіз қара дақ (көлеңке жұтқанда азаяды).
arm_angle: алдыңғы қолдың бұрышы (0 — төмен, π/2 — алға, π — жоғары).
"""
import math

import pygame

BODY = (5, 5, 8)
STEEL = (150, 156, 168)
_shadow_cache = {}


def _fade(c, k):
    return tuple(int(BODY[i] + (c[i] - BODY[i]) * k) for i in range(3))


def _shadow(width):
    key = int(width)
    if key not in _shadow_cache:
        surf = pygame.Surface((key, max(6, key // 5)), pygame.SRCALPHA)
        pygame.draw.ellipse(surf, (0, 0, 0, 110), surf.get_rect())
        _shadow_cache[key] = surf
    return _shadow_cache[key]


def draw_silhouette(surface, x, ground_y, accent, facing=1, swing=0.0,
                    scale=1.0, detail=1.0, t=0.0, part="all",
                    arm_angle=None, lean=0.0, weapon=0.0, body_color=None):
```

```
    """part: "body" (жарықтандыруға дейін), "accent" (кейін, эмиссия), "all".

    Қайтарады: (қол, пышақ ұшы) нүктелері — тек body кезінде және arm_angle берілсе.
    """
    s = scale
    col = body_color or BODY
    painted = body_color is None and detail > 0       # түсті киім (көлеңке жұтқанда
қараяды)
    if painted:
        coat = _fade(tuple(int(accent[i] * 0.42 + (40, 46, 70)[i] * 0.58) for i in
range(3)), detail)
        pants = _fade((34, 38, 56), detail)
        skin = _fade((232, 190, 154), detail)
        hair = _fade((46, 34, 30), detail)
        edge = _fade(tuple(int(v * 1.35) for v in coat), detail)
    else:
        coat = pants = skin = hair = edge = col
    hip_y = ground_y - 82 * s
    shoulder_y = ground_y - 142 * s
    head_y = ground_y - 162 * s
    lx = lean * s                       # денені еңкейту (алға оң)
    hx = x + facing * lx                # бас/иық орны
    sx = x + facing * lx * 0.5
    reach = None
    if part in ("body", "all"):
        sh = _shadow(70 * s)
        surface.blit(sh, sh.get_rect(center=(x, ground_y + 2)))
        for side in (-1, 1):  # аяқтар
            lift = max(0.0, swing * side) * 7 * s
            foot = (x + side * 7 * s + swing * side * 20 * s, ground_y - lift)
            pygame.draw.line(surface, pants, (x + side * 7 * s, hip_y), foot,
                             max(1, int(10 * s)))
        pygame.draw.polygon(surface, coat, [  # пальто
            (sx - 19 * s, shoulder_y), (sx + 19 * s, shoulder_y),
            (x + 25 * s, hip_y + 14 * s), (x - 25 * s, hip_y + 14 * s)])
        if painted:                                     # жарық жиегі + белбеу
            pygame.draw.line(surface, edge, (sx - 19 * s, shoulder_y), (x - 25 * s, hip_y +
14 * s),
                             max(1, int(3 * s)))
            pygame.draw.line(surface, pants, (x - 24 * s, hip_y + 8 * s), (x + 24 * s,
hip_y + 8 * s),
                             max(1, int(4 * s)))
        for side in (-1, 1):  # қолдар
            shoulder = (sx + side * 18 * s, shoulder_y + 4 * s)
            if arm_angle is not None and side == facing:
                d = (facing * math.sin(arm_angle), math.cos(arm_angle))
                hand = (shoulder[0] + d[0] * 52 * s, shoulder[1] + d[1] * 52 * s)
                pygame.draw.line(surface, coat, shoulder, hand, max(1, int(8 * s)))
                pygame.draw.circle(surface, skin, (int(hand[0]), int(hand[1])), max(1,
int(4 * s)))
                tip = hand
                if weapon:
                    tip = (hand[0] + d[0] * weapon * s, hand[1] + d[1] * weapon * s)
                    pygame.draw.line(surface, STEEL if not body_color else col,
                                     hand, tip, max(1, int(4 * s)))
                reach = (hand, tip)
            else:
                hand = (x + side * 22 * s - swing * side * 12 * s, shoulder_y + 52 * s)
                pygame.draw.line(surface, coat, shoulder, hand, max(1, int(8 * s)))
                pygame.draw.circle(surface, skin, (int(hand[0]), int(hand[1])), max(1,
int(4 * s)))
        pygame.draw.line(surface, skin, (sx, shoulder_y), (hx, head_y), int(9 * s))
        pygame.draw.circle(surface, skin, (int(hx), int(head_y)), max(2, int(15 * s)))
        if painted:                                     # шаш
            r = max(2, int(15 * s))
```

```
            pygame.draw.circle(surface, hair, (int(hx - facing * 2 * s), int(head_y - 3 *
s)), r,
                               draw_top_left=True, draw_top_right=True)
            pygame.draw.circle(surface, hair, (int(hx - facing * 8 * s), int(head_y)),
int(9 * s))
    if part in ("accent", "all") and detail > 0:
        c = tuple(int(BODY[i] + (accent[i] - BODY[i]) * detail) for i in range(3))
        neck_y = shoulder_y - 3 * s
        pygame.draw.rect(surface, c, pygame.Rect(sx - 14 * s, neck_y - 3 * s,
                                                 28 * s, 8 * s), border_radius=int(3 * s))
        wave = math.sin(t * 6) * 5 * s
        tx = sx - facing * 12 * s
        pygame.draw.polygon(surface, c, [
            (tx, neck_y), (tx + facing * 4 * s, neck_y + 4 * s),
            (tx - facing * 30 * s, neck_y + 10 * s + wave),
            (tx - facing * 30 * s, neck_y + 2 * s + wave)])
        if detail > 0.6:  # көз
            pygame.draw.circle(surface, c, (int(hx + facing * 6 * s),
                                            int(head_y - 2 * s)), max(1, int(2 * s)))
    return reach
```

### Файл: kolenke/render/ui.py

*Экрандағы интерфейс элементтері: түн индикаторы, журнал есептегіші.*

```
"""Экрандағы интерфейс элементтері: түн индикаторы, журнал есептегіші."""
import pygame

GREY = (120, 120, 120)
ACCENT = (200, 60, 60)
WHITE = (230, 230, 230)


def draw_night_indicator(surface, font, label, night, total=3, pos=(40, 28)):
    """«1-түн» жазуы және ағымдағы түнді көрсететін үш шеңбер."""
    x, y = pos
    surface.blit(font.render(label, True, WHITE), (x, y))
    cx = x + font.size(label)[0] + 30
    for i in range(1, total + 1):
        center = (cx + (i - 1) * 26, y + font.get_height() // 2)
        if i < night:
            pygame.draw.circle(surface, GREY, center, 8)
        elif i == night:
            pygame.draw.circle(surface, ACCENT, center, 8)
        else:
            pygame.draw.circle(surface, GREY, center, 8, 2)


def draw_journal_counter(surface, font, count, right_edge):
    text = font.render(f"Tab · {count}", True, GREY)
    surface.blit(text, (right_edge - text.get_width(), 30))
```

### Файл: kolenke/scenes/dialogue.py

*Диалог сахнасы (визуалды новелла) және естеліктер журналы.*

```
"""Диалог сахнасы (визуалды новелла) және естеліктер журналы."""
import random

import pygame

from kolenke.core.assets import ROOT
from kolenke.core.scene import Scene
from kolenke.core.text import wrap_text
from kolenke.explore.locations import Location
from kolenke.render.corruption import draw_corruption, hidden
from kolenke.render.lighting import Lighting, make_vignette
from kolenke.render.particles import Blizzard
```

```
from kolenke.render.portrait import CHARACTERS, draw_portrait
from kolenke.render.scenery import build_scenery
from kolenke.story.graph import DialogueGraph, DialogueRunner

ACCENT = (200, 60, 60)
WHITE = (232, 232, 238)
NARRATOR = (170, 178, 196)
GREY = (125, 125, 140)
CHARS_PER_SEC = 60
CHOICE_ACTIONS = {"choice1": 0, "choice2": 1, "choice3": 2, "choice4": 3}
_bg_cache = {}


def blurred_room(room, size):
    """Бөлме фоны: жарықтандырылып, бұлыңғыр етіледі (новелла фоны)."""
    key = (room, size)
    if key not in _bg_cache:
        w, h = size
        sc = build_scenery(Location(room, w, "", w // 2, []))
        img = sc.surface.copy()
        Lighting(size, sc.ambient).render(img, sc.lights, 0, 0.0)
        small = pygame.transform.smoothscale(img, (w // 10, h // 10))
        small = pygame.transform.smoothscale(small, (w // 25, h // 25))
        img = pygame.transform.smoothscale(small, (w, h))
        lift = pygame.Surface(size)
        lift.fill((58, 60, 84))
        img.blit(lift, (0, 0), special_flags=pygame.BLEND_RGB_ADD)   # қараңғыда да
көрінсін
        shade = pygame.Surface(size)
        shade.fill((240, 240, 248))
        img.blit(shade, (0, 0), special_flags=pygame.BLEND_RGB_MULT)
        _bg_cache[key] = img
    return _bg_cache[key]


def panel(width, height, alpha=215, border=None, radius=14):
    surf = pygame.Surface((width, height), pygame.SRCALPHA)
    pygame.draw.rect(surf, (6, 8, 14, alpha), surf.get_rect(), border_radius=radius)
    if border:
        pygame.draw.rect(surf, border, surf.get_rect(), 2, border_radius=radius)
    return surf


def glitch(text, level, t, node_id):
    """3-түн: мәтіннің кейбір әріптері жоғалады (естеліктер бұзылады)."""
    if level < 0.35:
        return text
    p = (level - 0.35) * 0.3
    bucket = int(t / 1.2)
    out = []
    for i, ch in enumerate(text):
        r = random.Random(hash((i, bucket, node_id)) & 0xFFFFFF)
        out.append("▒" if ch != " " and r.random() < p else ch)
    return "".join(out)


class JournalScene(Scene):
    """Tab басқанда ашылатын естеліктер журналы (кейіпкер бойынша топтастырылған)."""

    def __init__(self, game):
        super().__init__(game)
        self.t = 0.0
        w, h = game.screen.get_size()
        self.size = (w, h)
        self.snow = Blizzard(w, h, count=90, wind=-120, seed=2)
```

```
    def handle_action(self, action):
        if action in ("journal", "menu"):
            self.game.pop()

    def update(self, dt):
        self.t += dt
        self.snow.update(dt)

    def draw(self, surface):
        s, a = self.game.strings, self.game.assets
        w, h = self.size
        surface.blit(blurred_room("darkhan_room", self.size), (0, 0))
        dim = pygame.Surface(self.size)
        dim.set_alpha(150)
        surface.blit(dim, (0, 0))
        self.snow.draw(surface, brightness=0.3)
        surface.blit(panel(w - 120, h - 150, 205, (60, 62, 80)), (60, 80))
        title = a.font(38, bold=True).render(s["journal_title"], True, WHITE)
        surface.blit(title, (100, 100))
        entries = self.game.journal.entries()
        font = a.font(23)
        if not entries:
            surface.blit(font.render(s["journal_empty"], True, GREY), (100, 180))
        y = 170
        last_who = None
        for _, mem in sorted(entries, key=lambda e: list(CHARACTERS).index(e[1]["who"])
                             if e[1]["who"] in CHARACTERS else 99):
            who = mem["who"]
            col = CHARACTERS.get(who, ACCENT)
            if who != last_who:
                y += 8 if last_who else 0
                pygame.draw.circle(surface, col, (112, y + 12), 7)
                surface.blit(a.font(22, bold=True).render(s.get("who_" + who, who), True,
col),
                             (130, y))
                y += 34
                last_who = who
            for line in wrap_text(font, mem["text"], w - 300):
                if y > h - 110:
                    break
                surface.blit(font.render(line, True, WHITE), (130, y))
                y += 30
            y += 6
        hint = a.font(20).render(s["journal_hint"], True, GREY)
        surface.blit(hint, hint.get_rect(center=(w // 2, h - 48)))


class DialogueScene(Scene):
    def __init__(self, game, graph_path, night=1):
        super().__init__(game)
        self.night = night
        self.graph = DialogueGraph.load(graph_path)
        game.journal.catalog.update(self.graph.memories)
        self.runner = DialogueRunner(self.graph, game.flags, game.journal)
        self.t = 0.0
        self.shown = 0.0       # көрсетілген таңба саны (теру эффектісі)
        self.notice_time = 0.0  # «жаңа жазба» хабарының уақыты
        self._known = len(game.journal)
        w, h = game.screen.get_size()
        self.size = (w, h)
        self.bg = blurred_room(self.graph.bg, self.size)
        self.blizzard = Blizzard(w, h, count=120, wind=-200, seed=3)
        self.vignette = make_vignette((w, h))
        self._last_states = dict(self.runner.states)
        self._reset_text()
```

```
def on_enter(self):
    cast = self.graph.cast
    motif = self.game.audio.motif_for(cast[0]) if cast else None
    if motif:
        self.game.audio.play(motif, 0.45)       # кейіпкер мотиві

def _reset_text(self):
    self.shown = 0.0
    n = len(self.game.journal)
    if n > self._known:
        self.notice_time = 2.5
        self.game.audio.play("chime", 0.5)
    self._known = n
    if self.runner.states != self._last_states:    # көлеңке жұта бастады
        self.game.audio.play("swell", 0.8)
        self._last_states = dict(self.runner.states)

@property
def text(self):
    return self.runner.node.get("text", "")

@property
def text_done(self):
    return self.shown >= len(self.text)

def handle_action(self, action):
    r = self.runner
    if action == "journal":
        self.game.push(JournalScene(self.game))
    elif action == "menu":
        self.game.pop()
    elif action == "next":
        if not self.text_done:
            self.shown = len(self.text)
        elif r.finished:
            self.game.flags.set(self.graph.id + "_talk_done")
            self.game.pop()
        elif r.advance():
            self._reset_text()
    elif action in CHOICE_ACTIONS and self.text_done and r.has_choices:
        if r.choose(CHOICE_ACTIONS[action]):
            self.game.audio.play("click", 0.6)
            self._reset_text()

def update(self, dt):
    self.t += dt
    self.shown = min(len(self.text), self.shown + CHARS_PER_SEC * dt)
    self.notice_time = max(0.0, self.notice_time - dt)
    self.blizzard.update(dt)
    self.game.tick_corruption(dt)

# --- сызу ---
def _cast_positions(self):
    cast = self.graph.cast
    if len(cast) == 1:
        return {cast[0]: 880}
    if len(cast) == 2:
        return {cast[0]: 360, cast[1]: 920}
    return {c: 200 + i * 300 for i, c in enumerate(cast)}

def draw(self, surface):
    s, a, r = self.game.strings, self.game.assets, self.runner
    w, h = self.size
    c = self.game.corruption
    surface.blit(self.bg, (0, 0))
    self.blizzard.draw(surface, brightness=0.3)
```

```
        node = r.node
        speaker = node.get("speaker", "")
        for name, cx in self._cast_positions().items():
            active = speaker == name or speaker not in self.graph.cast
            draw_portrait(surface, name, r.states.get(name, "human"), cx, h + 40, self.t,
active)
        surface.blit(self.vignette, (0, 0))
        draw_corruption(surface, c, self.t)
        self._draw_box(surface, node, speaker, c)
        if not hidden(c, 0.7):
            night = a.font(22).render(s["night_label"].format(n=self.night), True, GREY)
            surface.blit(night, (40, 30))
        if not hidden(c, 0.55):
            j = a.font(22).render(f"Tab · {len(self.game.journal)}", True, GREY)
            surface.blit(j, (w - j.get_width() - 40, 30))
        if self.notice_time > 0:
            note = a.font(24, bold=True).render(s["new_memory"], True, (255, 230, 170))
            box = panel(note.get_width() + 50, 46, 220, (255, 230, 170))
            surface.blit(box, box.get_rect(center=(w // 2, 90)))
            surface.blit(note, note.get_rect(center=(w // 2, 90)))

    def _draw_box(self, surface, node, speaker, c):
        s, a, r = self.game.strings, self.game.assets, self.runner
        w, h = self.size
        choices = r.choices() if (self.text_done and r.has_choices) else []
        bh = 190 + 54 * len(choices) if choices else 200
        box = pygame.Rect(70, h - bh - 24, w - 140, bh)
        who_col = CHARACTERS.get(speaker, NARRATOR)
        surface.blit(panel(box.width, box.height, 200, (110, 112, 140)), box.topleft)
        pygame.draw.line(surface, who_col, (box.x + 18, box.y + 1), (box.x + 190, box.y +
1), 3)
        name = self.graph.speakers.get(speaker, "")
        if name:
            plate = a.font(28, bold=True).render(name, True, who_col)
            tab = panel(plate.get_width() + 44, 44, 235, who_col, 10)
            surface.blit(tab, (box.x + 24, box.y - 30))
            surface.blit(plate, (box.x + 46, box.y - 24))
        font = a.font(30)
        col = NARRATOR if speaker in ("narrator", "") else WHITE
        text = glitch(self.text[: int(self.shown)], c, self.t, r.node_id)
        y = box.y + 28
        for line in wrap_text(font, text, box.width - 80):
            surface.blit(font.render(line, True, col), (box.x + 40, y))
            y += 42
        if choices:
            y = max(y + 10, box.y + 104)
            for n, (_, ch) in enumerate(choices, start=1):
                row = pygame.Rect(box.x + 40, y, box.width - 80, 44)
                surface.blit(panel(row.width, row.height, 200, (90, 92, 120), 10),
row.topleft)
                pygame.draw.circle(surface, ACCENT, (row.x + 26, row.centery), 14)
                num = a.font(20, bold=True).render(str(n), True, WHITE)
                surface.blit(num, num.get_rect(center=(row.x + 26, row.centery)))
                label = a.font(26).render(glitch(ch["text"], c, self.t, r.node_id +
str(n)),
                                          True, WHITE)
                surface.blit(label, (row.x + 56, row.centery - label.get_height() // 2))
                y += 54
        elif self.text_done:
            if r.finished:
                done = a.font(22).render(s["night_done"], True, GREY)
                surface.blit(done, (box.right - done.get_width() - 40, box.bottom - 40))
            elif int(self.t * 2) % 2 == 0:       # ▼ — келесі реплика
                pygame.draw.polygon(surface, who_col, [(box.right - 60, box.bottom - 38),
                                                       (box.right - 40, box.bottom - 38),
                                                       (box.right - 50, box.bottom - 24)])
```

```
        if not hidden(c, 0.35):
            hint = a.font(18).render(s["dialogue_hint"], True, GREY)
            surface.blit(hint, hint.get_rect(center=(w // 2, h - 12)))
```

### Файл: kolenke/scenes/explore.py

*Жатақханада жүру: қозғалыс, жарық, боран, есіктер, сөйлесу мен шайқасты бастау.*

```
"""Жатақханада жүру: қозғалыс, жарық, боран, есіктер, сөйлесу мен шайқасты бастау."""
import math

import pygame

from kolenke.core.assets import ROOT
from kolenke.core.scene import Scene
from kolenke.explore.locations import load_locations
from kolenke.explore.player import Player
from kolenke.render.corruption import draw_corruption, hidden, protagonist_detail
from kolenke.render.effects import Fade
from kolenke.render.lighting import Lighting, make_vignette
from kolenke.render.particles import Blizzard, ParticleSystem
from kolenke.render.scenery import GROUND, build_scenery
from kolenke.render.silhouette import draw_silhouette
from kolenke.render.ui import draw_journal_counter, draw_night_indicator
from kolenke.scenes.dialogue import DialogueScene, JournalScene
from kolenke.scenes.fight import FightScene

LOCATIONS_PATH = ROOT / "kolenke" / "data" / "locations.json"
DATA_DIR = ROOT / "kolenke" / "data"
PLAYER_ACCENT = (110, 170, 230)
WHITE = (230, 230, 230)
GREY = (120, 120, 120)
ACCENT = (200, 60, 60)
CAM_LERP = 6.0
NIGHT_NPCS = {}   # кэш: түн -> NPC тізімі


class ExploreScene(Scene):
    def __init__(self, game, start="corridor", night=None):
        super().__init__(game)
        self.night = night or game.night
        self.locs = load_locations(LOCATIONS_PATH)
        self._scenery = {}
        self.t = 0.0
        w, h = game.screen.get_size()
        self.size = (w, h)
        self.blizzard = Blizzard(w, h, count=380, wind=-180)
        self.dust = ParticleSystem()
        self.lighting = Lighting((w, h))
        self.vignette = make_vignette((w, h))
        self.fade = Fade((w, h))
        self._summary_shown = False
        self._reset(start)

    def _reset(self, start="corridor"):
        self.loc = self.locs[start]
        self.player = Player(self.loc.start)
        self.cam = self._target_cam()
        self._last_swing = 0.0
        self._summary_shown = False

    # --- көмекші ---
    @property
    def scenery(self):
        key = (self.loc.id, self.night)
        if key not in self._scenery:
            self._scenery[key] = build_scenery(self.loc, self.night)
```

```
    return self._scenery[key]

def _target_cam(self):
    return max(0.0, min(self.loc.width - self.game.screen.get_width(),
                        self.player.x - self.game.screen.get_width() / 2))

# --- NPC күйі ---
def _fight_won(self, name):
    return bool(self.game.flags.get(name + "_fight_won"))

def _talk_done(self, o):
    return bool(self.game.flags.get(o.done_flag))

def _next_enemy(self, o):
    """Диалогтан кейін: осы NPC үшін әлі жеңілмеген бірінші қарсылас."""
    for name in o.enemies(self.game.flags):
        if not self._fight_won(name):
            return name
    return None

def _npc_finished(self, o):
    if not self._talk_done(o):
        return False
    return self._next_enemy(o) is None

def _npc_objects(self, night=None):
    night = night or self.night
    return [o for loc in self.locs.values() for o in loc.objects
            if o.kind == "npc" and o.active(night)]

def night_complete(self):
    npcs = self._npc_objects()
    return bool(npcs) and all(self._npc_finished(o) for o in npcs)

def _visible_npc(self, o):
    return o.active(self.night) and not self.game.flags.get(o.who + "_killed")

def interactables(self):
    out = []
    for o in self.loc.objects:
        if not o.active(self.night):
            continue
        if o.kind == "npc" and self._npc_finished(o):
            continue
        if abs(self.player.x - o.x) <= o.w / 2 + 40:
            out.append(o)
    return sorted(out, key=lambda o: abs(self.player.x - o.x))

@property
def near(self):
    found = self.interactables()
    return found[0] if found else None

def _prompt_label(self, o):
    if o.kind == "npc" and self._talk_done(o):
        enemy = self._next_enemy(o)
        if enemy:
            return "obj_fight_" + enemy
    return o.label

def _go(self, door):
    self.loc = self.locs[door.to]
    self.player.x = float(door.spawn)
    self.player.vx = 0.0
    self.cam = self._target_cam()
```

```
# --- енгізу ---
def handle_action(self, action):
    if self.fade.active:
        return
    if action == "menu":
        self.game.pop()
    elif action == "journal":
        self.game.push(JournalScene(self.game))
    elif action == "interact":
        obj = self.near
        if obj is None:
            return
        self.game.audio.play("click", 0.6)
        if obj.kind == "door":
            self.fade.start(lambda o=obj: self._go(o))
        elif obj.kind == "npc":
            if self._talk_done(obj):
                enemy = self._next_enemy(obj)
                if enemy:
                    self.game.push(FightScene(self.game, enemy, night=self.night))
            else:
                self.game.push(DialogueScene(self.game, DATA_DIR / obj.dialogue,
                                             night=self.night))

def on_resume(self):
    if self.night != self.game.night:      # жаңа түн басталды
        self.night = self.game.night
        self._scenery.clear()
        self._reset("corridor")

# --- логика ---
def update(self, dt):
    self.t += dt
    self.game.tick_corruption(dt)
    self.fade.update(dt)
    g = self.game
    if not self.fade.active:
        self.player.update(dt, g.is_held("left"), g.is_held("right"), self.loc.width)
    sw = self.player.swing
    if self.player.moving and sw * self._last_swing < 0:  # қадам -> шаң
        for k in range(3):
            self.dust.emit(self.player.x - self.player.facing * 8, GROUND - 4,
                           -self.player.facing * 30 + (hash((self.t, k)) % 40 - 20),
                           -25 - (hash((k, self.t)) % 20), 0.5, 3, (120, 112, 105), 40)
    self._last_swing = sw
    self.cam += (self._target_cam() - self.cam) * min(1.0, CAM_LERP * dt)
    self.blizzard.update(dt)
    self.dust.update(dt)
    if self.night_complete() and not self._summary_shown and not self.fade.active:
        self._summary_shown = True
        from kolenke.scenes.night_end import NightEndScene
        self.game.push(NightEndScene(self.game, self.night))

# --- сызу ---
def draw(self, surface):
    sc = self.scenery
    w, h = self.size
    cam = int(self.cam)
    surface.blit(sc.surface, (0, 0), pygame.Rect(cam, 0, w, h))
    for wnd in sc.windows:  # терезе артындағы боран
        r = wnd.move(-cam, 0)
        if r.right < 0 or r.left > w:
            continue
        prev = surface.get_clip()
        surface.set_clip(r)
        self.blizzard.draw(surface, brightness=0.9)
```

```
            surface.set_clip(prev)
            pygame.draw.line(surface, (10, 12, 16), (r.centerx, r.top), (r.centerx,
r.bottom), 4)
            pygame.draw.line(surface, (10, 12, 16), (r.left, r.centery), (r.right,
r.centery), 4)
        npcs = [o for o in self.loc.objects if o.kind == "npc" and self._visible_npc(o)]
        for o in npcs:
            draw_silhouette(surface, o.x - cam, GROUND, o.accent, facing=-1,
                            t=self.t, part="body")
        p = self.player
        c = self.game.corruption
        draw_silhouette(surface, p.x - cam, GROUND, PLAYER_ACCENT, p.facing,
                        p.swing, t=self.t, part="body")
        self.dust.draw(surface, cam)

        lights = sc.lights + [dict(x=p.x, y=GROUND - 150, r=320, color=(190, 215, 255),
                                   i=0.9 * (1 - 0.5 * c), flicker=0, seed=0)]
        self.lighting.ambient = sc.ambient
        self.lighting.render(surface, lights, cam, self.t)

        for o in npcs:  # акцент жарықтан кейін — «жарқырайды»
            draw_silhouette(surface, o.x - cam, GROUND, o.accent, facing=-1,
                            t=self.t, part="accent")
        draw_silhouette(surface, p.x - cam, GROUND, PLAYER_ACCENT, p.facing, p.swing,
                        t=self.t, part="accent", detail=protagonist_detail(c))
        for px in sc.pillars:  # алдыңғы план: параллакс 1.25x
            sx = int(px - self.cam * 1.25)
            if -100 < sx < w:
                pygame.draw.rect(surface, (3, 3, 5), (sx - 30, 0, 60, h))
                pygame.draw.line(surface, (22, 24, 30), (sx - 30, 0), (sx - 30, h), 2)
        surface.blit(self.vignette, (0, 0))
        draw_corruption(surface, c, self.t)
        self._draw_hud(surface, cam)
        self.fade.draw(surface)

    def _draw_hud(self, surface, cam):
        s, a = self.game.strings, self.game.assets
        w, h = self.size
        c = self.game.corruption
        if not hidden(c, 0.7):
            draw_night_indicator(surface, a.font(22),
s["night_label"].format(n=self.night),
                                 self.night)
        if not hidden(c, 0.55):
            draw_journal_counter(surface, a.font(22), len(self.game.journal), w - 40)
        if not hidden(c, 0.4):
            name = a.font(22).render(s[self.loc.name], True, GREY)
            surface.blit(name, name.get_rect(midtop=(w // 2, 30)))
        if not hidden(c, 0.35):
            hint = a.font(18).render(s["controls_explore"], True, GREY)
            surface.blit(hint, hint.get_rect(center=(w // 2, h - 30)))
        talked = [o for o in self._npc_objects() if self._talk_done(o)]
        if talked and not self.night_complete() and not hidden(c, 0.5):
            msg = a.font(24).render(s["fight_ready_hint"], True, ACCENT)
            surface.blit(msg, msg.get_rect(center=(w // 2, h - 80)))
        obj = self.near
        if obj and not self.fade.active:
            bob = math.sin(self.t * 4) * 4
            x = obj.x - cam
            y = GROUND - 250 + bob
            label = a.font(22).render(s[self._prompt_label(obj)], True, WHITE)
            box = pygame.Rect(0, 0, label.get_width() + 70, 40)
            box.midbottom = (x, y)
            pygame.draw.rect(surface, (8, 8, 12), box, border_radius=8)
            pygame.draw.rect(surface, ACCENT, box, 2, border_radius=8)
            key = a.font(20, bold=True).render("E", True, ACCENT)
```

```
            surface.blit(key, (box.left + 16, box.centery - key.get_height() // 2))
            surface.blit(label, (box.left + 50, box.centery - label.get_height() // 2))
```

### Файл: kolenke/scenes/fight.py

*Жекпе-жек сахнасы: кадр дәлдігіндегі ядро + кино-эффектілер.*

```
"""Жекпе-жек сахнасы: кадр дәлдігіндегі ядро + кино-эффектілер.

Логика бекітілген 60 Гц қадаммен (Fight.tick), сызу — кез келген FPS-те.
Эффектілер: hitstop, экран діріліі, парирлеудегі баяулау, ұшқындар, жарық жарқылы.
"""
import math
import random

import pygame

from kolenke.combat.ai.adaptive import AdaptiveAI
from kolenke.combat.fight import MEMORY_COST, Fight, PlayerController
from kolenke.combat.setup import RESTORE_RESERVE_COST, fight_params
from kolenke.combat.hitbox import GROUND
from kolenke.core.scene import Scene
from kolenke.explore.locations import Location
from kolenke.render.corruption import draw_corruption, hidden, protagonist_detail
from kolenke.render.effects import ScreenShake
from kolenke.render.fighter_draw import draw_fighter
from kolenke.render.lighting import Lighting, make_vignette
from kolenke.render.particles import Blizzard, ParticleSystem
from kolenke.render.scenery import build_scenery

TICK = 1 / 60
ARENA_ROOM = {"darkhan": "darkhan_room", "zhanel": "girls_room",
              "madina": "girls_room", "bekzat": "bekzat_room"}
WHITE = (230, 230, 230)
GREY = (120, 120, 120)
ACCENT = (200, 60, 60)
SLOW_SCALE = 0.25
SPARK = (255, 220, 150)
PARRY_COL = (170, 220, 255)


class FightScene(Scene):
    def __init__(self, game, enemy="darkhan", night=1, ai=None, seed=None,
                 tells_enabled=True):
        super().__init__(game)
        self.tells_enabled = tells_enabled
        self.enemy_name = enemy
        self.night = night
        self.ai_factory = ai or (lambda: AdaptiveAI.for_night(
            night, seed=random.randint(0, 9999) if seed is None else seed))
        w, h = game.screen.get_size()
        self.size = (w, h)
        self.canvas = pygame.Surface((w, h))
        room = ARENA_ROOM.get(enemy, "darkhan_room")
        self.scenery = build_scenery(Location(room, w, "", w // 2, []))
        self.lighting = Lighting((w, h))
        self.vignette = make_vignette((w, h))
        self.blizzard = Blizzard(w, h, count=260, wind=-180)
        self.particles = ParticleSystem(limit=500)
        self.shake = ScreenShake()
        self.zoom = 0.0          # кино-зум (парирлеу/КО кезінде)
        self.zoom_pt = (w // 2, GROUND - 100)
        self.t = 0.0
        self._start()

    def _start(self):
        params = fight_params(self.game.flags, self.game.journal, self.game.reserve,
```

```
                        self.enemy_name)
        self.tells_visible = self.tells_enabled
        self.fight = Fight(self.enemy_name, enemy_controller=self.ai_factory(), **params)
        self.fight.controllers[self.fight.player] = PlayerController(
            self.fight.buffer, self.game.is_held)
        self.acc = 0.0
        self.slow_left = 0.0
        self.time_scale = 1.0
        self.flashes = []     # [x, y, radius, life, max_life, color]
        self.popups = []      # [text, x, y, life, color]
        self.trails = {"player": [], self.enemy_name: []}
        self.result = None
        self.result_delay = 0.0
        self._cue_tick = -1
        self.stats = {"parries": 0, "hits_taken": 0, "hits_given": 0}

    # --- енгізу ---
    def handle_action(self, action):
        if action == "menu":
            self.game.pop()
        elif self.result:
            if action in ("next", "confirm"):
                if self.result == "win":
                    self._finish()
                    self.game.pop()
                else:
                    self._start()
        elif action in ("light", "heavy", "block"):
            self.fight.push_input(action)
        elif action in ("choice1", "choice2", "choice3"):
            self.fight.push_input("mem" + action[-1])

    def _finish(self):
        """Нәтижені флагтарға жазу: өлтіру немесе адамдық кейіпке қайтару."""
        f, name = self.game.flags, self.enemy_name
        f.set(name + "_fight_won")
        if self.fight.outcome == "restore":
            f.set(name + "_saved")
            self.game.reserve = max(0, self.game.reserve - RESTORE_RESERVE_COST)
        else:
            f.set(name + "_killed")

    # --- логика ---
    def update(self, dt):
        self.t += dt
        self.game.tick_corruption(dt)
        self.shake.update(dt)
        self.zoom = max(0.0, self.zoom - dt * 0.22)
        if self.slow_left > 0:
            self.slow_left -= dt
        self.time_scale = SLOW_SCALE if self.slow_left > 0 else min(
            1.0, self.time_scale + dt * 3)
        sim_dt = dt * self.time_scale
        if not self.fight.over or self.result is None:
            self.acc += sim_dt
            n = 0
            while self.acc >= TICK and n < 4:
                self.acc -= TICK
                n += 1
                for ev in self.fight.tick():
                    self._on_event(ev)
        self._audio_cues()
        self.particles.update(sim_dt)
        self.blizzard.update(sim_dt)
        self._update_flashes(sim_dt)
        self._update_popups(dt)
```

```
        self._update_trails()
        if self.fight.over and self.result is None:
            self.result_delay += dt
            if self.result_delay > (0.9 if self.fight.outcome == "restore" else 1.6):
                self.result = "win" if self.fight.winner is self.fight.player else "lose"

    def _audio_cues(self):
        """Қимыл басталғанда дыбыс: қарсылас мотиві дайындық белгісі қызметін де
атқарады."""
        f, au = self.fight, self.game.audio
        if self._cue_tick == f.frame_no:
            return
        self._cue_tick = f.frame_no
        e, p = f.enemy, f.player
        if e.phase == "startup" and e.frame == 1:
            au.play("whoosh", 0.45)
            motif = au.motif_for(self.enemy_name, e.state)
            if motif:
                au.play(motif, 0.7)
        if p.phase == "startup" and p.frame == 1:
            au.play("whoosh" if p.state == "heavy" else "whoosh_light", 0.4)

    def _update_flashes(self, dt):
        for f in self.flashes:
            f[3] -= dt
        self.flashes = [f for f in self.flashes if f[3] > 0]

    def _update_popups(self, dt):
        for p in self.popups:
            p[3] -= dt
            p[2] -= 40 * dt
        self.popups = [p for p in self.popups if p[3] > 0]

    def _update_trails(self):
        for name, fighter in ((self.fight.player.name, self.fight.player),
                              (self.enemy_name, self.fight.enemy)):
            tr = self.trails[name]
            if fighter.phase == "active":
                pass  # нүктелер сызу кезінде қосылады
            elif tr:
                tr.pop(0)

    # --- эффектілер ---
    def _on_event(self, ev):
        s = self.game.strings
        au = self.game.audio
        if ev.type == "parry":
            au.play("parry", 1.0)
            self.zoom, self.zoom_pt = 0.07, (ev.x, ev.y)
            self.shake.add(0.55)
            self.slow_left = 0.7
            self._burst(ev.x, ev.y, 34, PARRY_COL, 520)
            self.flashes.append([ev.x, ev.y, 420, 0.35, 0.35, (170, 215, 255)])
            self.popups.append([s["fight_parry"], ev.x, ev.y - 90, 0.9, PARRY_COL])
            if ev.defender == "player":
                self.stats["parries"] += 1
            else:   # Көлеңке ойыншыны «оқыды»
                self.popups.append([s["fight_read"], ev.x, ev.y - 140, 1.2, (230, 90, 90)])
        elif ev.type == "block":
            au.play("block", 0.8)
            self.shake.add(0.12)
            self._burst(ev.x, ev.y, 8, (200, 200, 210), 240)
            self.flashes.append([ev.x, ev.y, 180, 0.12, 0.12, (200, 200, 220)])
        elif ev.type == "hit":
            au.play("heavy_hit" if ev.heavy else "hit", 0.9)
            m = self.game.fight_moves[ev.move]
```

```
            self.shake.add(m["shake"])
            self._burst(ev.x, ev.y, 10 + int(m["damage"] * 0.8), SPARK, 380)
            self._shadow_motes(ev.x, ev.y, 12 if ev.heavy else 6)
            self.flashes.append([ev.x, ev.y, 260 if ev.heavy else 170, 0.18, 0.18,
                                 (255, 190, 120)])
            if ev.attacker == "player":
                self.stats["hits_given"] += 1
            else:
                self.stats["hits_taken"] += 1
        elif ev.type == "mem_heal":
            au.play("chime", 0.8)
            self._burst(ev.x, ev.y, 18, (255, 240, 200), 160)
            self.flashes.append([ev.x, ev.y, 300, 0.5, 0.5, (255, 235, 190)])
            self.popups.append([s["fight_heal"], ev.x, ev.y - 70, 0.9, (255, 240, 200)])
        elif ev.type == "mem_focus":
            au.play("tick", 0.6)
            self.flashes.append([ev.x, ev.y, 300, 0.4, 0.4, (170, 215, 255)])
            self.popups.append([s["fight_focus"], ev.x, ev.y - 70, 0.9, PARRY_COL])
        elif ev.type == "restore":
            au.play("chime", 1.0)
            self.slow_left = 1.2
            self.flashes.append([ev.x, ev.y, 520, 1.0, 1.0, (255, 245, 220)])
            self._burst(ev.x, ev.y, 60, (255, 245, 220), 260)
            self.popups.append([s["fight_restore"], ev.x, ev.y - 90, 1.6, (255, 245, 220)])
        elif ev.type == "ko":
            au.play("boom", 0.9)
            self.zoom, self.zoom_pt = 0.11, (ev.x, ev.y)
            self.slow_left = 1.4
            self.shake.add(0.8)
            self._burst(ev.x, ev.y, 50, SPARK, 600)

    def _burst(self, x, y, n, color, speed):
        for _ in range(n):
            a = random.uniform(0, math.tau)
            v = random.uniform(speed * 0.3, speed)
            self.particles.emit(x, y, math.cos(a) * v, math.sin(a) * v - 80,
                                random.uniform(0.25, 0.6), random.choice((2, 3, 4)),
                                color, 900)

    def _shadow_motes(self, x, y, n):
        for _ in range(n):
            self.particles.emit(x + random.uniform(-14, 14), y + random.uniform(-20, 30),
                                random.uniform(-30, 30), random.uniform(-110, -40),
                                random.uniform(0.6, 1.1), 5, (150, 120, 215), -20)

    # --- сызу ---
    def draw(self, surface):
        c = self.canvas
        w, h = self.size
        sc = self.scenery
        c.blit(sc.surface, (0, 0))
        for wnd in sc.windows:
            prev = c.get_clip()
            c.set_clip(wnd)
            self.blizzard.draw(c, brightness=0.9)
            c.set_clip(prev)
            pygame.draw.line(c, (10, 12, 16), (wnd.centerx, wnd.top), (wnd.centerx,
wnd.bottom), 4)
            pygame.draw.line(c, (10, 12, 16), (wnd.left, wnd.centery), (wnd.right,
wnd.centery), 4)

        fp, fe = self.fight.player, self.fight.enemy
        spec_p, spec_e = fp.spec, fe.spec
        ap, ae = tuple(spec_p["accent"]), tuple(spec_e["accent"])
        for fighter, accent in ((fe, ae), (fp, ap)):
            reach = draw_fighter(c, fighter, accent, self.t, part="body")
```

```
            self._push_trail(fighter, reach)
        self._draw_trails(c)
        self.particles.draw(c)

        lights = list(sc.lights)
        lights.append(dict(x=fp.x, y=GROUND - 150, r=330, color=(190, 215, 255),
                           i=0.85, flicker=0, seed=0))
        for x, y, r, life, mx, color in self.flashes:
            lights.append(dict(x=x, y=y, r=r, color=color, i=min(1.0, life / mx * 1.3),
                               flicker=0, seed=0))
        self.lighting.ambient = sc.ambient
        self.lighting.render(c, lights, 0, self.t)
        cor = self.game.corruption
        draw_fighter(c, fe, ae, self.t, part="accent")
        draw_fighter(c, fp, ap, self.t, part="accent", detail=protagonist_detail(cor))
        self._draw_tell(c, ae)
        c.blit(self.vignette, (0, 0))
        if self.time_scale < 0.6:  # баяулау: суық реңк
            tint = pygame.Surface((w, h))
            tint.fill((10, 22, 40))
            tint.set_alpha(70)
            c.blit(tint, (0, 0))

        surface.fill((0, 0, 0))
        ox, oy = self.shake.offset()
        if self.zoom > 0.004:       # зум: фокус нүктесіне қарай үлкейту
            z = 1 + self.zoom
            big = pygame.transform.smoothscale(c, (int(w * z), int(h * z)))
            fx, fy = self.zoom_pt
            surface.blit(big, (ox - int(fx * self.zoom), oy - int(fy * self.zoom)))
        else:
            surface.blit(c, (ox, oy))
        draw_corruption(surface, cor, self.t)
        self._draw_hud(surface)

    def _draw_tell(self, surface, accent):
        """Дайындық қимылының белгісі: сөйлесуде байқалған детальге ғана көрінеді."""
        e = self.fight.enemy
        m = e.move
        if not (self.tells_visible and m and e.phase == "startup"):
            return
        flag = m.get("tell_flag")
        if flag and not self.game.flags.get(flag):
            return
        su = m["startup"]
        left = su - e.frame
        ready = left <= self.fight.player.parry_frames   # L басатын сәт
        col = (255, 255, 255) if ready else accent
        cx, cy = int(e.x), GROUND - 215
        r = int(46 * (1 - e.frame / su)) + 12
        pygame.draw.circle(surface, col, (cx, cy), r, 3)
        mark = self.game.assets.font(30, bold=True).render("!", True, col)
        surface.blit(mark, mark.get_rect(center=(cx, cy)))

    def _push_trail(self, fighter, reach):
        tr = self.trails[fighter.name]
        if fighter.phase == "active" and reach:
            tr.append(reach[1])
            del tr[:-9]

    def _draw_trails(self, surface):
        for tr in self.trails.values():
            for i in range(1, len(tr)):
                k = i / len(tr)
                v = int(235 * k)
                pygame.draw.line(surface, (v, v, min(255, v + 15)), tr[i - 1], tr[i],
```

```
                                max(1, int(7 * k)))

    def _bar(self, surface, rect, frac, color, align_right=False):
        pygame.draw.rect(surface, (14, 14, 20), rect, border_radius=4)
        fill = rect.copy()
        fill.width = int(rect.width * max(0.0, min(1.0, frac)))
        if align_right:
            fill.right = rect.right
        pygame.draw.rect(surface, color, fill, border_radius=4)
        pygame.draw.rect(surface, (70, 70, 84), rect, 2, border_radius=4)

    def _draw_hud(self, surface):
        s, a = self.game.strings, self.game.assets
        w, h = self.size
        fp, fe = self.fight.player, self.fight.enemy
        ap, ae = tuple(fp.spec["accent"]), tuple(fe.spec["accent"])
        font = a.font(20, bold=True)
        cor = self.game.corruption
        surface.blit(font.render(s["who_player"], True, ap), (40, 22))
        self._bar(surface, pygame.Rect(40, 52, 460, 22), fp.hp / fp.max_hp, ap)
        if not hidden(cor, 0.8):          # 3-түн: қарсылас жолағы жоғала бастайды
            name = font.render(s["who_" + self.enemy_name], True, ae)
            surface.blit(name, (w - 40 - name.get_width(), 22))
            self._bar(surface, pygame.Rect(w - 500, 52, 460, 22), fe.hp / fe.max_hp, ae,
True)
        mem = pygame.Rect(40, 82, 300, 12)  # естелік жолағы
        if not hidden(cor, 0.6):
            self._bar(surface, mem, fp.memory / fp.max_memory, (230, 230, 245))
            for i in (1, 2):
                x = mem.x + mem.width * i // 3
                pygame.draw.line(surface, (70, 70, 84), (x, mem.top), (x, mem.bottom), 2)
            surface.blit(a.font(16).render(s["fight_memory"], True, GREY), (348, 79))
        for i in (1, 2, 3):
            if hidden(cor, 0.6):
                break
            ok = self.fight.usable(i)
            pulse = i == 3 and ok and int(self.t * 6) % 2 == 0
            col = (255, 255, 255) if pulse else ((230, 230, 245) if ok else (80, 80, 94))
            surface.blit(a.font(18, bold=ok).render(s[f"slot{i}"], True, col),
                         (40 + (i - 1) * 100, 100))
        pred = getattr(self.fight.controllers.get(self.fight.enemy), "predictor", None)
        if pred and len(pred.log) >= 4 and not hidden(cor, 0.5):     # Көлеңке сені
қаншалықты үйренді
            txt = a.font(18).render(
                s["fight_ai_acc"].format(p=int(pred.accuracy * 100)), True, (150, 120,
215))
            surface.blit(txt, (w - 40 - txt.get_width(), 82))
        if self.fight.can_restore and not self.result:
            hint = a.font(24, bold=True).render(s["fight_restore_hint"], True, (255, 245,
220))
            surface.blit(hint, hint.get_rect(center=(w // 2, 130)))
        for text, x, y, life, color in self.popups:
            img = a.font(30, bold=True).render(text, True, color)
            img.set_alpha(int(255 * min(1.0, life / 0.4)))
            surface.blit(img, img.get_rect(center=(x, y)))
        if not hidden(cor, 0.35):
            ctrl = a.font(18).render(s["fight_controls"], True, GREY)
            surface.blit(ctrl, ctrl.get_rect(center=(w // 2, h - 28)))
        if self.result:
            dim = pygame.Surface((w, h))
            dim.set_alpha(150)
            surface.blit(dim, (0, 0))
            title = (s["fight_win_restore"] if self.fight.outcome == "restore"
                     else s["fight_win"]) if self.result == "win" else s["fight_lose"]
            col = WHITE if self.result == "win" else ACCENT
            t1 = a.font(72, bold=True).render(title, True, col)
```

```
            surface.blit(t1, t1.get_rect(center=(w // 2, h // 2 - 30)))
            hint = s["fight_continue"] if self.result == "win" else s["fight_retry"]
            t2 = a.font(26).render(hint, True, GREY)
            surface.blit(t2, t2.get_rect(center=(w // 2, h // 2 + 40)))
```

### Файл: kolenke/scenes/menu.py

*Басты мәзір: кино-композиция (портреттер, боран, жарқыраған тақырып).*

```
"""Басты мәзір: кино-композиция (портреттер, боран, жарқыраған тақырып)."""
import math

import pygame

from kolenke.core.save import has_save, load_game
from kolenke.core.scene import Scene
from kolenke.render.lighting import make_vignette
from kolenke.render.particles import Blizzard
from kolenke.render.portrait import draw_portrait
from kolenke.scenes.dialogue import blurred_room
from kolenke.scenes.explore import ExploreScene

ACCENT = (200, 60, 60)
WHITE = (232, 232, 238)
GREY = (125, 125, 140)


def _glow_text(font, text, color, glow_color, spread=14):
    """Жарқылы мәтін: қара түсте бұлыңғыр көшірме (ADD-қа арналған) + анық мәтін."""
    base = font.render(text, True, color)
    w, h = base.get_size()
    pad = spread * 3
    layer = pygame.Surface((w + pad * 2, h + pad * 2))     # альфасыз қара фон
    layer.blit(font.render(text, True, glow_color), (pad, pad))
    small = pygame.transform.smoothscale(layer, (layer.get_width() // 8, layer.get_height()
// 8))
    blur = pygame.transform.smoothscale(small, layer.get_size())
    return base, blur, pad


class MenuScene(Scene):
    def __init__(self, game):
        super().__init__(game)
        self.index = 0
        self.t = 0.0
        w, h = game.screen.get_size()
        self.size = (w, h)
        self.bg = blurred_room("darkhan_room", self.size)
        self.snow = Blizzard(w, h, count=220, wind=-220, seed=21)
        self.vignette = make_vignette((w, h))
        self.title = _glow_text(game.assets.font(110, bold=True), game.strings["title"],
                                WHITE, (210, 90, 90))
        self._build_items()

    def on_enter(self):
        self.game.audio.loop("blizzard", 0.32)

    def _build_items(self):
        s = self.game.strings
        self.items = []
        if has_save():
            self.items.append((s["menu_continue"], self._continue))
        self.items += [(s["menu_new_game"], self._new_game), (s["menu_quit"],
self.game.quit)]
        self.index = 0

    def on_resume(self):
```

```
        self._build_items()

    def _continue(self):
        if load_game(self.game):
            self.game.push(ExploreScene(self.game))

    def _new_game(self):
        self.game.new_game_state()
        self.game.start_night(1)
        self.game.push(ExploreScene(self.game))

    def handle_action(self, action):
        if action == "up":
            self.index = (self.index - 1) % len(self.items)
            self.game.audio.play("click", 0.5)
        elif action == "down":
            self.index = (self.index + 1) % len(self.items)
            self.game.audio.play("click", 0.5)
        elif action == "confirm":
            self.game.audio.play("tick", 0.7)
            self.items[self.index][1]()
        elif action == "menu":
            self.game.quit()

    def update(self, dt):
        self.t += dt
        self.snow.update(dt)

    def draw(self, surface):
        s, a = self.game.strings, self.game.assets
        w, h = self.size
        surface.blit(self.bg, (0, 0))
        self.snow.draw(surface, brightness=0.55)
        draw_portrait(surface, "me", "human", 250, h + 60, self.t, True, 0.95)
        draw_portrait(surface, "darkhan", "shadow", w - 250, h + 60, self.t + 1, True,
0.95)
        surface.blit(self.vignette, (0, 0))
        base, blur, pad = self.title
        pulse = 0.75 + 0.25 * math.sin(self.t * 1.7)
        glow = blur.copy()
        glow.fill((int(255 * pulse),) * 3, special_flags=pygame.BLEND_RGB_MULT)
        pos = base.get_rect(center=(w // 2, 190))
        surface.blit(glow, (pos.x - pad, pos.y - pad), special_flags=pygame.BLEND_RGB_ADD)
        surface.blit(base, pos)
        sub = a.font(26).render(s["subtitle"], True, GREY)
        surface.blit(sub, sub.get_rect(center=(w // 2, 275)))
        font = a.font(40)
        y0 = 400
        for i, (label, _) in enumerate(self.items):
            selected = i == self.index
            text = font.render(label, True, WHITE if selected else GREY)
            rect = text.get_rect(center=(w // 2, y0 + i * 70))
            if selected:
                bar = pygame.Surface((rect.width + 120, rect.height + 10), pygame.SRCALPHA)
                pygame.draw.rect(bar, (200, 60, 60, 60), bar.get_rect(), border_radius=10)
                surface.blit(bar, bar.get_rect(center=rect.center))
                pygame.draw.polygon(surface, ACCENT, [(rect.left - 44, rect.centery - 11),
                                                      (rect.left - 44, rect.centery + 11),
                                                      (rect.left - 24, rect.centery)])
            surface.blit(text, rect)
        hint = a.font(20).render(s["menu_hint"], True, GREY)
        surface.blit(hint, hint.get_rect(center=(w // 2, h - 36)))
```

**Файл: kolenke/scenes/night_end.py**

*Түн соңы қорытындысы және финал (4 аяқталу: құтқарылғандар санына байланысты).*

```
"""Түн соңы қорытындысы және финал (4 аяқталу: құтқарылғандар санына байланысты)."""
import json
import math

import pygame

from kolenke.core.assets import ROOT
from kolenke.core.save import delete_save, save_game
from kolenke.core.scene import Scene
from kolenke.core.text import wrap_text
from kolenke.render.particles import Blizzard
from kolenke.render.portrait import draw_portrait

WHITE = (230, 230, 230)
GREY = (120, 120, 120)
ACCENT = (200, 60, 60)
PLAYER_ACCENT = (110, 170, 230)
FRIENDS = ("darkhan", "zhanel", "madina", "bekzat")
NIGHT_FRIENDS = {1: ("darkhan",), 2: ("zhanel", "madina"), 3: ("bekzat",)}
ACCENTS = {"darkhan": (230, 150, 50), "zhanel": (90, 210, 180),
           "madina": (205, 120, 215), "bekzat": (240, 215, 120)}
GROUND = 600


def saved_count(flags):
    return sum(1 for f in FRIENDS if flags.get(f + "_saved"))


def ending_key(count):
    """4 финал: 0 → 0, 1–2 → «жарым таң», 3, 4. Мәтін 1 және 2 үшін бөлек, атау ортақ."""
    return str(max(0, min(4, count)))


def _gradient(size, top, bottom):
    w, h = size
    surf = pygame.Surface(size)
    for y in range(h):
        k = y / (h - 1)
        pygame.draw.line(surf, tuple(int(top[i] + (bottom[i] - top[i]) * k) for i in
range(3)),
                         (0, y), (w, y))
    return surf


def _draw_row(surface, names, flags, t, scale, bottom, cx0, step, label_font, big_font,
              strings, lost_color):
    """Достардың портреттері: қайтқандары — адам, қалғандары — көлеңке; астында атауы."""
    for i, name in enumerate(names):
        saved = bool(flags.get(name + "_saved"))
        cx = cx0 + i * step
        draw_portrait(surface, name, "human" if saved else "shadow", cx, bottom, t, True,
scale)
        who = big_font.render(strings["who_" + name], True, ACCENTS[name] if saved else
lost_color)
        surface.blit(who, who.get_rect(center=(cx, bottom + 22)))
        st = strings["status_saved" if saved else "status_lost"]
        txt = label_font.render(st, True, ACCENTS[name] if saved else lost_color)
        surface.blit(txt, txt.get_rect(center=(cx, bottom + 50)))


class NightEndScene(Scene):
    """«N-түн аяқталды»: кім қайтып келді, кім көлеңкеде қалды. Space — әрі қарай."""
```

```
    def __init__(self, game, night):
        super().__init__(game)
        self.night = night
        self.t = 0.0
        w, h = game.screen.get_size()
        self.bg = _gradient((w, h), (10, 12, 22), (26, 30, 46))
        self.snow = Blizzard(w, h, count=140, wind=-120, seed=11)

    def handle_action(self, action):
        if action not in ("next", "confirm"):
            return
        g = self.game
        if self.night < 3:
            g.start_night(self.night + 1)
            try:
                save_game(g)
            except OSError:      # жазу мүмкін болмаса (мысалы, браузер) ойын жалғаса
береді
                pass
            g.pop()          # ExploreScene.on_resume жаңа түнді бастайды
        else:
            g.pop()
            g.push(EndingScene(g))

    def update(self, dt):
        self.t += dt
        self.game.tick_corruption(dt)
        self.snow.update(dt)

    def draw(self, surface):
        s, a, g = self.game.strings, self.game.assets, self.game
        w, h = surface.get_size()
        surface.blit(self.bg, (0, 0))
        self.snow.draw(surface, brightness=0.5)
        title = a.font(54, bold=True).render(s["night_done_title"].format(n=self.night),
True, WHITE)
        surface.blit(title, title.get_rect(center=(w // 2, 120)))
        names = NIGHT_FRIENDS[self.night]
        step = 360
        x0 = w // 2 - (len(names) - 1) * step // 2
        _draw_row(surface, names, g.flags, self.t, 0.68, GROUND, x0, step,
                  a.font(20), a.font(26, bold=True), s, GREY)
        res = a.font(24).render(s["reserve_label"].format(p=g.reserve), True, GREY)
        surface.blit(res, res.get_rect(center=(w // 2, 190)))
        hint = a.font(22).render(s["night_next_hint"] if self.night < 3 else
s["night_final_hint"],
                                 True, GREY)
        surface.blit(hint, hint.get_rect(center=(w // 2, h - 22)))


class EndingScene(Scene):
    """Финал: сәуле мөлшері құтқарылғандар санына байланысты; Space — келесі жол."""

    def __init__(self, game):
        super().__init__(game)
        with open(ROOT / "kolenke" / "data" / "endings.json", encoding="utf-8") as f:
            data = json.load(f)
        self.count = saved_count(game.flags)
        self.ending = data[ending_key(self.count)]
        self.index = 0
        self.t = 0.0
        self.shown = 0.0
        w, h = game.screen.get_size()
        d = self.ending["dawn"]
        top = tuple(int(a + (b - a) * d) for a, b in zip((8, 10, 18), (250, 170, 110)))
        bot = tuple(int(a + (b - a) * d) for a, b in zip((20, 24, 38), (255, 225, 170)))
```

```
        self.bg = _gradient((w, h), top, bot)
        self.snow = Blizzard(w, h, count=int(160 * (1 - 0.7 * d)), wind=-100, seed=5)

    def on_enter(self):
        self.game.audio.play("chime" if self.count >= 3 else "swell", 0.8)

    @property
    def line(self):
        return self.ending["lines"][self.index]

    def handle_action(self, action):
        if action not in ("next", "confirm"):
            return
        if self.shown < len(self.line):
            self.shown = len(self.line)
        elif self.index + 1 < len(self.ending["lines"]):
            self.index += 1
            self.shown = 0.0
        else:
            delete_save()
            self.game.new_game_state()
            self.game.pop_to_menu()

    def update(self, dt):
        self.t += dt
        self.shown = min(len(self.line), self.shown + 50 * dt)
        self.snow.update(dt)

    def draw(self, surface):
        s, a, g = self.game.strings, self.game.assets, self.game
        w, h = surface.get_size()
        d = self.ending["dawn"]
        surface.blit(self.bg, (0, 0))
        self.snow.draw(surface, brightness=0.8)
        tcol = WHITE if d < 0.6 else (40, 26, 30)
        title = a.font(60, bold=True).render(self.ending["title"], True, tcol)
        surface.blit(title, title.get_rect(center=(w // 2, 110)))
        lost = (110, 110, 124) if d < 0.6 else (90, 66, 70)
        state = "human" if g.corruption < 0.6 else "fading"
        draw_portrait(surface, "me", state, 120, GROUND, self.t, True, 0.56)
        _draw_row(surface, FRIENDS, g.flags, self.t, 0.56, GROUND, 410, 245,
                  a.font(17), a.font(22, bold=True), s, lost)
        font = a.font(30)
        y = 170
        for ln in wrap_text(font, self.line[: int(self.shown)], w - 360):
            surface.blit(font.render(ln, True, tcol), (180, y))
            y += 42
        hint = a.font(20).render(s["ending_hint"], True, GREY if d < 0.6 else (90, 70, 70))
        surface.blit(hint, hint.get_rect(center=(w // 2, h - 18)))
```

### Файл: tools/experiment.py

*Бейімделетін ИИ эксперименті: болжау дәлдігі және бейімделу әсері.*

```
"""Бейімделетін ИИ эксперименті: болжау дәлдігі және бейімделу әсері.

Іске қосу:  python tools/experiment.py --fights 30 --out docs/experiment
Нәтиже: CSV кестелер, сурет (PNG) және консольдегі қорытынды.
Бәрі seed бойынша қайталанады (бот: i, ИИ: 1000+i).
"""
import argparse
import csv
import os
import sys
from pathlib import Path

sys.path.insert(0, str(Path(__file__).resolve().parents[1]))
```

```
os.environ.setdefault("SDL_VIDEODRIVER", "dummy")

from kolenke.combat.ai.adaptive import AdaptiveAI
from kolenke.combat.ai.bots import BOTS
from kolenke.combat.ai.metrics import compare
from kolenke.combat.ai_scripted import ScriptedAI
from kolenke.combat.fight import Fight

MAX_TICKS = 5400   # 90 секунд
METHODS = ("random", "last", "frequency", "markov1", "markov2", "markov3")


ENEMY = {"name": "bekzat"}


def run_fight(bot_cls, ai, bot_seed):
    f = Fight(ENEMY["name"], enemy_controller=ai, player_controller=bot_cls(seed=bot_seed))
    while not f.over and f.frame_no < MAX_TICKS:
        f.tick()
    return f


def mean(xs):
    xs = list(xs)
    return sum(xs) / len(xs) if xs else 0.0


def part_accuracy(n):
    """Әр бот үшін: әдістердің болжау дәлдігі (ИИ-мен жекпе-жек кезінде жиналған
әрекеттер)."""
    rows, curves, in_fight = [], {}, {}
    for name, cls in BOTS.items():
        per_method = {m: [] for m in METHODS}
        live, curve_acc = [], []
        for i in range(n):
            ai = AdaptiveAI(seed=1000 + i, threshold=0.55)
            run_fight(cls, ai, i)
            seq = ai.predictor.history
            if len(seq) < 8:
                continue
            for m, v in compare(seq).items():
                per_method[m].append(v)
            live.append(ai.predictor.accuracy)
            # оқу қисығы: алғашқы 40 әрекет бойынша жылжымалы дәлдік
            hits = [p == a for p, a in ai.predictor.log][:40]
            curve_acc.append(hits)
        rows.append([name] + [round(mean(per_method[m]), 3) for m in METHODS]
                    + [round(mean(live), 3), len(live)])
        in_fight[name] = mean(live)
        L = max((len(h) for h in curve_acc), default=0)
        curves[name] = [mean(h[k] for h in curve_acc if len(h) > k) for k in range(L)]
    return rows, curves


def part_effect(n):
    """ИИ түрлері бойынша: ботқа қарсы нәтиже (скрипт vs бейімделетін)."""
    rows = []
    for name, cls in BOTS.items():
        for label, make in (("scripted", lambda i: ScriptedAI(seed=1000 + i)),
                            ("adaptive", lambda i: AdaptiveAI(seed=1000 + i,
threshold=0.55))):
            lost, dealt, wins, dur, timeouts = [], [], 0, [], 0
            for i in range(n):
                f = run_fight(cls, make(i), i)
                lost.append(1 - f.player.hp / f.player.max_hp)
                dealt.append(1 - f.enemy.hp / f.enemy.max_hp)
```

```
                wins += f.over and f.winner is f.player
                timeouts += not f.over
                dur.append(f.frame_no / 60)
            rows.append([name, label, round(mean(lost), 3), round(mean(dealt), 3),
                         round(wins / n, 3), round(mean(dur), 1), timeouts])
    return rows


def write_csv(path, header, rows):
    with open(path, "w", newline="", encoding="utf-8") as fh:
        w = csv.writer(fh)
        w.writerow(header)
        w.writerows(rows)


def plot(out, acc_rows, curves, effect_rows):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt

    fig, ax = plt.subplots(figsize=(8, 4.2))
    width = 0.13
    for j, m in enumerate(METHODS):
        ax.bar([i + j * width for i in range(len(acc_rows))],
               [r[1 + j] for r in acc_rows], width, label=m)
    ax.set_xticks([i + width * 2.5 for i in range(len(acc_rows))])
    ax.set_xticklabels([r[0] for r in acc_rows])
    ax.axhline(1 / 3, color="gray", ls=":", lw=1)
    ax.set_ylabel("accuracy")
    ax.set_ylim(0, 1.05)
    ax.legend(ncol=3, fontsize=8)
    fig.tight_layout()
    fig.savefig(out / "accuracy.png", dpi=150)
    plt.close(fig)

    fig, ax = plt.subplots(figsize=(8, 4.2))
    for name, ys in curves.items():
        w = 5
        sm = [mean(ys[max(0, k - w + 1):k + 1]) for k in range(len(ys))]
        ax.plot(range(1, len(sm) + 1), sm, label=name)
    ax.axhline(1 / 3, color="gray", ls=":", lw=1)
    ax.set_xlabel("observed player actions")
    ax.set_ylabel("accuracy (moving avg, 5)")
    ax.set_ylim(0, 1.05)
    ax.legend()
    fig.tight_layout()
    fig.savefig(out / "learning_curve.png", dpi=150)
    plt.close(fig)

    names = list(BOTS)
    fig, ax = plt.subplots(figsize=(8, 4.2))
    for j, label in enumerate(("scripted", "adaptive")):
        vals = [next(r[2] for r in effect_rows if r[0] == n and r[1] == label) for n in
names]
        ax.bar([i + j * 0.38 for i in range(len(names))], vals, 0.38, label=label)
    ax.set_xticks([i + 0.19 for i in range(len(names))])
    ax.set_xticklabels(names)
    ax.set_ylabel("player HP lost (fraction)")
    ax.legend()
    fig.tight_layout()
    fig.savefig(out / "effect.png", dpi=150)
    plt.close(fig)


def main():
    ap = argparse.ArgumentParser()
```

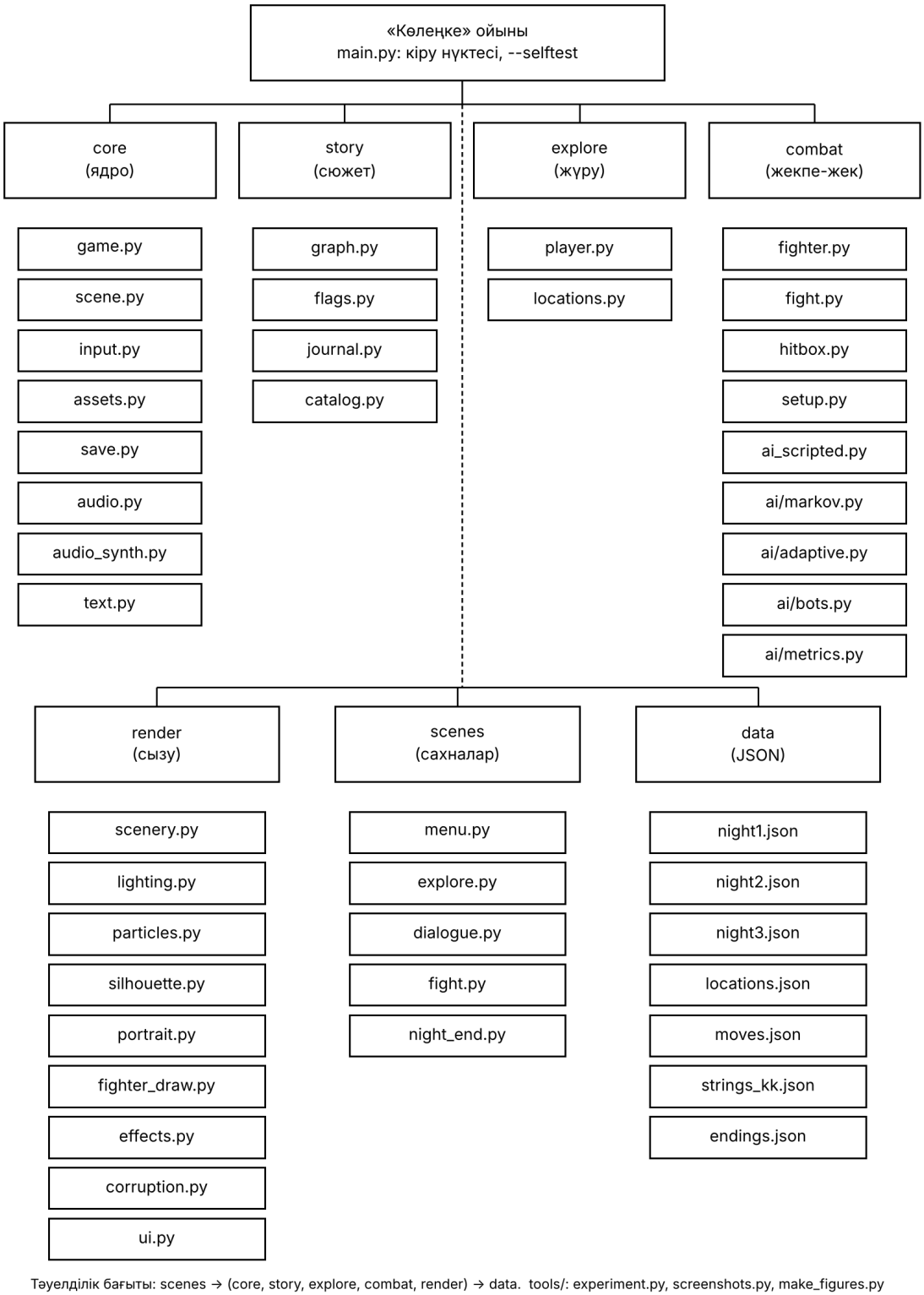
```
    ap.add_argument("--fights", type=int, default=30)
    ap.add_argument("--out", default="docs/experiment")
    ap.add_argument("--enemy", default="bekzat", help="darkhan | zhanel | madina | bekzat")
    args = ap.parse_args()
    ENEMY["name"] = args.enemy
    out = Path(args.out)
    out.mkdir(parents=True, exist_ok=True)

    acc_rows, curves = part_accuracy(args.fights)
    write_csv(out / "accuracy.csv", ["bot", *METHODS, "ai_live_accuracy", "fights_used"],
acc_rows)
    write_csv(out / "learning_curve.csv", ["bot", "step", "accuracy"],
              [[b, k + 1, round(v, 3)] for b, ys in curves.items() for k, v in
enumerate(ys)])
    effect_rows = part_effect(args.fights)
    write_csv(out / "effect.csv", ["bot", "ai", "player_hp_lost", "enemy_hp_lost",
                                   "player_win_rate", "avg_seconds", "timeouts"],
effect_rows)
    try:
        plot(out, acc_rows, curves, effect_rows)
    except ImportError:
        print("matplotlib жоқ: суреттер жасалмады")

    print("\n== Болжау дәлдігі (бот -> әдіс) ==")
    print("bot        " + " ".join(f"{m:>9}" for m in METHODS) + "   ai_live")
    for r in acc_rows:
        print(f"{r[0]:<10} " + " ".join(f"{v:9.3f}" for v in r[1:1 + len(METHODS)]) + f"
{r[-2]:.3f}")
    print("\n== Бейімделу әсері ==")
    print("bot        ai         hp_lost  enemy_lost  win_rate  sec  timeouts")
    for r in effect_rows:
        print(f"{r[0]:<10} {r[1]:<9} {r[2]:8.3f} {r[3]:11.3f} {r[4]:9.3f} {r[5]:5.1f}
{r[6]:5d}")

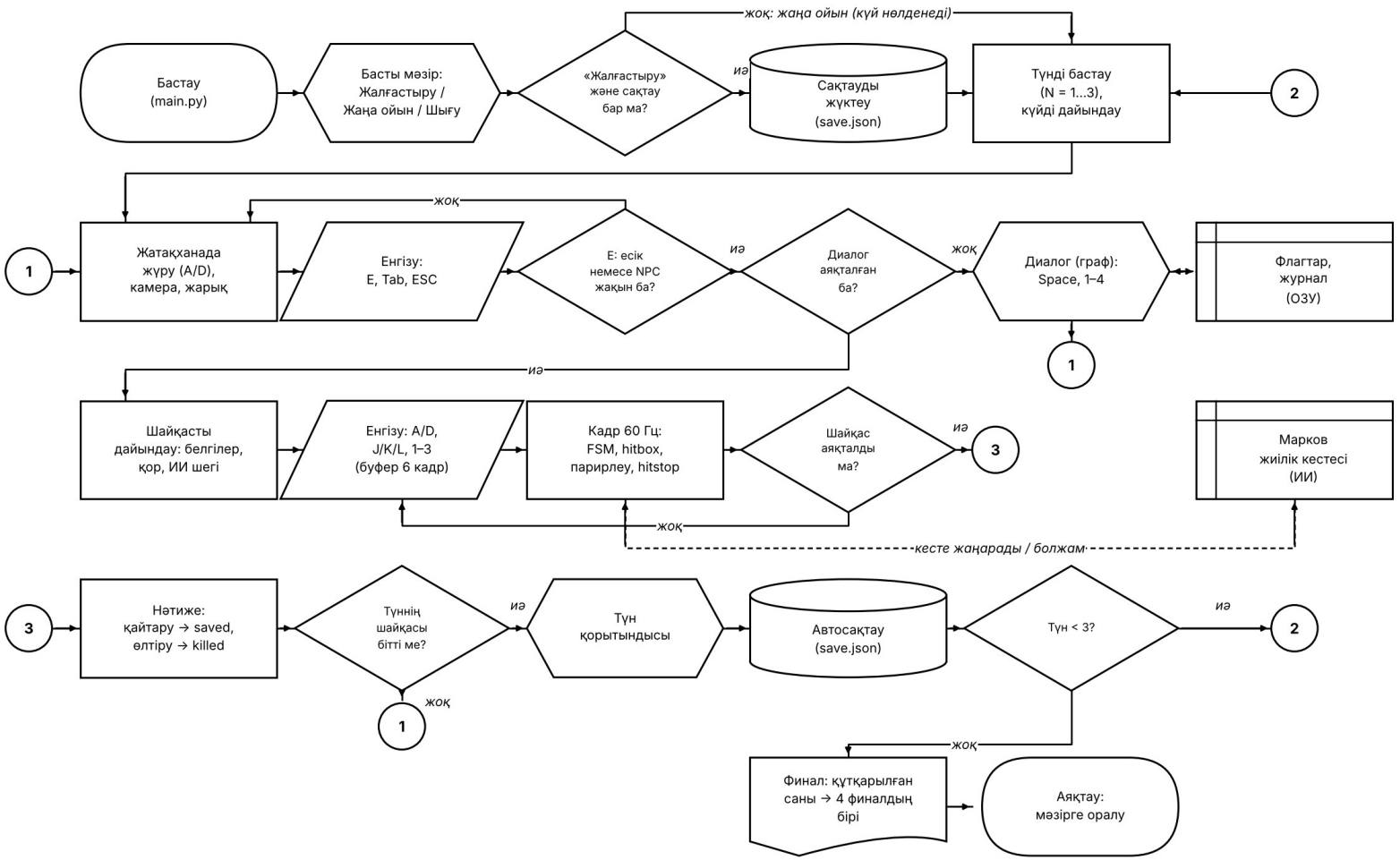

if __name__ == "__main__":
    main()
```

# ҚОСЫМША В. ҚҰРЫЛЫМДЫҚ СЫЗБА




В.1-сурет — «Көлеңке» бағдарламасының құрылымдық сызбасы

# ҚОСЫМША Г. ФУНКЦИОНАЛДЫҚ СЫЗБА




1 — жүру (explore) циклына оралу; 2 — келесі түнді бастау; 3 — шайқас нәтижесіне өту

Г.1-сурет — «Көлеңке» бағдарламасының функционалдық сызбасы